

Universidad Internacional de La Rioja

Escuela Superior de Ingeniería y Tecnología

Máster Universitario en Ingeniería de Software y Sistemas Informáticos

Catástrofe-Logística

**Plataforma Offline-First para la gestión logística de emergencias y la coordinación
de ayuda humanitaria**

Trabajo de fin de estudios presentado por:	Francisco Capote García Álvaro Martín Muñoz
Titulación:	Máster Universitario en Ingeniería de Software y Sistemas Informáticos
Tipo de trabajo:	Desarrollo práctico
Director:	David Escuin Finol
Fecha:	Junio de 2026

Resumen

La gestión logística en situaciones de catástrofe se ve limitada por la pérdida de infraestructuras de comunicación, la fragmentación de la información y la dificultad de coordinar de forma eficaz a los numerosos actores que intervienen en la respuesta. En estos escenarios, las herramientas que dependen de una conexión permanente a Internet o de servidores centralizados resultan poco fiables justo en el momento en que más se necesitan.

El presente trabajo propone el diseño y desarrollo de una plataforma web progresiva (PWA) con arquitectura Offline-First orientada a mejorar la coordinación, la trazabilidad y la eficiencia en la distribución de recursos durante una emergencia. La solución articula cuatro perfiles de usuario —ciudadano, voluntario, puesto de emergencia y coordinador— en torno a un mapa operativo común, e integra el reporte colaborativo de incidencias en las vías, la gestión de un inventario dual por puesto, la trazabilidad de donaciones mediante códigos QR de un solo uso, el cálculo de rutas que evitan los tramos cortados y un registro verificable encadenado de las operaciones críticas.

La metodología seguida es un modelo de desarrollo iterativo e incremental, estructurado en fases de análisis, diseño, implementación, integración y validación, apoyado en prácticas ágiles, control de versiones distribuido y pruebas automatizadas. El sistema se ha construido como un monolito modular en Node.js con Fastify y PostgreSQL en el servidor, y React con TypeScript empaquetado como PWA en el cliente, con persistencia local en IndexedDB y sincronización diferida mediante una cola de operaciones con reintentos e idempotencia.

La evaluación combina pruebas automatizadas (unitarias, de integración y extremo a extremo), una evaluación heurística de usabilidad según los principios de Nielsen, un análisis de amenazas, un banco de pruebas reproducible de enrutamiento y una prueba piloto simulada con ocho participantes que arrojó una puntuación SUS media de 76,6 puntos y una tasa de completitud de tareas del 89,3 %. Los resultados muestran que una arquitectura resiliente combinada con mecanismos de coordinación distribuida proporciona una base sólida y defendible para sostener la operativa básica de una emergencia incluso en condiciones de conectividad degradada.

Palabras clave: gestión logística de emergencias, sistemas colaborativos, arquitectura Offline-First, crowdsourcing, trazabilidad de recursos, Progressive Web App.

Abstract

Logistics management during disasters is hampered by the loss of communication infrastructure, fragmented information, and the difficulty of effectively coordinating the many actors involved in the response. In such scenarios, tools that depend on a permanent Internet connection or on centralized servers become unreliable precisely when they are needed most.

This work proposes the design and development of a Progressive Web App (PWA) with an Offline-First architecture aimed at improving coordination, traceability, and efficiency in the distribution of resources during an emergency. The solution brings together four user profiles —citizen, volunteer, emergency post, and coordinator— around a shared operational map, and integrates collaborative reporting of road incidents, a dual inventory per post, donation traceability through single-use QR codes, route calculation that avoids blocked road segments, and a verifiable hash-chained log of critical operations.

The methodology follows an iterative and incremental development model structured into requirements analysis, design, implementation, integration, and validation phases, supported by agile practices, distributed version control, and automated testing. The system is built as a modular monolith in Node.js with Fastify and PostgreSQL on the server side, and React with TypeScript packaged as a PWA on the client side, with local persistence in IndexedDB and deferred synchronization through an operation queue with retries and idempotency.

Evaluation combines automated tests (unit, integration, and end-to-end), a heuristic usability evaluation following Nielsen principles, a threat analysis, a reproducible routing benchmark, and a simulated pilot study with eight participants that yielded a mean SUS score of 76.6 and a task completion rate of 89.3 %. The results show that a resilient architecture combined with distributed coordination mechanisms provides a solid and defensible basis for sustaining the basic operation of an emergency even under degraded connectivity.

Keywords: emergency logistics management, collaborative systems, Offline-First architecture, crowdsourcing, resource traceability, Progressive Web App.

Agradecimientos

A nuestro director, David Escuin Finol, por su orientación, su criterio técnico y su disponibilidad a lo largo de todo el desarrollo de este trabajo.

A la Universidad Internacional de La Rioja y al profesorado del Máster Universitario en Ingeniería de Software y Sistemas Informáticos, por la formación recibida y por el marco académico en el que se ha desarrollado este Trabajo de Fin de Estudios.

A las personas que participaron de forma voluntaria en la prueba piloto, cuyas observaciones han contribuido de manera directa a mejorar la usabilidad de la plataforma.

Y a nuestras familias y personas cercanas, por el apoyo y la paciencia durante el tiempo dedicado a este proyecto.

Lista de acrónimos y abreviaturas

API	Interfaz de Programación de Aplicaciones
CRDT	Conflict-Free Replicated Data Type
CSP	Content Security Policy
E2E	Pruebas de extremo a extremo (end-to-end)
JWT	JSON Web Token
LOPDGDD	Ley Orgánica de Protección de Datos y Garantía de Derechos Digitales
MVP	Producto Mínimo Viable (Minimum Viable Product)
ORM	Mapeo Objeto-Relacional (Object-Relational Mapping)
OSM	OpenStreetMap
PWA	Aplicación Web Progresiva (Progressive Web App)
QR	Quick Response (código de respuesta rápida)
RBAC	Control de Acceso Basado en Roles (Role-Based Access Control)
RF / RNF	Requisito Funcional / Requisito No Funcional
RGPD	Reglamento General de Protección de Datos
SUS	System Usability Scale
TFE / TFM	Trabajo de Fin de Estudios / de Máster
TSA	Autoridad de Sellado Temporal (Time-Stamping Authority)
WCAG	Web Content Accessibility Guidelines

Índice de contenidos

1. Introducción.....	18
1.1. Justificación.....	19
1.2. Planteamiento del problema.....	19
1.3. Estructura del trabajo.....	20
1.4. Aportaciones del trabajo.....	21
2. Contexto y estado del arte.....	23
2.1. Contexto del problema.....	23
2.2. Estado del arte.....	24
Antecedentes académicos y tecnológicos.....	24
Plataformas y soluciones existentes.....	25
Comparativa de herramientas.....	26
2.3. Marco normativo y consideraciones éticas.....	27
2.4. Conclusiones del análisis.....	28
3. Objetivos y metodología de trabajo.....	29
3.1. Objetivo general.....	29
3.2. Objetivos específicos.....	29
3.3. Metodología de trabajo.....	30
Fases del desarrollo.....	31
Herramientas y técnicas.....	31
3.4. Planificación de incrementos.....	32
3.5. Gestión de riesgos.....	32
3.6. Aseguramiento de la calidad.....	33
4. Desarrollo específico de la contribución.....	34
4.1. Desarrollo práctico.....	34
4.1.1. Identificación de requisitos.....	34
4.1.1.1. Trazabilidad entre requisitos, implementación y pruebas.....	35
4.1.1.2. Actores del sistema.....	36
4.1.1.2.1. Ciudadano.....	38

4.1.1.2.2. Voluntario.....	38
4.1.1.2.3. Puesto de emergencia.....	38
4.1.1.2.4. Coordinador.....	39
4.1.2. Descripción del sistema software desarrollado.....	39
4.1.2.1. Stack tecnológico.....	39
4.1.2.2. Arquitectura general del sistema.....	40
4.1.2.3. Frontend y experiencia de usuario.....	42
4.1.2.4. Funcionamiento Offline-First.....	43
4.1.2.4.1. Cacheo y persistencia local.....	44
4.1.2.4.2. Cola de operaciones, reintentos e idempotencia.....	44
4.1.2.5. Backend y gestión de datos.....	46
4.1.2.6. Seguridad, autenticación y cumplimiento del RGPD.....	47
4.1.2.7. Gestión de incidencias y enrutamiento.....	49
4.1.2.7.1. Enrutamiento local-first consciente de las vías cortadas.....	49
4.1.2.8. Gestión de inventario y donaciones.....	50
4.1.2.9. Modelo de datos.....	51
4.1.2.10. API REST y eventos en tiempo real.....	54
4.1.2.11. Registro verificable encadenado.....	55
4.1.2.12. Despliegue y entorno de ejecución.....	56
4.1.2.13. Casos de uso detallados.....	57
4.1.2.14. Decisiones de arquitectura significativas.....	58
4.1.2.15. Rendimiento y optimización del cliente.....	59
4.1.2.16. Modelado complementario: actividad y clases.....	59
4.1.3. Evaluación.....	62
4.1.3.1. Plan de pruebas.....	62
4.1.3.2. Resultados de las pruebas automatizadas.....	63
4.1.3.3. Escenarios de prueba representativos.....	64
4.1.3.4. Evaluación heurística de usabilidad.....	65
4.1.3.5. Prueba piloto simulada.....	65
4.1.3.6. Banco de pruebas del subsistema de enrutamiento.....	67

4.1.3.7. Análisis de amenazas (STRIDE).....	69
4.1.3.8. Resultados y discusión.....	70
5. Conclusiones y trabajo futuro.....	72
5.1. Conclusiones.....	72
5.2. Lecciones aprendidas.....	73
Monolito modular frente a microservicios.....	73
Complejidad del enfoque Offline-First.....	73
Valor de las pruebas extremo a extremo.....	73
Coordinación en un equipo de dos personas.....	74
5.3. Trabajo futuro.....	74
Validación en escenario real.....	74
Mejoras en sincronización y resiliencia.....	74
Integración con sistemas de emergencias oficiales.....	74
Inteligencia artificial y observabilidad.....	74
6. Bibliografía.....	76
Anexo A. Artículo científico.....	78
Resumen.....	78
I. Introducción.....	78
II. Estado del arte.....	79
III. Objetivos y metodología.....	79
IV. Contribución.....	79
V. Resultados.....	80
VI. Discusión.....	80
VII. Conclusiones.....	80
Referencias.....	80
Anexo B. Especificación operativa por rol y matriz de permisos.....	81
B.1. Capacidades por rol.....	81
B.2. Matriz de permisos.....	81
B.3. Flujos operativos críticos.....	82
Anexo C. Guía de validación, despliegue y defensa.....	83

C.1. Lista de comprobación de entrega.....	83
C.2. Riesgos residuales y mitigaciones futuras.....	83
C.3. Guion de demostración recomendado.....	84
Anexo D. Referencia de la API REST.....	85
Anexo E. Diccionario de datos.....	87
E.1. Enumerados.....	87
E.2. Entidad Usuario.....	87
E.3. Entidad PuestoEmergencia.....	88
E.4. Entidades de logística (Producto, Inventario, Donacion).....	88
E.5. Entidades del entorno (IncidenciaVia, Voluntario).....	89
E.6. Entidades de trazabilidad y resiliencia.....	89
Anexo F. Glosario de términos y acrónimos.....	90
Anexo G. Especificación detallada de los casos de prueba.....	91
G.1. Caso E-01: Reporte de incidencia y propagación en tiempo real.....	91
G.2. Caso E-02: Creación offline y sincronización posterior.....	91
G.3. Caso E-03: Entrega de productos mediante QR.....	91
G.4. Caso E-04: Cálculo de ruta evitando incidencias activas.....	92
G.5. Caso E-05: Aprobación de una solicitud de puesto.....	92
G.6. Caso E-06: Control de acceso por rol.....	92
Anexo H. Cobertura de los objetivos específicos.....	93
Anexo I. Estructura del repositorio y guía de lectura del código.....	94
Anexo J. Capturas de la interfaz de usuario.....	96
J.1. Acceso y gestión de identidad.....	96
J.2. Rol Ciudadano.....	99
J.3. Rol Voluntario.....	103
J.4. Rol Puesto de emergencia.....	106
J.5. Rol Coordinador.....	109

Índice de figuras

Figura 1. Diagrama de casos de uso principales por actor.....	37
Figura 2. Diagrama de casos de uso detallado por actor.....	37
Figura 3. Arquitectura general de la plataforma Catástrofe-Logística.....	40
Figura 4. Diagrama de componentes principales del sistema.....	41
Figura 5. Flujo general de sincronización Offline-First.....	43
Figura 6. Diagrama de secuencia del proceso de sincronización Offline-First.....	45
Figura 7. Diagrama de estados de una operación offline.....	45
Figura 8. Diagrama de secuencia del proceso de autenticación con JWT.....	47
Figura 9. Diagrama de estados de una incidencia de vía.....	49
Figura 10. Diagrama de secuencia de validación de una entrega mediante QR.....	51
Figura 11. Diagrama de estados de una donación.....	51
Figura 12. Modelo entidad-relación simplificado del sistema.....	52
Figura 13. Modelo entidad-relación ampliado de la base de datos: las 19 entidades del esquema y sus relaciones.....	53
Figura 14. Flujo de una petición autenticada y difusión del evento resultante.....	55
Figura 15. Diagrama de despliegue del entorno de ejecución.....	57
Figura 16. Diagrama de actividad del flujo de una donación.....	60
Figura 17. Diagrama de clases del dominio (simplificado).....	61
Figura 18. Diagrama de comunicación (colaboración) de la confirmación de entrega mediante QR.....	62
Figura 19. Tasa de éxito por estrategia de enrutamiento, según el número de vías cortadas.....	68
Figura 20. Gap de optimalidad de la heurística frente a la ruta óptima.....	68
Figura 21. Coste algorítmico de A* frente a Dijkstra sobre el grafo local.....	69
Figura 22. Coste de red de la heurística basada en OSRM.....	69
Figura 23. Pantalla de inicio de sesión, con acceso mediante correo electrónico o DNI/NIE.....	96
Figura 24. Registro de una cuenta de ciudadano o voluntario.....	97
Figura 25. Solicitud de alta de un puesto de emergencia.....	97
Figura 26. Recuperación de contraseña mediante DNI/NIE y código de recuperación, sin dependencia de correo.....	98
Figura 27. Selección del rol activo para usuarios con varios perfiles.....	98
Figura 28. Verificación pública de un código QR de entrega.....	99
Figura 29. Pantalla de inicio del rol ciudadano.....	99
Figura 30. Panel principal del ciudadano con accesos a las funciones disponibles.....	100
Figura 31. Mapa operativo con puestos e incidencias sobre Leaflet y OpenStreetMap.....	100
Figura 32. Reporte de una incidencia en la vía con categoría y ubicación.....	101
Figura 33. Búsqueda de productos disponibles en los puestos cercanos.....	101
Figura 34. Resultado de la búsqueda de un producto (agua) ordenado por cercanía.....	102
Figura 35. Pantalla de perfil de usuario.....	102
Figura 36. Panel principal del voluntario.....	103
Figura 37. Acción de donación a un puesto de emergencia.....	103

Figura 38. Acción de atención de una incidencia.....	104
Figura 39. Acción de solicitud de participación en un puesto.....	104
Figura 40. Flujo de donación (1): selección de los productos a donar.....	105
Figura 41. Flujo de donación (2): especificación de las cantidades.....	105
Figura 42. Flujo de donación (3): cálculo de la ruta óptima evitando las vías cortadas.....	106
Figura 43. Panel del puesto de emergencia con el inventario dual (disponible y necesario).....	106
Figura 44. Alta de un nuevo producto en el inventario del puesto.....	107
Figura 45. Gestión de los voluntarios asignados al puesto.....	107
Figura 46. Historial de movimientos del inventario.....	108
Figura 47. Generación del código QR de entrega para la recepción de una donación.....	108
Figura 48. Panel de supervisión global del coordinador.....	109
Figura 49. Pestaña de resumen operativo de la red de puestos.....	109
Figura 50. Pestaña de gestión y aprobación de puestos.....	110
Figura 51. Pestaña de gestión de incidencias.....	110
Figura 52. Pestaña de gestión de usuarios.....	111

Índice de tablas

Tabla 1. Organización del trabajo en grupo.....	15
Tabla 2. Comparativa de herramientas existentes para la gestión de emergencias.....	27
Tabla 3. Planificación de incrementos del desarrollo, reconstruida a partir del historial de commits y pull requests del repositorio.....	32
Tabla 4. Registro de riesgos del proyecto, con probabilidad e impacto estimados y mitigación adoptada.	33
Tabla 5. Catálogo completo de requisitos del sistema (26 funcionales y 11 no funcionales).....	34
Tabla 6. Matriz de trazabilidad entre requisitos, implementación real en el repositorio y pruebas asociadas.....	35
Tabla 7. Stack tecnológico de la plataforma, por capa, con su función y justificación.....	39
Tabla 8. Componentes de la arquitectura física y su responsabilidad.....	41
Tabla 9. Medidas de seguridad en el diseño y su correspondencia con el RGPD.....	48
Tabla 10. Entidades del modelo de datos, con su responsabilidad y relaciones principales.....	54
Tabla 11. Casos de uso principales del sistema, con actor, flujo y requisitos asociados.....	58
Tabla 12. Registro de decisiones de arquitectura (ADR) significativas.....	58
Tabla 13. Resultados de las suites de pruebas automatizadas (Vitest, Playwright y Lighthouse).....	63
Tabla 14. Cobertura de las pruebas automatizadas por área funcional.....	63
Tabla 15. Resultados de los escenarios de prueba representativos (E-01 a E-06).....	64
Tabla 16. Evaluación heurística de usabilidad según los principios de Nielsen (1994).....	65
Tabla 17. Perfil de los participantes en la prueba piloto.....	65
Tabla 18. Resultados operativos por participante en la prueba piloto.....	66
Tabla 19. Puntuaciones SUS por participante y comentario cualitativo destacado.....	67
Tabla 20. Resumen del banco de pruebas reproducible de enrutamiento, por número de vías cortadas..	67
Tabla 21. Análisis de amenazas según el modelo STRIDE y contramedidas implementadas.....	70
Tabla 22. Matriz de permisos por funcionalidad y rol, coherente con el control de acceso implementado.	81
Tabla 23. Flujos operativos críticos de la plataforma y riesgo que controla cada uno.....	82
Tabla 24. Lista de comprobación de entrega y coherencia entre memoria y código.....	83
Tabla 25. Riesgos residuales del MVP, impacto y líneas de mitigación futura.....	83
Tabla 26. Guion de demostración recomendado para la defensa del trabajo.....	84
Tabla 27. Superficie real de la API REST del backend, por router.....	85
Tabla 28. Tipos enumerados del modelo de datos.....	87
Tabla 29. Diccionario de datos de la entidad Usuario (tabla usuarios).....	87
Tabla 30. Diccionario de datos de la entidad PuestoEmergencia (tabla puestos_emergencia).....	88
Tabla 31. Diccionario de datos de la entidad Producto (tabla productos).....	88
Tabla 32. Diccionario de datos de la entidad Inventario (tabla inventario).....	88
Tabla 33. Diccionario de datos de la entidad Donacion (tabla donaciones).....	88
Tabla 34. Diccionario de datos de la entidad IncidenciaVia (tabla incidencias_via).....	89
Tabla 35. Diccionario de datos de la entidad Voluntario (tabla voluntarios).....	89
Tabla 36. Diccionario de datos de la entidad ChainEvent (tabla chain_events).....	89

Tabla 37. Diccionario de datos de las entidades AuditLog, IdempotencyRecord y QrConsumption.....	89
Tabla 38. Glosario de términos y acrónimos.....	90
Tabla 39. Especificación del caso de prueba E-01.....	91
Tabla 40. Especificación del caso de prueba E-02.....	91
Tabla 41. Especificación del caso de prueba E-03.....	91
Tabla 42. Especificación del caso de prueba E-04.....	92
Tabla 43. Especificación del caso de prueba E-05.....	92
Tabla 44. Especificación del caso de prueba E-06.....	92
Tabla 45. Cobertura de los objetivos específicos, grado de consecución y evidencia.....	93
Tabla 46. Estructura del repositorio: zonas principales, responsabilidad y valor para el trabajo.....	94

Organización del trabajo en grupo

Este Trabajo de Fin de Estudios ha sido desarrollado en grupo por Francisco Capote García y Álvaro Martín Muñoz. En este apartado se detalla el reparto del trabajo entre ambos integrantes, su justificación desde el punto de vista de la adquisición de conocimientos del Máster y los mecanismos de coordinación empleados durante el desarrollo.

Partes que aborda el TFE

El trabajo se ha estructurado de modo que cada integrante asuma partes que, individualmente, tienen entidad suficiente y son evaluables de forma diferenciada, manteniendo a la vez una visión común del sistema:

- **Francisco Capote García** — Diseño e implementación del front-end (PWA en React y TypeScript): navegación por rol, cartografía, capa offline (Service Worker, IndexedDB y cola de sincronización), enrutamiento local-first y experiencia de usuario; y la evaluación de usabilidad de la herramienta (evaluación heurística y prueba piloto con la escala SUS).
- **Álvaro Martín Muñoz** — Diseño e implementación del back-end (Node.js y Fastify): modelo de datos y persistencia con Prisma sobre PostgreSQL, API REST por dominios, autenticación y control de acceso basado en roles (RBAC), comunicación en tiempo real y registro verificable encadenado; y el análisis de seguridad de la aplicación.
- **Trabajo conjunto** — Identificación de requisitos, integración del front-end y el back-end, despliegue (Docker Compose y Render) y redacción de los apartados transversales de la memoria (introducción, estado del arte, objetivos, conclusiones y artículo).

Distribución y estructura de la memoria

La Tabla 1 refleja el reparto de responsabilidades en la elaboración de la memoria y en las partes del trabajo, garantizando que la contribución de cada integrante sea identificable y evaluable.

Tabla 1. Organización del trabajo en grupo.

Apartado o parte del TFE	Responsable(s)
Introducción, contexto y estado del arte, objetivos y metodología	Francisco Capote, Álvaro Martín
Front-end: PWA, mapa, capa offline y enrutamiento local-first	Francisco Capote
Back-end: API REST, modelo de datos, tiempo real y registro encadenado	Álvaro Martín
Integración del sistema y despliegue (Docker Compose y Render)	Francisco Capote, Álvaro Martín
Análisis de seguridad de la aplicación	Álvaro Martín
Evaluación de usabilidad (evaluación heurística y prueba piloto SUS)	Francisco Capote
Conclusiones, trabajo futuro y artículo científico	Francisco Capote, Álvaro Martín

Fuente: *Elaboración propia.*

Objetivo del TFE desde el punto de vista de la adquisición de conocimientos

Desde la perspectiva de la adquisición de conocimientos, este trabajo integra y pone en práctica varias materias cursadas en el Máster: la ingeniería de requisitos y el modelado con UML (casos de uso, diagramas de secuencia, estados, actividad y clases); el diseño e implementación de aplicaciones web y de arquitecturas resilientes; el modelado y la gestión de bases de datos; la ciberseguridad y la protección de datos (control de acceso, análisis de amenazas y cumplimiento del RGPD); la interacción persona-ordenador y la evaluación de la usabilidad; y la gestión y las metodologías de proyectos software. Cada integrante ha profundizado, además, en su área de especialización —desarrollo frontend y experiencia de usuario, en un caso, y desarrollo backend y seguridad, en el otro—, lo que ha permitido afianzar competencias complementarias dentro de un mismo proyecto.

Mecanismos de coordinación empleados

La coordinación se ha apoyado en prácticas propias del trabajo en equipos de desarrollo software. Se empleó control de versiones distribuido con Git, con un flujo de ramas por funcionalidad e integración mediante solicitudes de incorporación (pull requests) revisadas por el otro integrante. La planificación y la priorización del trabajo se organizaron en un tablero de tareas de estilo Kanban, que permitió visualizar el flujo de trabajo y limitar las tareas en curso. Se mantuvieron reuniones periódicas de sincronización para acordar los contratos de la API y resolver las dependencias entre el front-end y el back-end, y se utilizó un entorno reproducible con Docker Compose para asegurar que ambos integrantes trabajaban sobre la

misma base. Esta combinación de herramientas y prácticas permitió el avance en paralelo de las dos líneas de trabajo, minimizando la fricción en los puntos de integración.

1. Introducción

Las catástrofes de gran escala —inundaciones, terremotos, incendios forestales o episodios meteorológicos extremos— comparten un patrón recurrente: en las primeras horas, cuando la respuesta es más determinante para salvar vidas y reducir daños, la información sobre el terreno es escasa, contradictoria y se degrada con rapidez. Las infraestructuras de telecomunicaciones se saturan o se caen, las vías de acceso quedan cortadas y la ayuda —tanto la institucional como la espontánea de la ciudadanía— se moviliza sin una visión compartida de qué se necesita, dónde y con qué prioridad. El resultado es una paradoja conocida en la logística humanitaria: coexisten la escasez de recursos en unos puntos y el exceso o la duplicación en otros.

Este Trabajo de Fin de Estudios aborda ese problema desde la ingeniería del software. Se propone el diseño y desarrollo de Catástrofe-Logística, una plataforma web progresiva (PWA) concebida para operar incluso cuando la conectividad es intermitente o inexistente, y para coordinar a los distintos actores que intervienen en una emergencia en torno a un mismo mapa operativo. La plataforma no pretende sustituir a los organismos oficiales de protección civil, sino ofrecer una capa tecnológica ligera, resiliente y accesible que complemente su labor y canalice de forma ordenada la participación ciudadana.

La elección de una arquitectura Offline-First no es un detalle de implementación, sino el eje conceptual del trabajo. En un escenario donde la red es el primer recurso que falla, condicionar la utilidad de la herramienta a la existencia de conexión equivale a inutilizarla en el momento crítico. Por ello, el sistema se diseña para que las funciones esenciales —consultar el mapa, registrar una incidencia, localizar un puesto o preparar una entrega— funcionen en local y se sincronicen de forma diferida y fiable cuando la conectividad se recupera.

El desarrollo se ha llevado a cabo en equipo de dos personas mediante un modelo iterativo e incremental, con una clara separación de responsabilidades entre la capa cliente y la capa servidor, y con apoyo continuo en pruebas automatizadas y en un entorno reproducible. A lo largo de la memoria se detallan las decisiones de diseño, su justificación y la evaluación realizada para contrastar el grado de cumplimiento de los objetivos.

1.1. Justificación

La motivación del trabajo parte de una carencia bien documentada en la literatura de logística humanitaria: la eficacia de la respuesta a una emergencia depende, en gran medida, de la visibilidad de la situación y de la capacidad de coordinar a los actores implicados (Van Wassenhove, 2006; Comfort, 2007). Cuando no existe un canal común de información, cada organización y cada voluntario opera con su propia imagen parcial de la realidad, lo que genera ineficiencias, solapamientos y decisiones tardías.

A esta carencia organizativa se suma una limitación tecnológica. La mayoría de las soluciones disponibles asumen un modelo de conectividad permanente y de servidor centralizado que choca con la realidad física de una catástrofe, donde la red móvil colapsa y la energía es un bien escaso. Las herramientas que sí contemplan el trabajo sin conexión suelen restringirlo a la mera consulta de mapas, sin permitir registrar operaciones ni coordinar recursos en ese estado.

Por otra parte, el auge del crowdsourcing en la gestión de crisis —popularizado por plataformas como Ushahidi (Okolloh, 2009) y consolidado por la corriente de los digital humanitarians (Meier, 2015)— ha demostrado el valor de incorporar a la ciudadanía como sensor distribuido. Sin embargo, esa información colaborativa rara vez se integra con la gestión física de recursos: saber que una calle está cortada o que un punto necesita agua aporta poco si no se conecta con quién puede transportar esa agua evitando esa calle.

Catástrofe-Logística se justifica precisamente en la intersección de estas tres carencias: la falta de un canal común de coordinación, la fragilidad de las soluciones dependientes de la red y la desconexión entre la información colaborativa y la logística física. El trabajo aporta valor al integrar, en una única plataforma ligera y de código abierto, el reporte colaborativo del estado del entorno, la gestión de inventario en puntos de distribución, la trazabilidad verificable de las entregas y el enrutamiento consciente de las vías cortadas, todo ello sobre una base resiliente que no se desploma cuando desaparece la conexión.

1.2. Planteamiento del problema

El problema concreto que se aborda puede formularse así: ¿cómo coordinar de forma eficiente y trazable la logística de ayuda humanitaria en un entorno de conectividad

degradada, integrando tanto a las organizaciones como a la ciudadanía, y manteniendo la fiabilidad de los datos pese a la operación intermitente?

De esta pregunta se derivan varios subproblemas técnicos que el sistema debe resolver de forma conjunta. En primer lugar, la continuidad operativa sin red: el cliente debe poder generar operaciones (reportes, solicitudes, entregas) en local y reproducirlas más tarde sin pérdidas ni duplicados. En segundo lugar, la consistencia de los datos tras la reconexión, lo que exige mecanismos de idempotencia y una política explícita de resolución de conflictos. En tercer lugar, la trazabilidad de las operaciones críticas, de modo que una entrega o un movimiento de inventario quede registrado de forma verificable y resistente a manipulaciones. En cuarto lugar, la coordinación entre roles heterogéneos con permisos diferenciados. Y, por último, la accesibilidad y el rendimiento en dispositivos modestos y redes lentas, condición indispensable para que la herramienta sea realmente utilizable en campo.

El alcance del trabajo se acota deliberadamente al de un producto mínimo viable (MVP) de calidad profesional: una prueba técnica integrada que demuestra la viabilidad del enfoque y ejercita de extremo a extremo los flujos principales, pero que no incluye la integración con sistemas oficiales de emergencia ni la validación en una catástrofe real, líneas que se reservan como trabajo futuro. El planteamiento se enmarca en los ámbitos de la ingeniería del software, el desarrollo web y la ciberseguridad.

1.3. Estructura del trabajo

La memoria se organiza en seis capítulos. Tras esta introducción, el capítulo 2 sitúa el contexto del problema y revisa el estado del arte, comparando las soluciones existentes e identificando los principios de diseño que la literatura recomienda. El capítulo 3 define el objetivo general y los objetivos específicos, y detalla la metodología de trabajo, la planificación por incrementos y la gestión de riesgos.

El capítulo 4 constituye el núcleo técnico del trabajo: presenta el catálogo de requisitos, los actores del sistema, el stack tecnológico, la arquitectura y la implementación de cada subsistema y, en su sección final, la evaluación del sistema, que combina pruebas automatizadas, evaluación heurística, un banco de pruebas de enrutamiento, un análisis de amenazas y una prueba piloto simulada con métricas de usabilidad. El capítulo 5 recoge las

conclusiones —redactadas en función de los objetivos—, las lecciones aprendidas y las líneas de trabajo futuro. El capítulo 6 reúne la bibliografía.

Cierran el documento las referencias bibliográficas y un conjunto de anexos que aportan material complementario sin duplicar el cuerpo principal: el artículo científico que sintetiza el trabajo, la especificación operativa por rol con la matriz de permisos, la guía de validación y despliegue, la referencia de la API, el diccionario de datos y el glosario.

1.4. Aportaciones del trabajo

Más allá del desarrollo de una aplicación funcional, este trabajo realiza varias aportaciones que conviene explicitar, pues constituyen su valor diferencial y se retoman en la evaluación y en las conclusiones:

- **Una arquitectura Offline-First completa y verificada**, que no se limita al cacheo de recursos, sino que incorpora una cola de operaciones de escritura con reintentos, idempotencia de extremo a extremo y resolución de identificadores locales, garantizando que ninguna acción se pierda ni se duplique al sincronizar.
- **La integración del crowdsourcing con la logística física**, articulando en una misma plataforma el reporte colaborativo del estado de las vías, la gestión de inventario por puesto y la trazabilidad de las entregas, una combinación que las soluciones existentes solo cubren de forma parcial.
- **Un enrutamiento local-first consciente de las vías cortadas**, con penalización blanda y un banco de pruebas reproducible que cuantifica su ventaja frente a la alternativa puramente online en condiciones de múltiples cortes.
- **Un registro verificable encadenado (hash-chain)** de las operaciones críticas, que aporta transparencia y trazabilidad sobre las entregas sin el coste ni la complejidad de una cadena de bloques distribuida.
- **Una validación multifacética y honesta**, que combina pruebas automatizadas, evaluación heurística, análisis de amenazas, banco de pruebas de enrutamiento y una prueba piloto simulada con métricas SUS, delimitando con rigor el alcance del MVP.

En conjunto, la principal aportación no es ninguna de estas piezas por separado, sino su integración coherente en una plataforma ligera, abierta y resiliente, concebida específicamente para las condiciones adversas de una emergencia.

2. Contexto y estado del arte

Este capítulo describe el contexto en el que se enmarca la gestión logística de emergencias, revisa los antecedentes académicos y las plataformas reales más relevantes, compara sus capacidades y limitaciones, sitúa el marco normativo aplicable y destila los principios de diseño que han orientado las decisiones técnicas del trabajo.

2.1. Contexto del problema

La logística humanitaria se distingue de la logística comercial por operar en condiciones de incertidumbre extrema, con cadenas de suministro improvisadas, demanda impredecible y un horizonte temporal comprimido (Kovács y Spens, 2007). Van Wassenhove (2006) describe esta disciplina como una «cadena de suministro en marcha forzada», en la que la planificación cede el paso a la capacidad de adaptación y la respuesta debe construirse sobre infraestructuras dañadas.

En este escenario, el cuello de botella no es únicamente la escasez de recursos, sino la falta de información actualizada y compartida. Comfort (2007) identifica cuatro capacidades críticas en la gestión de crisis —cognición, comunicación, coordinación y control— y subraya que el fallo en la comunicación temprana se propaga al resto, degradando la coordinación global. La consecuencia práctica es bien conocida: recursos que no llegan, rutas que se intentan una y otra vez pese a estar cortadas, y puntos de distribución saturados junto a otros desabastecidos.

Un segundo rasgo del contexto es la heterogeneidad de los actores. En una emergencia conviven organismos oficiales, ONG, empresas y, de forma creciente, una ciudadanía que se autoorganiza y que aporta tanto necesidades como capacidad de ayuda. Esta participación espontánea es un activo valioso, pero sin un canal estructurado puede generar ruido, congestión e incluso interferir con la respuesta oficial. Diseñar para esta diversidad de perfiles, con permisos y vistas diferenciadas, es por tanto un requisito de partida.

Por último, el contexto impone una restricción tecnológica ineludible: la conectividad es el primer recurso que falla y el último en restablecerse. Cualquier herramienta concebida para este dominio debe asumir la intermitencia de la red como estado normal de operación, y no como una excepción a gestionar.

A estos rasgos se añade una dimensión temporal que condiciona todo el diseño: el valor de la información en una emergencia decae rápidamente. Un dato sobre una vía cortada o sobre la necesidad de un producto es extremadamente útil en los primeros minutos y puede ser irrelevante, o incluso contraproducente, horas después. Esta caducidad de la información exige mecanismos de actualización colaborativa y de difusión inmediata, y penaliza las arquitecturas que introducen latencias o que dependen de procesos manuales de consolidación. El sistema responde a esta exigencia con la combinación de validación colaborativa de incidencias y difusión en tiempo real que se describe en el capítulo 4.

2.2. Estado del arte

Antecedentes académicos y tecnológicos

La investigación en sistemas de información para emergencias ha evolucionado desde los primeros sistemas de gestión de desastres hacia plataformas que incorporan información geográfica voluntaria y, más recientemente, técnicas de inteligencia artificial. Goodchild y Glennon (2010) acuñaron el concepto de crowdsourcing de información geográfica para la respuesta a desastres, mostrando cómo las contribuciones ciudadanas pueden complementar las fuentes oficiales con una cobertura y una inmediatez difíciles de igualar. Meier (2015) sistematiza esta corriente bajo el término digital humanitarians, documentando el papel de las comunidades de voluntarios digitales en crisis reales. Esta evolución se enmarca en la disciplina de la crisis informatics (Palen y Anderson, 2016), que estudia el aprovechamiento de los datos generados por la ciudadanía en situaciones extraordinarias; revisiones sistemáticas del campo han analizado tanto el procesamiento de mensajes en redes sociales durante emergencias (Imran et al., 2015) como la proliferación de aplicaciones móviles para la gestión de crisis (Tan et al., 2017) y la trayectoria de quince años de medios sociales en emergencias (Reuter y Kaufhold, 2018).

Desde la perspectiva de la optimización, la distribución de recursos en emergencias se ha modelado como variantes del problema de rutas de vehículos (VRP). Sicilia et al. (2016) proponen algoritmos para el enrutamiento en distribución urbana de mercancías, un problema cercano al de llevar suministros a puntos de necesidad evitando vías inutilizadas. Estas técnicas inspiran el subsistema de enrutamiento del presente trabajo, si bien aquí se prioriza la robustez ante información incompleta sobre la optimalidad estricta.

En el plano de la arquitectura de datos, Kleppmann (2017) ofrece el marco de referencia sobre sistemas intensivos en datos, replicación y consistencia, y Shapiro et al. (2011) formalizan los tipos de datos replicados libres de conflicto (CRDT), una técnica que permite la convergencia automática de réplicas modificadas de forma concurrente. Aunque los CRDT representan el horizonte teórico de la sincronización offline, su complejidad llevó a adoptar en este MVP una estrategia más pragmática basada en una cola de operaciones con idempotencia, reservando los CRDT como línea de evolución.

En el terreno de la tecnología cliente, la consolidación de las Progressive Web Apps (Russell, 2015; Biørn-Hansen et al., 2017) y la concienciación sobre el coste del JavaScript en dispositivos modestos (Osmani, 2018) han hecho viable construir aplicaciones instalables, capaces de funcionar sin conexión mediante Service Workers y de ofrecer una experiencia próxima a la nativa sin las fricciones de distribución de las tiendas de aplicaciones. Esta tecnología es la que habilita el enfoque Offline-First del trabajo.

Conviene además situar el trabajo respecto a dos tendencias recientes. Por un lado, la computación en el borde (edge) y los enfoques local-first han ganado tracción como respuesta a la latencia y a la dependencia de la nube, desplazando capacidad de cómputo y de almacenamiento hacia el dispositivo del usuario; el cálculo de rutas sobre un grafo local que se describe en el capítulo 4 se inscribe en esta corriente. Por otro lado, la verificabilidad criptográfica de registros mediante cadenas de hashes —sin recurrir a una cadena de bloques pública— se ha consolidado como una técnica ligera para garantizar la integridad de historiales sensibles, y es la que sustenta el registro de auditoría del sistema. Ambas tendencias confluyen en la propuesta y refuerzan su pertinencia tecnológica.

Plataformas y soluciones existentes

En el ámbito de las plataformas reales, Sahana Eden (Careem et al., 2006) es un referente de software libre para la gestión de desastres, con módulos para personas, organizaciones, inventario y voluntariado; su principal limitación en el escenario que nos ocupa es la dependencia de conectividad y una interfaz orientada a personal especializado. Ushahidi (Okolloh, 2009) popularizó el mapeo colaborativo de incidencias mediante el envío de reportes ciudadanos, pero se centra en la visualización de información y no aborda la gestión de inventario ni la coordinación logística de la ayuda.

La plataforma GO de la Federación Internacional de Sociedades de la Cruz Roja y de la Media Luna Roja (IFRC, 2023) integra información operativa a escala global, aunque está orientada a organizaciones y no contempla la participación de la ciudadanía espontánea. En el extremo de la navegación, servicios como Google Maps o Waze ofrecen información de tráfico en tiempo real e incluso mapas offline parciales, pero carecen por completo de gestión de recursos y de participación cívica estructurada. Finalmente, los ERP logísticos comerciales (SAP, Odoo) proporcionan una gestión de inventario avanzada, a costa de exigir conectividad estable, personal cualificado y un despliegue que no encaja con la inmediatez de una emergencia.

Más allá de las plataformas generalistas de gestión de desastres, han surgido herramientas especializadas en la comunicación resiliente. Las aplicaciones de mensajería en malla, como Bridgefy o FireChat, permiten el intercambio de mensajes entre dispositivos cercanos mediante Bluetooth y wifi directo cuando la red celular se cae, y soluciones como Zello convierten el teléfono en un sistema de radio sobre IP. Estas herramientas resuelven el problema de la conectividad punto a punto, pero no abordan la gestión logística ni la trazabilidad de los recursos, que constituyen el foco del presente trabajo.

En el ámbito de la información geográfica voluntaria, iniciativas como el Humanitarian OpenStreetMap Team han demostrado el valor de la cartografía colaborativa para la respuesta a desastres, generando mapas detallados de las zonas afectadas a partir de contribuciones ciudadanas (Goodchild y Glennon, 2010). Catástrofe-Logística se apoya precisamente en OpenStreetMap como base cartográfica, pero incorpora sobre ella una capa logística —inventario dual por puesto, trazabilidad de donaciones y enrutamiento consciente de las vías cortadas— que estos proyectos de mapeo, centrados en la generación del mapa, no contemplan.

Comparativa de herramientas

La Tabla 2 sintetiza las capacidades de las soluciones analizadas frente a los requisitos que impone el escenario de emergencia: funcionamiento sin conexión, gestión de inventario, trazabilidad física de las entregas, soporte de roles diferenciados y carácter abierto. La comparación evidencia que ninguna herramienta cubre simultáneamente todas las dimensiones, y que el hueco más claro se sitúa en la combinación de operación offline, trazabilidad física y participación ciudadana.

Tabla 2. Comparativa de herramientas existentes para la gestión de emergencias.

Herramienta	Tipo	Offline	Gest. inventario	Trazabilidad física	Roles	Código abierto	Limitaciones principales en emergencias
Sahana Eden (Careem et al., 2006)	Gestión de desastres	No	Sí (básico)	No	Sí	Sí	Interfaz compleja; requiere conectividad; sin trazabilidad física de entregas
Ushahidi (Okolloh, 2009)	Mapeo colaborativo	No	No	No	Limitado	Sí	Sin gestión de inventario ni coordinación de voluntarios; solo reporte de incidencias
IFRC GO	Plataforma humanitaria	Parcial	Sí	No	Sí	Sí	Enfocado en organizaciones; sin soporte para ciudadanía espontánea; complejidad de despliegue
Google Maps / Waze	Navegación y tráfico	Parcial (mapas)	No	No	No	No	Sin gestión de recursos ni participación ciudadana activa
ERP logísticos (SAP, Odoo)	Gestión logística	No	Sí (avanzado)	Sí	Sí	Parcial	Requieren conectividad estable y personal especializado; sin participación ciudadana
Catástrofe-Logística (este trabajo)	Plataforma cívica + logística	Sí (Offline-First)	Sí (dual)	Sí (QR)	Sí (4 roles)	Sí	MVP; validación en escenario real pendiente; sin integración con sistemas oficiales

Fuente: Fuentes: Careem et al. (2006), Okolloh (2009), IFRC (2023) y elaboración propia.

2.3. Marco normativo y consideraciones éticas

Una plataforma que trata datos personales en un contexto sensible debe diseñarse desde el principio con las garantías legales y éticas adecuadas. El marco de referencia es el Reglamento (UE) 2016/679 (RGPD) y su desarrollo en España mediante la Ley Orgánica 3/2018 (LOPDGDD). De este marco se derivan principios directamente aplicables al diseño del sistema: la licitud y la limitación de la finalidad (los datos se recogen para coordinar la respuesta a la emergencia y no para otros fines), la minimización (solo se recogen los datos estrictamente necesarios), la exactitud, la limitación del plazo de conservación, y la integridad y confidencialidad mediante medidas técnicas adecuadas (art. 32).

El escenario de emergencia introduce además una tensión ética particular: el equilibrio entre la utilidad de compartir información —ubicación, necesidades, evidencias— y la protección de la privacidad de las personas afectadas, que con frecuencia se encuentran en situación de vulnerabilidad. El trabajo resuelve esta tensión con decisiones concretas: la ubicación se utiliza en tiempo real pero no se persiste por defecto, los datos públicos de las incidencias excluyen los identificadores del reportante, y el registro público de auditoría expone únicamente la cadena de eventos verificable, nunca las tablas de usuarios. Estas decisiones, materializadas en la arquitectura y detalladas en el capítulo 4, encarnan el principio de protección de datos desde el diseño y por defecto (art. 25 del RGPD).

Desde una perspectiva ética más amplia, el sistema se concibe como un complemento a la acción de los organismos oficiales, no como un sustituto. Esta delimitación es deliberada:

evitar que una herramienta ciudadana genere una falsa sensación de cobertura o interfiera con la cadena de mando oficial es, en sí mismo, un requisito de diseño responsable.

2.4. Conclusiones del análisis

Del contexto y del estado del arte se extraen cuatro principios de diseño que vertebran la propuesta y que se retomarán en las decisiones técnicas del capítulo 4:

- **Resiliencia ante la red.** El sistema debe garantizar las funciones esenciales en condiciones de conectividad degradada, asumiendo la intermitencia como estado normal y no como excepción.
- **Crowdsourcing integrado con la logística.** La información colaborativa sobre el estado del entorno solo aporta valor si se conecta con la gestión física de recursos: enrutamiento que evita cortes, inventario por puesto y trazabilidad de entregas.
- **Transparencia y control.** La gestión de recursos y donaciones exige mecanismos de trazabilidad verificable que generen confianza entre actores que no necesariamente se conocen.
- **Coordinación flexible entre roles.** La heterogeneidad de los actores obliga a un modelo de permisos diferenciado y a vistas adaptadas a cada perfil, sin sacrificar la simplicidad de uso.

Estos principios justifican tanto la arquitectura Offline-First como las funcionalidades nucleares del sistema, y constituyen el criterio frente al cual se evalúa la contribución en la evaluación (apartado 4.1.3).

3. Objetivos y metodología de trabajo

A partir del problema planteado y de los principios de diseño identificados, este capítulo formula el objetivo general del trabajo, lo descompone en objetivos específicos verificables y describe la metodología, la planificación por incrementos y la gestión de riesgos que han guiado el desarrollo.

3.1. Objetivo general

Diseñar y desarrollar una plataforma software basada en una arquitectura Offline-First que mejore la coordinación, la eficiencia y la trazabilidad en la gestión logística de la ayuda humanitaria durante situaciones de emergencia, integrando a ciudadanía, voluntariado, puntos de distribución y coordinación en torno a un mapa operativo común y garantizando la continuidad de las funciones esenciales en condiciones de conectividad limitada.

3.2. Objetivos específicos

El objetivo general se concreta en los siguientes objetivos específicos, cada uno de los cuales se traduce posteriormente en requisitos y se contrasta en la evaluación:

1. Analizar las limitaciones de los sistemas actuales de gestión logística de emergencias e identificar los principios de diseño aplicables (capítulo 2).
2. Modelar los actores implicados —ciudadano, voluntario, puesto de emergencia y coordinador— con responsabilidades, alcance y permisos diferenciados.
3. Diseñar e implementar un mecanismo de reporte colaborativo (crowdsourcing) de incidencias geolocalizadas que actualice de forma dinámica el estado de las vías.
4. Implementar la gestión de un inventario dual por puesto (productos disponibles y necesarios) con historial trazable de movimientos.
5. Diseñar un sistema de trazabilidad de donaciones mediante códigos QR de un solo uso que verifique y registre las entregas.
6. Desarrollar un servicio de enrutamiento que tenga en cuenta el estado dinámico de las vías para optimizar el transporte de recursos.

7. Implementar una arquitectura Offline-First que garantice el funcionamiento del sistema en condiciones de conectividad limitada, con sincronización diferida fiable.
8. Implementar autenticación y control de acceso basado en roles (RBAC) que diferencie las capacidades de cada perfil.
9. Diseñar un registro verificable encadenado de las operaciones críticas, basado en el encadenamiento criptográfico de eventos.
10. Incorporar un mecanismo de comunicación en tiempo real que difunda los cambios críticos a los clientes conectados.
11. Evaluar el sistema mediante pruebas automatizadas, evaluación heurística y una prueba piloto simulada, analizando su impacto en la coordinación y su usabilidad.

3.3. Metodología de trabajo

El desarrollo se ha abordado mediante un modelo iterativo e incremental, adecuado para un sistema con múltiples subsistemas interdependientes y requisitos que se refinan a medida que se valida cada parte. Este enfoque permite una evolución progresiva, con validación continua de cada funcionalidad antes de construir sobre ella, y reduce el riesgo frente a un modelo en cascada, que pospondría la integración hasta el final, o frente a un ágil estricto de sprints fijos, difícil de sostener en un equipo de dos personas con dedicación variable.

En cuanto al marco de trabajo ágil concreto, no se adoptó Scrum en su forma canónica —con sprints de duración fija y la totalidad de sus roles y ceremonias—, por resultar desproporcionado para un equipo de dos personas, sino un enfoque Kanban combinado con el modelo iterativo e incremental. El trabajo se visualizó en un tablero con columnas por estado (pendiente, en curso, en revisión y terminado), limitando deliberadamente el número de tareas en curso y tirando de las siguientes a medida que se completaban las anteriores. Esta elección prioriza el flujo continuo y la capacidad de adaptación frente a una cadencia rígida, encaja de forma natural con la integración frecuente mediante pull requests revisadas y permite reordenar las prioridades cuando la validación de un incremento revela nuevas necesidades. La elección se alinea, además, con las prácticas de desarrollo seguro propuestas en la literatura para integrar la seguridad de forma transversal a lo largo de todo el ciclo ágil (De Vicente Mohino et al., 2019).

Fases del desarrollo

El proceso se ha estructurado en cinco fases que se repiten en cada incremento:

- **Identificación y análisis de requisitos.** Análisis del problema y del contexto de uso para derivar los requisitos funcionales y no funcionales de cada incremento.
- **Diseño del sistema.** Definición de la arquitectura y de los componentes, modelado de datos y de las interacciones entre subsistemas.
- **Implementación.** Desarrollo modular de las funcionalidades, con integración frecuente sobre la rama principal.
- **Integración.** Ensamblado de los módulos en una plataforma unificada y verificación de su comunicación.
- **Validación y evaluación.** Pruebas funcionales, automatizadas y de usabilidad para verificar el cumplimiento de los requisitos.

Herramientas y técnicas

El trabajo se ha apoyado en prácticas ágiles adaptadas al tamaño del equipo:

- Control de versiones distribuido con Git y flujo basado en ramas por funcionalidad, con integración frecuente sobre la rama principal.
- Documentación técnica compartida en el repositorio (README, documentación por rol, guías de implementación y de desarrollo local).
- Pruebas automatizadas como parte del ciclo de desarrollo: Vitest para pruebas unitarias y de integración, y Playwright para pruebas extremo a extremo.
- Entorno reproducible con Docker Compose, que evita discrepancias entre las máquinas de los autores y facilita la demostración.
- Comunicación periódica para sincronizar el estado del trabajo y tomar decisiones técnicas conjuntas, especialmente en los puntos de integración entre frontend y backend.

La separación de responsabilidades estructuró el trabajo en dos líneas paralelas: Francisco Capote asumió el desarrollo del frontend y la evaluación de usabilidad, y Álvaro Martín, el

backend y el análisis de seguridad; la integración, el despliegue y los apartados transversales de la memoria se abordaron de forma conjunta.

3.4. Planificación de incrementos

La Tabla 3 resume los ocho incrementos en los que se organizó el desarrollo, reconstruidos a partir del historial de commits y de las pull requests del repositorio. Cada incremento entrega funcionalidad utilizable y verificable, de modo que el sistema crece de forma controlada desde la base de autenticación y mapa hasta la optimización de rutas y el despliegue en producción.

Tabla 3. Planificación de incrementos del desarrollo, reconstruida a partir del historial de commits y pull requests del repositorio.

Incremento	Periodo	Commits representativos	Entregables clave	PRs / Hito
I-01 Planificación y diseño	03/01/2026 – 18/03/2026	Initial plan; Add comprehensive technical implementation guide; docs: Add role documentation	Documentación de roles (ciudadano, voluntario, puesto, coordinador), arquitectura técnica, plan de implementación	PRs #1-4 (docs)
I-02 Estructura base y autenticación	27/04/2026 – 30/04/2026	Estructura inicial; Inicial ciudadano mapa interactivo; Registro y login por email/DNI, roles iniciales y recuperación mediante código	Proyecto scaffolded (React + Fastify + Prisma + Docker); auth JWT; registro de usuario y puesto; mapa base con Leaflet/OSM	Base del repo
I-03 Incidencias, búsqueda de productos y voluntario	27/04/2026 – 06/05/2026	feat(incidencias): crear módulo; faet: ruta no puede pasar por calle cortada; feature(buscar-productos); Funcionalidad voluntario; Test incidencias vías; Test buscar productos	Módulo incidencias de vías (reporte + routing que las evita); búsqueda de productos por ciudadano; flujo de voluntario; pruebas unitarias	PRs #5, #6, #8, #20
I-04 Puesto de emergencia e inventario	05/05/2026 – 13/05/2026	Merge branch Puesto-de-emergencia; test: add emergency post test suite; feature(ayuda-puesto): gestionar inventario; Mejora inventario de puestos y voluntarios; feat: enhance role management	Módulo puesto de emergencia (alta, aprobación por coordinador); inventario dual (disponible/necesario); gestión de roles; pruebas del puesto	PRs #26, #27
I-05 Donaciones, QR y navegación asistida	18/05/2026 – 22/05/2026	Implement QR request and donation workflows; añadido navegación asistida con brújula; fix: navegación evita incidencias cortadas; feature: eliminar flujo catástrofes, middleware idempotencia, sincronización offline	Ciclo completo de donaciones (pendiente→en camino→entregada); códigos QR one-shot; navegación asistida; middleware de idempotencia; cola offline (IndexedDB)	PR #28
I-06 Seguridad, offline-first y registro inalterable	23/05/2026 – 26/05/2026	Asegura sesiones con cookie HttpOnly y CSP; Protege datos personales en incidencias; Refina control de acceso por roles; Completa registro público inalterable y validación; Protege eventos realtime con autenticación	RBAC completo; CSP y HttpOnly; registro inalterable con encadenamiento criptográfico (ChainEvent); endpoint público /verificar; PWA offline-first completo	PR #30
I-07 Coordinador, UX y despliegue en producción	27/05/2026 – 09/06/2026	Implementa funcionalidad de coordinador; Rediseña panel puesto de emergencia; Mejora panel administrativo del coordinador; Prepare Render and Supabase deployment; ajusta despliegue same-origin y recuperación sin SMTP obligatorio; refactor: unify test configurations	Panel coordinador (aprobación puestos, gestión participaciones); rediseño UX de todos los roles; despliegue en Render + Supabase; configuración SMTP transaccional; suites Playwright unificadas	PRs #31, #32, #33
I-08 Optimización de rutas y correcciones finales	17/06/2026 – 19/06/2026	feat: add OSM graph extraction and routing functionality + Benchmark; feat: add routing benchmark tests; arreglado fallo en login que diferenciaba mayúsculas	Extracción de grafo OSM y cálculo de rutas optimizado; benchmarks de rendimiento de routing; corrección de bugs críticos en autenticación	—

Fuente: Elaboración propia.

3.5. Gestión de riesgos

El desarrollo de una plataforma para emergencias implica riesgos técnicos, de seguridad y de proyecto que conviene identificar y mitigar de forma anticipada. La Tabla 4 recoge el registro de riesgos, con su probabilidad e impacto estimados y la mitigación adoptada o prevista. La probabilidad se estima en una escala cualitativa (alta, media, baja).

Tabla 4. Registro de riesgos del proyecto, con probabilidad e impacto estimados y mitigación adoptada.

ID	Riesgo	Tipo	Prob.	Impacto	Mitigación adoptada
R-01	Pérdida de datos en sincronización offline	Técnico	Media	Alto	Idempotencia (IdempotencyRecord); cola persistente IndexedDB; reintentos con backoff
R-02	Conflictos de datos no resolubles automáticamente	Técnico	Media	Medio	Last-writer-wins para inventario; append-only para incidencias; CRDTs como trabajo futuro
R-03	Escalabilidad insuficiente en emergencia masiva	Técnico	Baja	Alto	API sin estado (JWT); Redis distribuido; escalado horizontal con balanceador (RNF-07)
R-04	Brecha de seguridad en datos personales	Seguridad/Legal	Baja	Alto	RBAC estricto; bcrypt; HTTPS/WSS; cumplimiento RGPD (RNF-05)
R-05	Caída del servidor durante emergencia activa	Disponibilidad	Media	Alto	Offline-First garantiza funcionalidad básica; Docker con healthchecks
R-06	Falsos positivos en validación colaborativa de vías	Funcional	Media	Medio	Cambio de estado mediante comentarios autenticados y override explícito del coordinador; umbral automático de validaciones queda como mejora futura (RF-10).
R-07	Dependencia de OpenStreetMap (tile server externo)	Externo	Baja	Medio	Precaché de teselas para la zona de interés; degradación controlada offline
R-08	Retraso por scope creep o mala priorización	Proyecto	Media	Alto	Modelo iterativo con incrementos acotados; priorización por requisitos MVP

Fuente: *Elaboración propia.*

3.6. Aseguramiento de la calidad

La calidad del software no se ha tratado como una fase final, sino como una práctica transversal a todo el desarrollo. Se adoptó una definición de «terminado» (definition of done) común: una funcionalidad solo se consideraba completa cuando incluía su validación con esquemas, sus pruebas automatizadas, la actualización de la documentación pertinente y la verificación de los flujos afectados. Este criterio evitó la acumulación de deuda técnica y mantuvo el sistema en un estado integrable de forma continua.

Sobre el código se aplicaron herramientas de calidad estática —ESLint y el compilador de TypeScript en modo estricto— que detectan errores de tipado y desviaciones de estilo antes de la ejecución. El tipado estático de extremo a extremo, desde el esquema de la base de datos (Prisma) hasta el contrato de la API (Zod) y la interfaz (TypeScript), constituye en sí mismo una red de seguridad que descarta clases enteras de errores en tiempo de compilación. La revisión cruzada del código entre los dos autores, articulada mediante las pull requests del repositorio, complementó estas comprobaciones automáticas con una validación humana del diseño y de la legibilidad.

Finalmente, la organización del repositorio por funcionalidades, con su correspondiente carpeta de pruebas, no solo facilita la mantenibilidad (RNF-09), sino que materializa la trazabilidad entre requisitos, implementación y verificación que vertebra la evaluación (apartado 4.1.3).

4. Desarrollo específico de la contribución

Este capítulo describe el desarrollo técnico de Catástrofe-Logística. Parte del catálogo de requisitos y del modelo de actores, justifica el stack tecnológico y, a continuación, detalla la arquitectura y la implementación de cada subsistema. La exposición integra las evidencias verificadas en el repositorio del proyecto, de modo que cada decisión de diseño pueda contrastarse con el código realmente construido.

4.1. Desarrollo práctico

Este Trabajo de Fin de Estudios corresponde a la tipología de desarrollo práctico. En los apartados siguientes se identifican los requisitos que han guiado la construcción del sistema (4.1.1), se describe en detalle el sistema software desarrollado y cada uno de sus subsistemas (4.1.2) y se presenta su evaluación (4.1.3).

4.1.1. Identificación de requisitos

El desarrollo parte de la necesidad de mejorar la coordinación logística en situaciones de emergencia, traducida en un conjunto de requisitos funcionales (RF), que describen las capacidades observables del sistema, y no funcionales (RNF), que condicionan su arquitectura y su operación. La Tabla 5 presenta el catálogo completo: veintiséis requisitos funcionales y once no funcionales, cada uno con identificador único, descripción y prioridad.

Tabla 5. Catálogo completo de requisitos del sistema (26 funcionales y 11 no funcionales).

ID	Descripción	Tipo	Prioridad
RF-01	Registro de usuario con email y contraseña, validando formato y unicidad.	Funcional	Alta
RF-02	Inicio de sesión con credenciales; emisión de access token JWT y refresh token persistente.	Funcional	Alta
RF-03	Cuatro roles diferenciados (ciudadano, voluntario, puesto de emergencia, coordinador) con acceso exclusivo a sus rutas y vistas.	Funcional	Alta
RF-04	Recuperación de contraseña mediante código de recuperación, email y DNI/NIE, sin dependencia obligatoria de un servidor de correo (SMTP); el envío por correo es una capacidad opcional.	Funcional	Media
RF-05	Generación de código de recuperación en el registro y posibilidad de regenerarlo desde una sesión autenticada.	Funcional	Alta
RF-06	Cierre de sesión con invalidación del refresh token en servidor.	Funcional	Media
RF-07	El coordinador puede supervisar puestos, usuarios, incidencias y solicitudes de participación, aprobando o rechazando altas y reasignaciones. El módulo de catástrofes fue retirado del flujo final, por lo que la emergencia se gestiona operativamente mediante puestos, incidencias y asignaciones.	Funcional	Alta
RF-08	El coordinador puede revisar, aprobar o rechazar solicitudes de alta de puestos de emergencia, registrando el motivo de rechazo.	Funcional	Alta
RF-09	Los ciudadanos y voluntarios pueden reportar incidencias geolocalizadas (cortada, inundada, peligrosa) con foto y comentario opcionales.	Funcional	Alta
RF-10	Las incidencias se resuelven por validación colaborativa mediante comentarios autenticados o por acción directa del coordinador (el umbral automático de validaciones queda como mejora futura).	Funcional	Alta
RF-11	Un usuario verificado puede dar de alta un puesto de emergencia; el puesto queda pendiente de aprobación hasta la revisión del coordinador.	Funcional	Alta
RF-12	El responsable del puesto puede actualizar su estado operativo (operativo, saturado, sin recursos, necesita	Funcional	Alta

ID	Descripción	Tipo	Prioridad
	voluntarios, cerrado).		
RF-13	El puesto gestiona inventario dual: productos disponibles y necesarios, con prioridad y unidades.	Funcional	Alta
RF-14	El sistema registra el historial de movimientos de inventario (entradas y salidas) con autor y timestamp.	Funcional	Alta
RF-15	El sistema genera códigos QR de un solo uso para validar entregas de productos a un puesto.	Funcional	Alta
RF-16	El voluntario puede escanear el QR del puesto para confirmar la entrega y actualizar el inventario.	Funcional	Alta
RF-17	Los ciudadanos pueden solicitar productos o ayuda directa asociada a una emergencia activa.	Funcional	Media
RF-18	El sistema calcula rutas entre la ubicación del usuario y un destino evitando incidencias activas (cortadas/peligrosas).	Funcional	Alta
RF-19	El sistema proporciona navegación asistida con brújula e indicaciones paso a paso hacia el destino seleccionado.	Funcional	Media
RF-20	El mapa muestra de forma diferenciada puestos, incidencias, rutas y estado de navegación, con leyenda documentada y legible en móviles. No existe una entidad Catastrofe persistida en el esquema final.	Funcional	Alta
RF-21	El sistema permite la creación offline de incidencias y solicitudes, encolando las operaciones pendientes para sincronización posterior sin pérdida de datos.	Funcional	Alta
RF-22	El sistema difunde en tiempo real los cambios críticos (incidencias, inventario, donaciones, puestos) a los clientes conectados mediante WebSocket (Socket.IO).	Funcional	Alta
RF-23	Los usuarios pueden compartir su geolocalización en tiempo real durante una emergencia activa.	Funcional	Media
RF-24	El sistema permite la coordinación de mano de obra: solicitud, aceptación y asignación de voluntarios a puestos e incidencias activas.	Funcional	Media
RF-25	El sistema registra operaciones críticas (entregas, movimientos de inventario, altas de puesto) en un registro público inalterable con encadenamiento criptográfico de eventos verificable por terceros.	Funcional	Media
RF-26	Los usuarios pueden consultar y editar su perfil (datos personales, rol activo, historial de acciones propias).	Funcional	Baja
RNF-01	Las funcionalidades críticas (mapa, incidencias, inventarios cacheados) operan en modo offline. Lighthouse PWA > 90.	No funcional	Alta
RNF-02	Las operaciones offline no se pierden aunque el navegador se cierre; se sincronizan sin duplicados al reconectar (idempotencia).	No funcional	Alta
RNF-03	Las contraseñas se almacenan con bcrypt (coste configurable); tokens JWT firmados con clave >= 256 bits.	No funcional	Alta
RNF-04	La API protege sus endpoints con rate limiting por IP y cabeceras de seguridad HTTP (CSP, HSTS, X-Frame-Options).	No funcional	Alta
RNF-05	El sistema gestiona datos personales (email, ubicación, evidencias) conforme al RGPD y la LOPDGDD (minimización, acceso, rectificación, supresión).	No funcional	Alta
RNF-06	El tiempo de carga inicial en 3G simulada no excede 4 s en First Contentful Paint (Lighthouse).	No funcional	Media
RNF-07	La arquitectura permite escalar horizontalmente la capa de API y el bus de eventos sin cambios en el código.	No funcional	Media
RNF-08	La interfaz cumple WCAG 2.1 nivel AA en contrastes, áreas táctiles y navegación por teclado. Lighthouse accesibilidad > 90.	No funcional	Media
RNF-09	El código se organiza por módulos cohesivos (backend/src/modules, frontend/src/features) con pruebas unitarias y E2E por módulo.	No funcional	Media
RNF-10	Las operaciones críticas se registran en un log de auditoría inmutable (AuditLog) con autor, timestamp y diff de la operación.	No funcional	Alta
RNF-11	El entorno completo (API, PWA, PostgreSQL, Redis, MinIO) se levanta de forma reproducible con un único comando Docker Compose.	No funcional	Media

Fuente: Elaboración propia.

4.1.1.1. Trazabilidad entre requisitos, implementación y pruebas

Para reforzar la correspondencia entre lo especificado y lo construido, la Tabla 6 establece una matriz de trazabilidad que vincula cada grupo de requisitos con los módulos del repositorio que lo implementan y con las pruebas que lo verifican. Esta matriz es una de las garantías de rigor del trabajo: permite comprobar que ningún requisito queda sin implementación ni sin verificación, y orienta la lectura del resto del capítulo.

Tabla 6. Matriz de trazabilidad entre requisitos, implementación real en el repositorio y pruebas asociadas.

Requisito(s)	Funcionalidad validada	Implementación real en el repositorio	Pruebas / verificación
RF-01, RF-02, RF-04, RF-05, RF-06	Autenticación, verificación de email, recuperación de contraseña y cierre de sesión	backend/src/modules/auth/*; backend/src/middleware/auth.middleware.ts; frontend/src/features/auth/*;	TEST/feature-registro-usuarios/backend/ auth.service.test.ts; auth.controller.test.ts; frontend/auth-flow.test.tsx

Requisito(s)	Funcionalidad validada	Implementación real en el repositorio	Pruebas / verificación
		frontend/src/store/auth.store.ts	
RF-03	Control de acceso por roles	backend/src/middleware/rbac.middleware.ts; frontend/src/router/RoleGuard.tsx; AuthGuard.tsx	TEST/feature-registro-usuarios/frontend/auth-store.test.ts; TEST/funcionalidad-coordinador/*
RF-07, RF-08, RF-12	Coordinación y gestión de puestos	backend/src/modules/puestos/*; frontend/src/features/coordinador/pages/Dashboard.tsx; frontend/src/features/puesto/pages/Dashboard.tsx	TEST/coordinar-puesto-19-estados-puesto/*; TEST/funcionalidad-coordinador/*
RF-09, RF-10	Reporte y validación colaborativa de incidencias	backend/src/modules/incidencias/*; frontend/src/features/ciudadano/*; frontend/src/components/shared/Map.tsx	TEST/feature-incidencias-vias/*; TEST/ayudar-incidencia-16/*
RF-13, RF-14	Inventario dual y movimientos de stock	backend/src/modules/inventario/*; frontend/src/features/puesto/*; frontend/src/features/voluntario/components/InventarioPuestoSheets.tsx	TEST/feature-puesto-emergencia/backend/inventario*.test.ts; frontend/PuestoDashboard.test.tsx
RF-15, RF-16	Entrega mediante QR de un solo uso	backend/src/modules/donaciones/*; backend/prisma/model/QrConsumption; frontend/src/components/shared/QrScanner.tsx; ReadableQRCode.tsx	TEST/feature-qr-solicitudes-donaciones/*; TEST/feature-puesto-emergencia/backend/inventario-qr*.test.ts
RF-17, RF-24	Solicitudes de ayuda y participación de voluntarios	backend/src/modules/puestos/trabajadores.service.ts; voluntarios.router.ts; frontend/src/features/voluntario/*	TEST/ayudar-en-un-puesto/*; TEST/issue-18-crear-puesto/*
RF-18, RF-19, RF-20	Rutas, navegación asistida y mapa	frontend/src/utlis/routing.ts; navegacion.ts; osmGraph.ts; osmGraphStore.ts; frontend/src/features/ciudadano/components/PanelNavegacionActiva.tsx	TEST/feature-buscar-productos-ciudadano/frontend/navegacion.test.ts; TEST/routing-grafo-local/*; frontend/benchmark/benchmark.test.ts
RF-21, RNF-01, RNF-02	Cola offline, IndexedDB, idempotencia y sincronización	frontend/src/lib/api/offline.ts; frontend/src/store/sync.store.ts; frontend/src/lib/db/index.ts; backend/src/middleware/idempotency.middleware.ts	TEST/offline-first/frontend/sync-queue.test.ts; frontend/e2e/offline-session.spec.ts
RF-22, RF-23	Tiempo real y geolocalización	backend/src/lib/realtime.ts; frontend/src/hooks/useRealtime.ts; useGeolocation.ts	TEST/geolocalizacion-tiempo-real/*; TEST/feature-incidencias-vias/backend/realtime.test.ts
RF-25, RNF-10	Registro de auditoría y cadena de eventos verificable	backend/src/modules/audit/*; backend/src/lib/chain.ts; chainWorker.ts; tsa.ts; frontend/src/features/public/pages/Verificar.tsx	TEST/registro-publico-inalterable/*
RNF-03, RNF-04, RNF-05	Seguridad, protección de API y privacidad	backend/src/app.ts; backend/src/lib/security.ts; auth.service.ts; @fastify/helmet, cookie, rate-limit, JWT	TEST/feature-registro-usuarios/*; TEST/operational-health/backend/maintenanceWorker.test.ts
RNF-09, RNF-11	Modularidad, mantenibilidad y entorno reproducible	backend/src/modules/*; frontend/src/features/*; docker-compose.yml; render.yaml	Compilación TypeScript, Vitest, Playwright y scripts de Docker Compose

Fuente: Elaboración propia.

4.1.1.2. Actores del sistema

El sistema se estructura en torno a cuatro perfiles de usuario con responsabilidades, alcance y permisos diferenciados. Una misma persona puede acumular varios roles —el modelo Usuario almacena un array de roles—, lo que permite, por ejemplo, que un voluntario actúe también como ciudadano. La Figura 1 resume los casos de uso principales por actor, y la Figura 2 los detalla.

Diagrama de casos de uso principales por actor

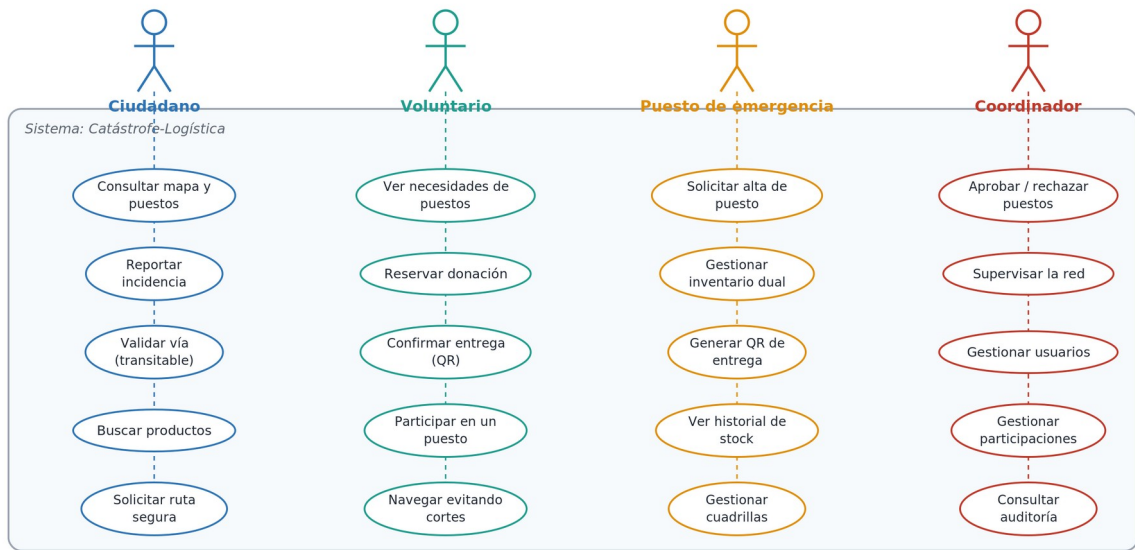


Figura 1. Diagrama de casos de uso principales por actor.

Fuente: elaboración propia.

Diagrama de casos de uso detallado por actor

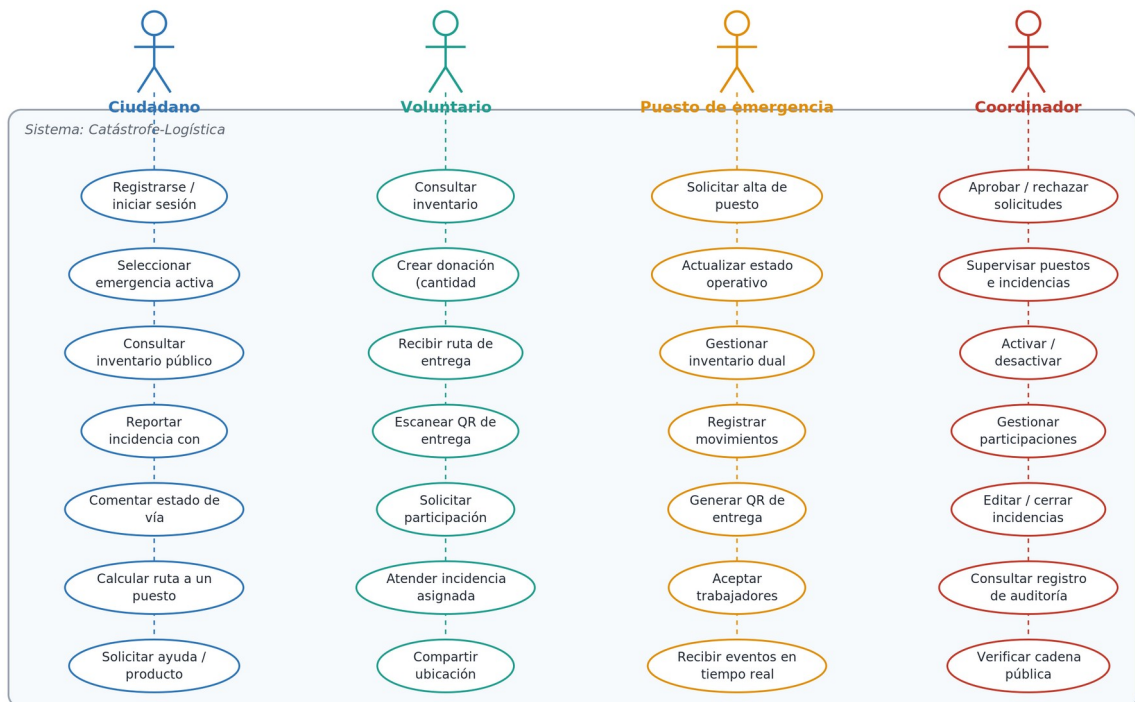


Figura 2. Diagrama de casos de uso detallado por actor.

Fuente: elaboración propia.

4.1.1.2.1. Ciudadano

Usuario general con un perfil simplificado, orientado a recibir información del estado del entorno y a contribuir a su actualización. Sus responsabilidades principales son consultar el mapa operativo y los puestos cercanos con su inventario y distancia; reportar incidencias geolocalizadas sobre vías o zonas afectadas; marcar como transitable una vía previamente reportada como bloqueada, contribuyendo a la validación colaborativa; buscar productos concretos (agua, alimentos, medicamentos) en los puestos y solicitar una ruta optimizada; y solicitar ayuda o donaciones para necesidades particulares. En cuanto a permisos, tiene lectura de puestos, inventarios públicos y mapa, y creación de incidencias y solicitudes propias, sin acceso a paneles de coordinación ni a operaciones de inventario.

4.1.1.2.2. Voluntario

Perfil dual que puede actuar como donante logístico —transporta bienes a puntos de necesidad— o como mano de obra en un puesto. Consulta la lista priorizada de productos más necesitados y los puestos que los demandan, reserva la entrega de productos a un puesto concreto y recibe una ruta optimizada que evita las incidencias activas, confirma la entrega mediante el escaneo del código QR generado por el puesto receptor, y puede inscribirse en cuadrillas de mano de obra cumpliendo los requisitos definidos por el coordinador. Sus permisos abarcan la lectura de puestos, inventarios y mapa, la creación y consumo de tokens de entrega (QR) asociados a su usuario y la autoinscripción en plazas disponibles, sin capacidad para modificar inventarios ajenos ni realizar coordinación global.

4.1.1.2.3. Puesto de emergencia

Punto solidario de distribución gestionado por una persona u organización responsable; constituye el nodo logístico donde se reciben, almacenan y entregan recursos. Crea y configura el puesto (nombre, ubicación, responsable, horario, capacidad y tipo de espacio), mantiene actualizado el inventario dual —productos disponibles para entregar y productos necesarios con su prioridad—, genera códigos QR de entrega para validar la recepción de donaciones, registra los movimientos de stock con historial trazable y coordina localmente cuadrillas de mano de obra. Dispone de control total (CRUD) sobre los recursos del propio puesto y de la generación y validación de tokens QR, sin acceso a la configuración global de la emergencia ni a puestos de otros responsables.

4.1.1.2.4. Coordinador

Rol con la mayor jerarquía operativa, encargado de supervisar la red de puestos de una emergencia y de garantizar la gobernanza del sistema. Revisa las solicitudes de alta de puestos y las aprueba o rechaza —registrando el motivo del rechazo—, supervisa el estado operativo de todos los puestos (operativo, saturado, sin recursos, necesita voluntarios o cerrado), gestiona las solicitudes de participación de voluntarios y consulta el detalle de cada puesto. Tiene acceso de coordinación sobre el conjunto de puestos y lectura del registro de auditoría, pero no administra los inventarios concretos, que corresponden a sus responsables. Una salvaguarda de gobernanza impide eliminar la última cuenta de coordinador activa, evitando dejar el sistema sin supervisión.

4.1.2. Descripción del sistema software desarrollado

Este apartado describe la solución construida: su stack tecnológico, su arquitectura y cada uno de los subsistemas que la componen, desde el funcionamiento sin conexión hasta el registro verificable y el despliegue.

4.1.2.1. Stack tecnológico

La selección del stack responde a tres criterios: madurez en producción, soporte explícito al funcionamiento sin conexión y ausencia de dependencias de pago que dificulten un despliegue accesible. La Tabla 7 detalla las tecnologías por capa, su versión de referencia y la justificación de su elección.

Tabla 7. Stack tecnológico de la plataforma, por capa, con su función y justificación.

Capa	Tecnología / Herramienta	Versión	Función y justificación
Frontend	React 18	18.x	Biblioteca UI con renderizado concurrente y Suspense; soporte amplio de ecosistema.
Frontend	TypeScript	5.x	Tipado estático en cliente y servidor; reduce errores en tiempo de desarrollo.
Frontend	Vite + vite-plugin-pwa	5.x	Bundler ultrarrápido con HMR; genera Service Worker y manifest PWA automáticamente.
Frontend	React Router v6	6.x	Enrutamiento declarativo con rutas protegidas por rol (RBAC en cliente).
Frontend	Zustand	4.x	Estado global ligero (usuario, catástrofe activa, modo conectividad) sin boilerplate.
Frontend	TanStack Query	5.x	Caché del estado remoto, invalidación automática y reintentos sin código manual.
Frontend	Leaflet + react-leaflet	4.x	Cartografía interactiva con OpenStreetMap; ligero y sin dependencia de APIs de pago.
Frontend	TailwindCSS	3.x	CSS utilitario; permite UI consistente sin CSS personalizado por componente.
Offline	Workbox (Service Worker)	7.x	Pre-caching del shell; estrategias Network-First/Cache-First por tipo de recurso.
Offline	Dexie.js (IndexedDB)	3.x	API ergonómica sobre IndexedDB nativo; almacena cola offline, inventario y teselas.
Backend	Node.js 20 LTS	20.x	Plataforma estable con soporte extendido; rendimiento I/O adecuado para API REST+WS.
Backend	Fastify + TypeScript	4.x	Framework HTTP de bajo overhead; sistema de plugins y esquemas JSON integrado.
Backend	Socket.IO	4.x	WebSocket con fallback a polling; difusión de eventos críticos en tiempo real.
Backend	Zod	3.x	Validación y parseado de esquemas en tiempo de ejecución; tipado inferido en TS.

Capa	Tecnología / Herramienta	Versión	Función y justificación
Backend	JWT + bcrypt	—	Autenticación sin estado con access/refresh tokens; hash adaptativo de contraseñas.
Datos	PostgreSQL 15 + PostGIS	15.x	BD relacional robusta; PostGIS habilita consultas geoespaciales (POINT, distancia).
Datos	Prisma ORM 5	5.x	Migraciones versionadas, cliente tipado y Prisma Studio para inspección en desarrollo.
Datos	Redis 7	7.x	Caché de consultas, rate limiting distribuido y almacén de sesiones activas.
Datos	MinIO (S3-compatible)	RELEASE	Almacenamiento de objetos (evidencias gráficas); API compatible con AWS S3.
Despliegue	Docker Compose	v2	Entorno reproducible con servicios (app, postgres, redis, minio); healthchecks.
Testing	Vitest	1.x	Tests unitarios sobre lógica de servicios y utilidades; integrado con Vite.
Testing	Playwright	1.x	Tests E2E que validan los flujos principales en navegador real.

Fuente: Elaboración propia.

4.1.2.2. Arquitectura general del sistema

La arquitectura sigue un modelo cliente-servidor con tres capas claramente diferenciadas, representado en la Figura 3. La capa de cliente es una Progressive Web App instalable, organizada por funcionalidades y por rol, con persistencia local en IndexedDB. La capa de aplicación es una API HTTP en Fastify, expuesta bajo el prefijo /api y organizada en routers modulares por dominio. La capa de datos se apoya en PostgreSQL como almacén relacional principal, con IndexedDB en el cliente para la persistencia offline y servicios auxiliares (Redis, MinIO) en el entorno local.

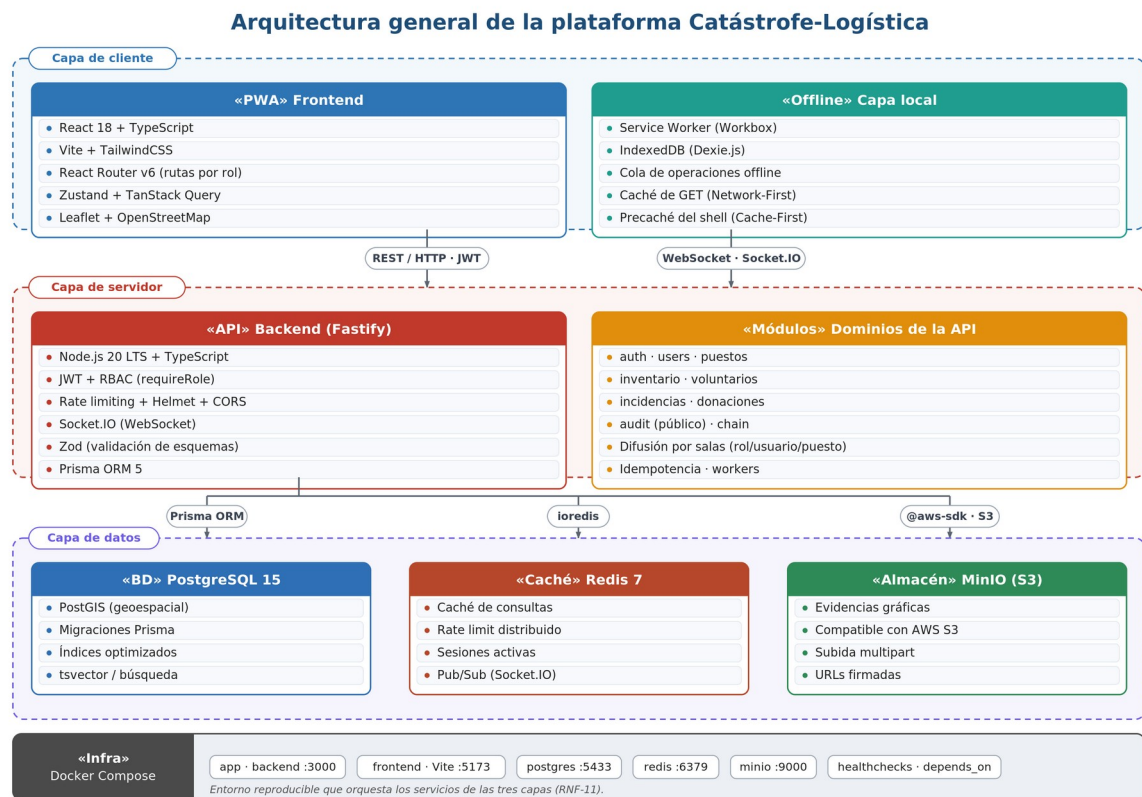


Figura 3. Arquitectura general de la plataforma Catástrofe-Logística.

Fuente: elaboración propia.

La comunicación entre cliente y servidor combina dos canales: una API REST para las peticiones síncronas y una conexión WebSocket (Socket.IO) para la difusión de cambios críticos en tiempo real. Conceptualmente, el sistema se organiza en torno a tres flujos transversales: el de incidencias y validación colaborativa, el de inventario y donaciones, y el de coordinación de puestos y voluntarios. La Figura 4 detalla los componentes principales y sus relaciones, desde la interfaz PWA hasta los módulos de backend y la base de datos.

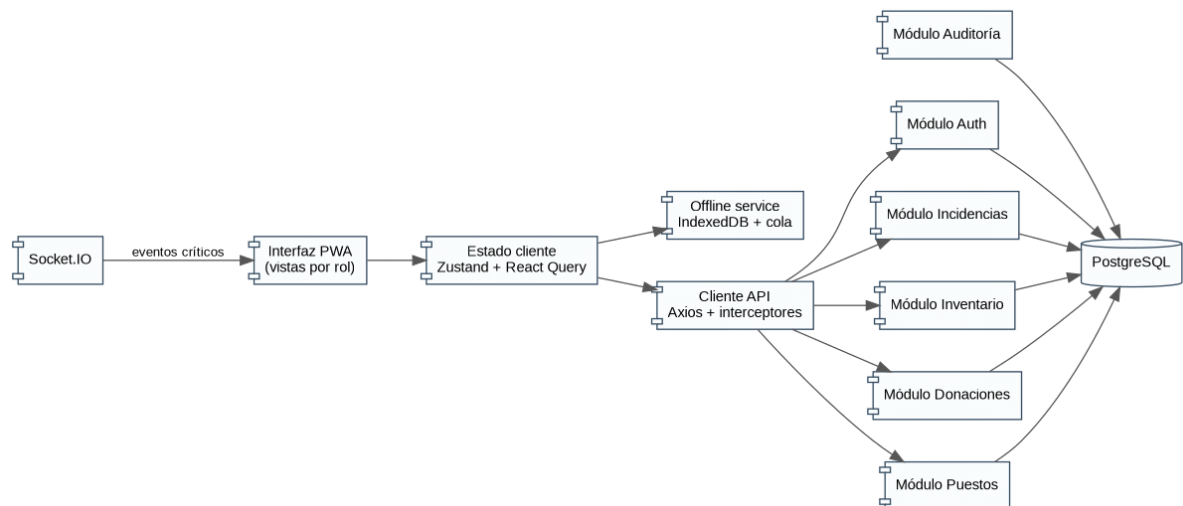


Figura 4. Diagrama de componentes principales del sistema.

Fuente: elaboración propia.

Esta separación en tres capas con dos canales de comunicación no es gratuita: responde directamente a los principios de diseño del capítulo 2. La capa de cliente, autónoma y con persistencia propia, materializa la resiliencia ante la red; el canal REST sostiene las operaciones transaccionales, mientras que el canal de eventos en tiempo real habilita la coordinación flexible entre actores que observan un mismo recurso. La frontera entre capas, definida por contratos tipados y validados, permite además evolucionar cada una de forma independiente, lo que resultó decisivo para el trabajo en paralelo de los dos autores.

La Tabla 8 resume la responsabilidad de cada componente de la arquitectura física. Conviene precisar el alcance real del despliegue: Redis y MinIO se incluyen en el entorno local reproducible como capacidades auxiliares, pero los flujos principales no dependen de ellos; el despliegue en la nube se realiza como un servicio combinado y se presenta como entorno preparado, no como una prueba de alta disponibilidad.

Tabla 8. Componentes de la arquitectura física y su responsabilidad.

Componente	Tecnología	Responsabilidad
Cliente web	PWA React + TypeScript + Vite	Interfaz de usuario, rutas por rol, mapas, QR, cola offline y caché local.
API	Fastify + TypeScript	Expone /api/auth, /api/users, /api/puestos, /api/inventario, /api/voluntarios, /api/incidencias, /api/donaciones y /api/public/audit.
Base de datos	PostgreSQL 15 + PostGIS	Persistencia principal del dominio, datos relacionales y soporte geoespacial.
Caché / soporte operativo	Redis 7	Soporte para cacheo, rate limiting y elementos operativos desacoplados.
Objetos	MinIO compatible S3	Almacenamiento de evidencias gráficas y ficheros asociados.
Tiempo real	Socket.IO	Difusión de incidencias, inventario, donaciones y actualizaciones restringidas por sala/rol.
Despliegue local	Docker Compose	Levanta API, frontend, PostgreSQL, Redis y MinIO de forma reproducible.
Despliegue cloud	Render + base de datos gestionada	Configuración declarativa en render.yaml; el despliegue productivo queda preparado, no se presenta como prueba de alta disponibilidad.

Fuente: Elaboración propia.

4.1.2.3. Frontend y experiencia de usuario

La aplicación se distribuye como una única PWA instalable construida con React 18 y TypeScript sobre Vite. La navegación se implementa con React Router v6 mediante un sistema de rutas protegidas: tras la autenticación, el cliente carga el árbol de rutas correspondiente al rol del usuario, de modo que cada perfil solo accede a sus vistas. El estado se gestiona con Zustand para el estado global ligero (usuario autenticado, modo de conectividad) y con TanStack Query para el estado remoto, que se cachea, se invalida automáticamente al recibir eventos en tiempo real y se reintenta de forma transparente.

El código del frontend se organiza por funcionalidades (carpeta features) más que por tipos de fichero, agrupando en cada feature sus páginas, componentes y lógica de acceso a datos. Esta organización mejora la cohesión y la mantenibilidad (RNF-09) y facilita la carga diferida por rol. Entre los componentes transversales destacan el AppShell común, que reduce las inconsistencias entre roles; el ConnectivityBanner, que advierte del estado offline/online; el LocationPermissionBanner, que gestiona los permisos y errores de geolocalización; y el PanelNavegacionActiva, que guía al usuario durante el seguimiento de una ruta.

La interfaz se orienta deliberadamente al uso móvil, coherente con un escenario de campo. La carga diferida (lazy loading) de las páginas reduce el peso inicial, y el manifest declara orientación vertical e idioma español para una experiencia instalada cercana a la nativa. En materia de accesibilidad (RNF-08) se cuidan los contrastes, el tamaño de las áreas táctiles y la navegación por teclado, siguiendo las recomendaciones WCAG 2.1 AA, si bien una validación formal con tecnologías de asistencia se reconoce como mejora futura. La cartografía se integra con Leaflet y react-leaflet sobre teselas de OpenStreetMap, con posibilidad de precaché para las zonas sin conectividad. Cada rol dispone de vistas específicas adaptadas a sus tareas.

La organización por funcionalidades se complementa con una estrategia de composición de componentes reutilizables y con un sistema de diseño basado en utilidades (TailwindCSS), que garantiza la coherencia visual entre los cuatro roles sin necesidad de mantener hojas de estilo por componente. Los componentes de presentación se mantienen separados de la lógica de acceso a datos, que se encapsula en hooks reutilizables apoyados en TanStack Query; de este modo, un cambio en un endpoint o en su política de caché se propaga a todas las vistas que lo consumen sin duplicar código. Esta separación, habitual en aplicaciones React de cierta envergadura, facilita además las pruebas: la lógica puede verificarse de forma aislada y los componentes de interfaz, mediante pruebas de renderizado con Testing Library.

4.1.2.4. Funcionamiento Offline-First

El funcionamiento sin conexión es la contribución técnica central del trabajo y no se reduce al uso de un Service Worker. Se articula en tres niveles complementarios: el cacheo del shell y de los recursos estáticos, la persistencia local de los datos de lectura y, sobre todo, una cola de operaciones de escritura que garantiza que ninguna acción se pierda y que ninguna se duplique al sincronizar. La Figura 5 ofrece una visión global del flujo, desde que el usuario genera una operación sin red hasta que esta se confirma en el servidor.

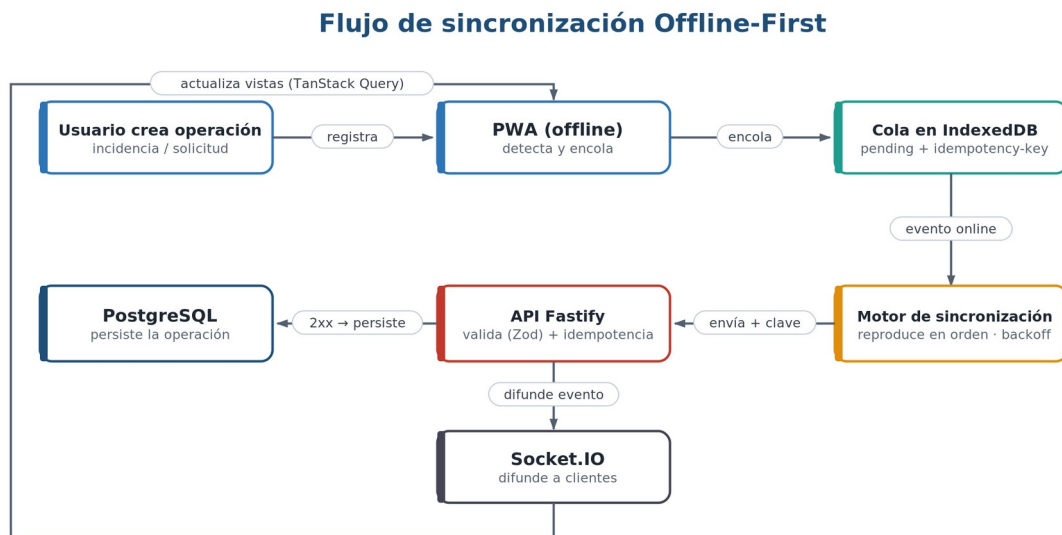


Figura 5. Flujo general de sincronización Offline-First.

Fuente: elaboración propia.

4.1.2.4.1. *Cacheo y persistencia local*

El Service Worker, registrado mediante vite-plugin-pwa y gestionado con Workbox, realiza el pre-caching del shell de la aplicación (HTML, CSS, JavaScript) y aplica estrategias de caché diferenciadas por tipo de recurso. La capa de persistencia local, construida sobre IndexedDB mediante Dexie.js, almacena los puestos visibles con su inventario, las incidencias activas de la zona de interés, una caché de respuestas GET por usuario (que excluye las rutas de autenticación) y las teselas cartográficas descargadas, cuya cuota se gestiona con la API `navigator.storage.estimate()`. El sistema distingue tres modos de conectividad —normal, degradado y offline— y los comunica al usuario mediante un indicador persistente.

4.1.2.4.2. *Cola de operaciones, reintentos e idempotencia*

Las operaciones de escritura generadas sin conexión no se descartan: se encolan en IndexedDB. Cada entrada de la cola almacena la entidad afectada, el método HTTP, el endpoint, el cuerpo de la petición, su estado (pendiente, sincronizando, conflicto o error), el número de reintentos y una clave de idempotencia. Al detectar el evento online del navegador, el cliente reproduce la cola contra el servidor en el orden en que se generó, aplicando reintentos con espera progresiva (backoff) y jitter para no saturar la API cuando muchos clientes se reconectan a la vez.

La fiabilidad de este proceso descansa en un middleware de idempotencia del lado del servidor. Dicho middleware procesa la cabecera `idempotency-key` en las operaciones de escritura: si una clave ya se procesó con éxito, devuelve la respuesta almacenada en lugar de ejecutar de nuevo la operación, marcándola con la cabecera `x-idempotency-replayed`. Las claves se conservan veinticuatro horas en la tabla `IdempotencyRecord` —tiempo suficiente para cubrir la mayoría de reconexiones—, las respuestas correctas se guardan como completadas y las fallidas se eliminan para que el cliente pueda reintentar sin bloquearse. Un worker de mantenimiento depura periódicamente los registros caducados para evitar el crecimiento indefinido de la tabla.

Un problema clásico de las aplicaciones offline-first es la resolución de identificadores: un cliente puede crear localmente una entidad y referenciarla en operaciones posteriores antes de conocer el identificador definitivo que asignará el servidor. El sistema lo resuelve mediante un mecanismo de `idMappings` que traduce los identificadores locales provisionales a los

definitivos durante la sincronización, preservando la integridad de las operaciones encadenadas. La Figura 6 formaliza la secuencia de sincronización completa y la Figura 7 representa el ciclo de vida de una operación encolada, desde su creación hasta su confirmación o su marcado como conflicto.

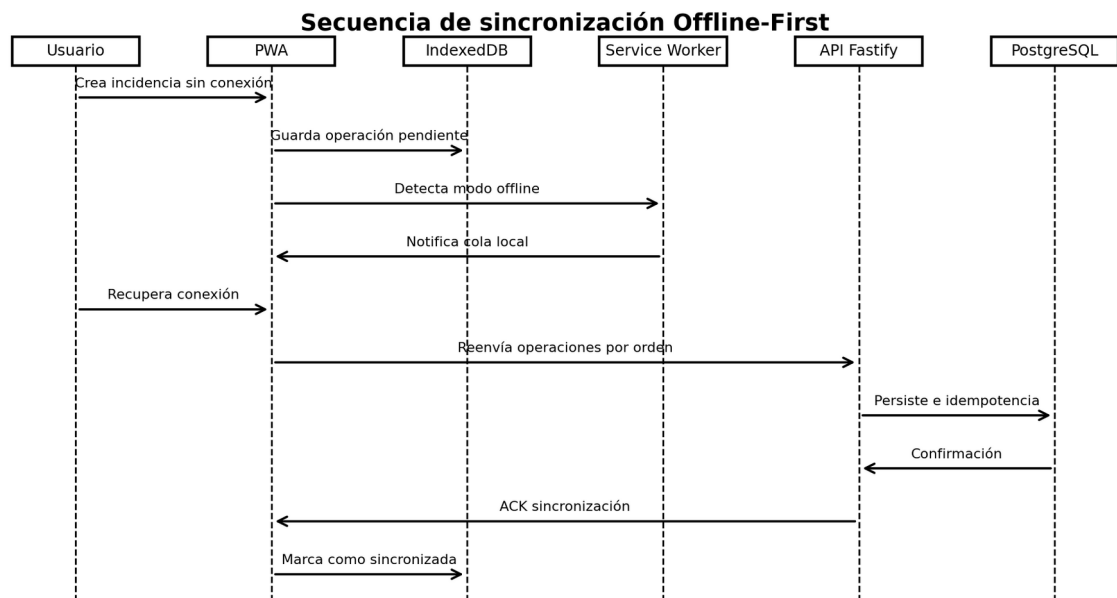


Figura 6. Diagrama de secuencia del proceso de sincronización Offline-First.

Fuente: elaboración propia.

Estados de una operación offline (cola de sincronización)

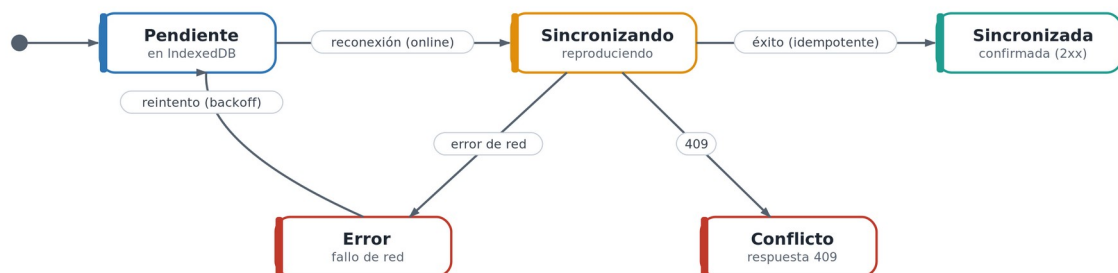


Figura 7. Diagrama de estados de una operación offline.

Fuente: elaboración propia.

De forma esquemática, el bucle de sincronización que ejecuta el cliente al recuperar la conectividad puede resumirse en el siguiente pseudocódigo, que recoge el orden de reproducción, la idempotencia, la traducción de identificadores y la clasificación del resultado en éxito, conflicto o error reintentable:

```

al evento online del navegador:
operaciones <- colaOffline.ordenarPorTimestamp()
para cada op en operaciones:
  op.endpoint <- idMappings.resolver(op.endpoint) // id local -> id servidor
  op.payload <- idMappings.resolver(op.payload)
  respuesta <- API.enviar(op, cabecera: { "idempotency-key": op.clave })
  segun respuesta.estado:
    2xx      -> colaOffline.eliminar(op)
              si crea entidad: idMappings.registrar(idLocal, idServidor)
    409 conflicto -> op.estado = "conflict"; resolver(politica(op.entidad))
    error de red -> op.reintentos++ ; esperar(backoff * 2^n + jitter)
    4xx permanente-> op.estado = "error"; notificar(usuario)

```

Pseudocódigo del bucle de sincronización de la cola offline al recuperar la conectividad.

La política de resolución de conflictos es deliberadamente sencilla y adecuada al MVP: «último en escribir gana» (last-writer-wins) para el inventario y append-only para las incidencias y sus comentarios, de modo que las contribuciones colaborativas nunca se sobrescriben. La adopción de tipos de datos replicados libres de conflicto (CRDT) se reserva como línea de evolución para escenarios de concurrencia más exigentes.

4.1.2.5. Backend y gestión de datos

El backend se ha desarrollado como un monolito modular en Node.js 20 LTS con Fastify y TypeScript. La elección de un monolito modular frente a una arquitectura de microservicios fue una decisión consciente y, a posteriori, acertada para un equipo de dos personas: ofrece la cohesión y la separación de responsabilidades de los módulos sin la sobrecarga operativa de desplegar, observar y coordinar múltiples servicios. Cada módulo se organiza con un router que declara los endpoints y aplica las guardas de autenticación y autorización, un servicio que concentra la lógica de negocio y un esquema Zod que valida las entradas y salidas.

Los dominios implementados son: auth (registro, inicio de sesión, refresh de tokens, recuperación de cuenta y revocación de sesiones); users (perfil del usuario autenticado); puestos (alta y administración de puestos, solicitudes de creación y gestión de participaciones); inventario (inventario dual, historial de movimientos y confirmación de entregas por QR); voluntarios (perfil y asignaciones); incidencias (reporte y validación colaborativa, con comentarios y asignaciones); donaciones (ciclo de vida de las donaciones y emisión de códigos de entrega); y audit junto con public/audit (registro de auditoría y endpoint público de consulta del registro verificable). La persistencia se gestiona con Prisma ORM 5 sobre PostgreSQL 15, con migraciones versionadas en el repositorio.

La organización modular del backend tiene una consecuencia práctica relevante para la evolución del sistema: añadir un nuevo dominio —por ejemplo, una futura integración con un

sistema oficial— se reduce a incorporar un nuevo módulo con su router, su servicio y su esquema, sin alterar los existentes. Esta propiedad de extensibilidad, junto con la ausencia de estado en la capa de API (gracias a los JWT), es la que sustenta la afirmación de escalabilidad horizontal del requisito RNF-07: replicar el proceso tras un balanceador no exige cambios en el código, siempre que el bus de eventos en tiempo real se respalde, en su caso, con un adaptador distribuido sobre Redis.

4.1.2.6. Seguridad, autenticación y cumplimiento del RGPD

La seguridad y la privacidad se han incorporado desde la fase de arquitectura, siguiendo el principio de privacy by design (art. 25 del RGPD) y en coherencia con los ciclos de desarrollo seguro (S-SDLC) propuestos para integrar la seguridad en todas las fases de un desarrollo ágil (De Vicente Mohino et al., 2019). La autenticación se basa en JSON Web Tokens emitidos por el backend tras validar las credenciales. El usuario puede iniciar sesión con su correo o con su DNI/NIE, y el sistema combina access tokens de vida corta (15 minutos) con refresh tokens persistidos en base de datos como hash y revocables individualmente. La Figura 8 detalla el proceso de validación de credenciales y emisión de tokens.

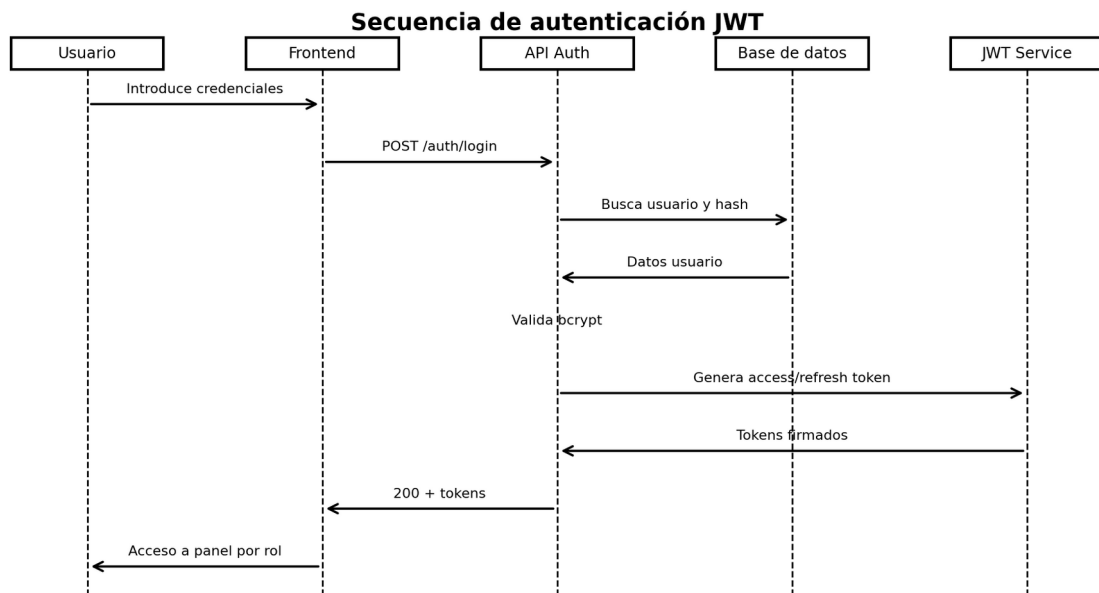


Figura 8. Diagrama de secuencia del proceso de autenticación con JWT.

Fuente: elaboración propia.

El control de acceso sigue un modelo RBAC: el modelo Usuario incluye un array de roles, y cada endpoint se protege con un middleware `requireRole()` que verifica el rol activo; en el

cliente, las rutas se cargan según el rol, de modo que la protección es doble. La protección perimetral de la API se completa con varios plugins de Fastify: @fastify/helmet para las cabeceras de seguridad (Content-Security-Policy, X-Frame-Options, HSTS), @fastify/cors para restringir los orígenes permitidos, @fastify/rate-limit con ventana deslizante y soporte de almacén distribuido en Redis, y @fastify/multipart con límites de tamaño y lista blanca de tipos MIME para la subida de evidencias. Toda la entrada se valida con Zod.

Una particularidad relevante para un sistema de emergencia es que la recuperación de cuenta no depende de un proveedor de correo. Durante el registro se genera un código de recuperación que, junto con el email y el DNI/NIE, permite restablecer la contraseña sin necesidad de un servidor SMTP operativo; el envío de correo es una capacidad opcional, no una dependencia crítica del flujo. Esta decisión reduce la dependencia de servicios externos justamente en el escenario donde estos pueden no estar disponibles.

En el plano de la privacidad, el tratamiento de datos personales (email, teléfono, DNI/NIE, ubicación aproximada y, ocasionalmente, evidencias gráficas) se rige por los principios de minimización y limitación de la finalidad. Los datos de ubicación se utilizan únicamente en tiempo real y no se persisten por defecto; los payloads públicos de incidencias excluyen los identificadores personales del reportante; y los endpoints de auditoría pública consultan la cadena de eventos, no las tablas de usuarios. La Tabla 9 sistematiza las medidas de seguridad implementadas y su correspondencia con los artículos del RGPD, lo que permite afirmar que el sistema cubre los requisitos no funcionales RNF-03, RNF-04 y RNF-05.

Tabla 9. Medidas de seguridad en el diseño y su correspondencia con el RGPD.

Área	Amenaza cubierta	Medida implementada	Principio / Norma
Autenticación	Suplantación de identidad (spoofing)	JWT firmado con HS256 (clave mínima de 256 bits). Access token de 15 min; refresh token de 7 días persistido como hash SHA-256 en BD.	Art. 32 RGPD — medidas técnicas adecuadas
Autorización	Escalada de privilegios; acceso no autorizado a recursos	RBAC con 4 roles. Middleware requireRole() en cada endpoint; rutas de cliente cargadas por rol.	Art. 5.1.f RGPD — integridad y confidencialidad
Contraseñas	Robo de credenciales por volcado de BD	Hash bcrypt con factor de coste configurable (por defecto 12). Nunca se almacena la contraseña en claro ni en logs.	Art. 32 RGPD — seudonimización y cifrado
Cifrado en tránsito	Interceptación de tráfico (MITM)	Comunicación sobre HTTPS y WSS (TLS 1.2+). @fastify/helmet activa HSTS para forzar conexiones seguras.	Art. 32 RGPD — cifrado en tránsito
Protección perimetral	Fuerza bruta; inyección; clickjacking	@fastify/helmet (CSP, X-Frame-Options, X-Content-Type-Options). @fastify/rate-limit con ventana deslizante en Redis. Validación con Zod.	Privacy by design (Art. 25 RGPD)
Gestión de sesión	Robo o reutilización de tokens	Refresh tokens como hash; invalidación individual al cerrar sesión. Tokens de un solo uso para verificación y reset.	Art. 32 RGPD — control de acceso
Datos personales	Acceso no autorizado a datos sensibles	Minimización: solo se recogen los datos necesarios. La ubicación se usa en tiempo real y no se persiste por defecto.	Art. 5.1.c RGPD — minimización
Auditoría	Falta de trazabilidad ante incidentes	AuditLog con actor, entidad y marca temporal. ChainEvent encadena hashes SHA-256 para garantizar la inalterabilidad.	Art. 5.1.f y Art. 30 RGPD
Derechos del interesado	Incumplimiento de acceso, rectificación y supresión	Acceso, rectificación y supresión del usuario autenticado; borrado lógico con anonimización para preservar integridad referencial.	Arts. 15-20 RGPD
CORS	Peticiones entre orígenes no autorizados	@fastify/cors con lista blanca de orígenes configurable por entorno.	Art. 32 RGPD — control de acceso a la API

Fuente: Elaboración propia.

4.1.2.7. Gestión de incidencias y enrutamiento

El subsistema de incidencias gestiona el reporte geolocalizado y la validación colaborativa del estado de las vías. Una incidencia se crea con coordenadas, título o categoría (cortada, inundada o peligrosa), estado y reportante, con foto y comentario opcionales. La actualización colaborativa se modela mediante comentarios (modelo ComentarioIncidenciaVia) de carácter append-only: cada comentario aporta una validación del estado, y el cambio se produce por acumulación de validaciones autenticadas o por intervención directa del coordinador. Los voluntarios pueden asociarse a una incidencia mediante una asignación (AsignacionIncidencia) para coordinar su atención. La Figura 9 formaliza los estados por los que puede pasar una incidencia.

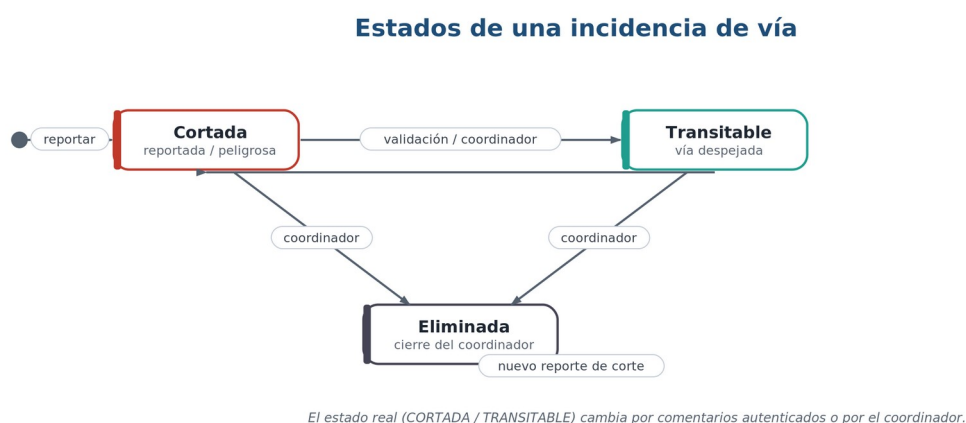


Figura 9. Diagrama de estados de una incidencia de vía.

Fuente: elaboración propia.

4.1.2.7.1. Enrutamiento local-first consciente de las vías cortadas

El cálculo de rutas es una de las aportaciones diferenciales del proyecto. La aplicación combina un enfoque local-first con un mecanismo de respaldo en línea: en lugar de depender exclusivamente de un servicio de rutas remoto, el cliente puede calcular trayectos sobre un grafo de calles obtenido a partir de extractos de OpenStreetMap (OSM) y almacenado localmente para su reutilización. El orquestador de rutas (frontend/src/utils/routing.ts) coordina las llamadas, el almacén de grafo (osmGraphStore.ts) carga y persiste los extractos OSM por zona, y el motor de grafo (osmGraph.ts) ejecuta una búsqueda de caminos mínimos sobre la red de calles.

El motor implementa una búsqueda de tipo A* —que utiliza la distancia geográfica restante como heurística admisible para acelerar la exploración— con la posibilidad de degradar a

Dijkstra cuando no procede emplear heurística. Sobre esta base se aplica la estrategia distintiva del sistema: la penalización blanda (soft penalty) de las aristas correspondientes a vías reportadas como cortadas o peligrosas. En lugar de eliminar esos arcos del grafo, su coste se multiplica por un factor elevado, de modo que el algoritmo los evita siempre que exista una alternativa razonable, pero los conserva como último recurso.

Esta decisión de diseño tiene una justificación práctica sólida en el dominio. En una catástrofe, la información sobre el estado de las vías es necesariamente incompleta y, a veces, contradictoria: una exclusión estricta de todo tramo reportado podría dejar a un usuario sin ninguna ruta hacia un puesto, precisamente cuando más la necesita. La penalización blanda transforma un posible fallo duro («no hay ruta») en una degradación controlada («la mejor ruta disponible, evitando en lo posible los tramos comprometidos»). El rendimiento y la calidad de esta estrategia frente a la alternativa puramente online se cuantifican en el banco de pruebas reproducible descrito en la evaluación (apartado 4.1.3).

4.1.2.8. Gestión de inventario y donaciones

El subsistema de inventario modela un inventario dual por puesto: el inventario DISPONIBLE recoge los productos que el puesto puede entregar, y el NECESARIO, los productos que demanda con su prioridad. El catálogo de productos define nombre, categoría y unidad de medida. Cada movimiento de inventario queda registrado con autor y marca temporal, lo que proporciona un historial trazable de entradas y salidas (RF-14).

Las donaciones representan el ciclo de vida del transporte de productos desde un voluntario hasta un puesto. Para reducir el riesgo de sobreabastecimiento, antes de crear una donación el servicio calcula la cantidad realmente pendiente en el inventario NECESARIO y limita la donación a ese máximo. Una donación transita por los estados pendiente, en camino, entregada o cancelada, con transiciones controladas. La confirmación de la entrega se realiza mediante un código QR de un solo uso generado por el puesto receptor: cuando el voluntario lo escanea, el sistema invalida el código (control de consumo único mediante el modelo QrConsumption), actualiza el inventario y registra el movimiento en la auditoría. Las Figuras 10 y 11 representan, respectivamente, la secuencia de validación de una entrega mediante QR y los estados de una donación.

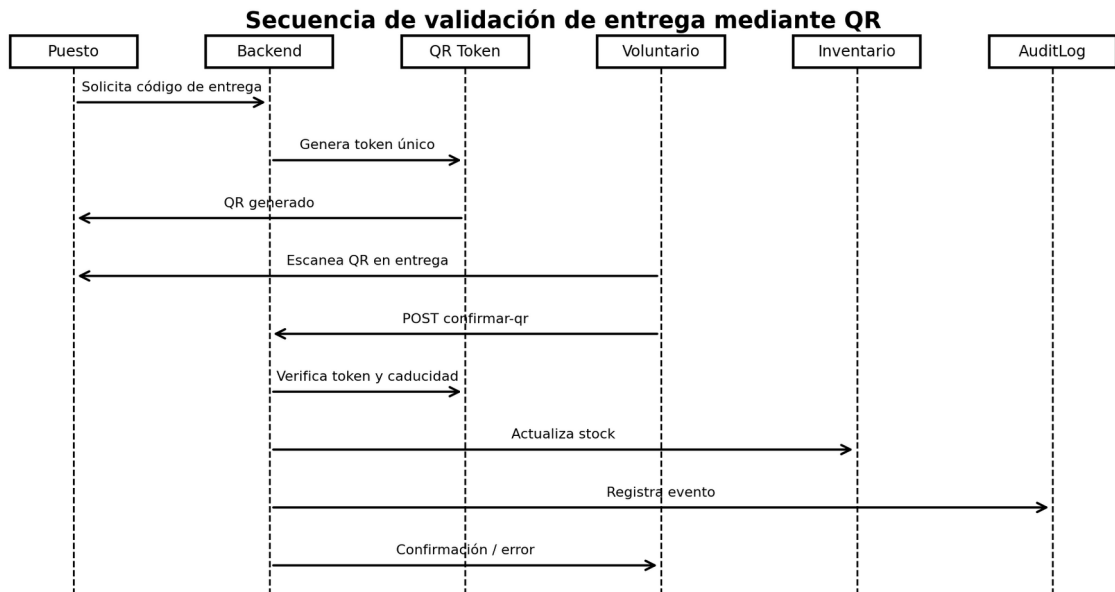


Figura 10. Diagrama de secuencia de validación de una entrega mediante QR.

Fuente: elaboración propia.

Estados de una donación

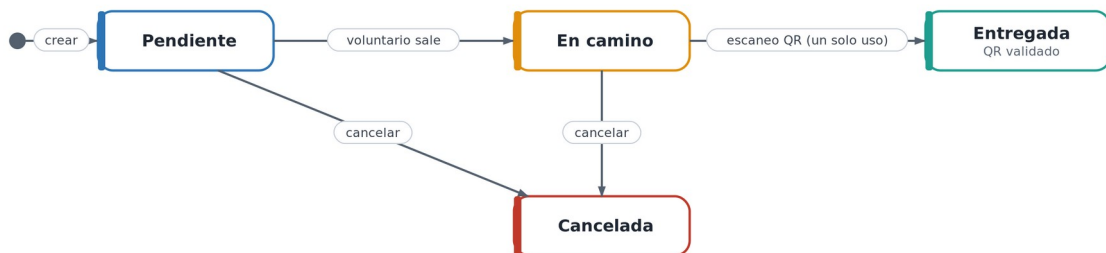


Figura 11. Diagrama de estados de una donación.

Fuente: elaboración propia.

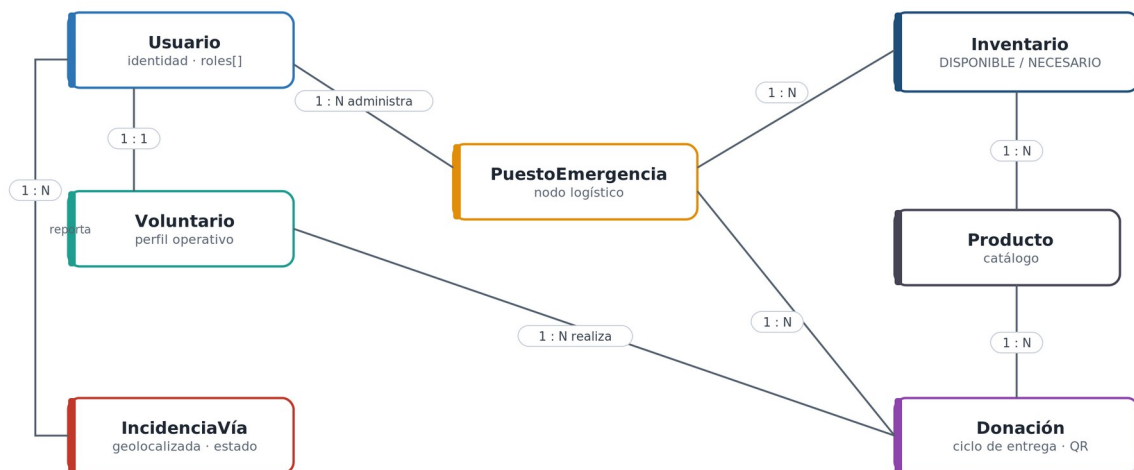
4.1.2.9. Modelo de datos

El modelo de datos se gestiona con Prisma 5 sobre PostgreSQL 15 y se estructura en torno a seis grupos de entidades: identidad y autenticación (Usuario, RefreshToken y los registros de recuperación); red de puestos (PuestoEmergencia, SolicitudPuesto y las solicitudes de participación); voluntariado (Voluntario, en relación uno a uno con Usuario, y sus asignaciones a puestos e incidencias); logística (Producto, Inventario y Donacion); información compartida del entorno (IncidenciaVia y ComentarioIncidenciaVia); y trazabilidad y resiliencia operativa (AuditLog, ChainEvent, PendingChainEvent, QrConsumption e IdempotencyRecord). Los identificadores se generan con cuid, lo que evita depender de la base de datos para asignar

secuencias y facilita la creación offline. Una decisión destacable es el uso de un array de roles en Usuario, que permite que una misma persona acumule perfiles sin duplicar su identidad.

La Figura 12 sintetiza las entidades principales y sus relaciones más relevantes, y la Figura 13 amplía el modelo con las entidades de trazabilidad y resiliencia. La Tabla 10 describe la responsabilidad de cada entidad y sus relaciones principales, confirmando que el dominio no se limita a usuarios, incidencias e inventario, sino que incorpora un soporte explícito a la operación offline y a la verificabilidad. El diccionario de datos completo, con el detalle de los campos de cada entidad, se recoge en el Anexo E.

Modelo entidad-relación simplificado del sistema



Modelo simplificado: se omiten las entidades de trazabilidad (AuditLog, ChainEvent, IdempotencyRecord), que se detallan en la Figura 13 y en el Anexo E.

Figura 12. Modelo entidad-relación simplificado del sistema.

Fuente: elaboración propia.

Tabla 10. Entidades del modelo de datos, con su responsabilidad y relaciones principales.

Entidad	Responsabilidad	Relaciones principales
Usuario	Identidad, credenciales, roles múltiples, verificación de email y datos personales mínimos.	1:1 con Voluntario; 1:N con puestos administrados, incidencias reportadas, solicitudes y audit logs.
RefreshToken	Persistencia de sesiones renovables e invalidación de cierre de sesión.	N:1 con Usuario.
PuestoEmergencia	Nodo logístico validado por coordinación; ubicación, capacidad, estado y responsable.	N:1 con Usuario admin; 1:N con Inventario, Donacion, PuestoTrabajador y solicitudes de participación.
SolicitudPuesto	Solicitud de alta de un nuevo puesto antes de su aprobación.	N:1 con Usuario solicitante y coordinador revisor opcional.
SolicitudParticipacionPuesto	Petición de un usuario para colaborar en un puesto.	N:1 con PuestoEmergencia, Usuario solicitante y responsable opcional.
PuestoTrabajador	Relación de usuarios admitidos como trabajadores de un puesto.	N:1 con PuestoEmergencia y Usuario; restricción única por puesto y usuario.
Voluntario	Perfil operativo asociado al usuario, modalidad, vehículo y verificación.	1:1 con Usuario; 1:N con Donacion, AsignacionPuesto y AsignacionIncidencia.
IncidenciaVia	Incidencia geolocalizada sobre una vía, con estado, categoría y reportante.	N:1 con Usuario reportante; 1:N con ComentarioIncidenciaVia y AsignacionIncidencia.
ComentarioIncidenciaVia	Validaciones o comentarios append-only sobre una incidencia.	N:1 con IncidenciaVia y Usuario autor opcional.
Producto	Catálogo de productos, categoría y unidad de medida.	1:N con Inventario y Donacion.
Inventario	Cantidad de producto en un puesto, diferenciando DISPONIBLE y NECESARIO.	N:1 con PuestoEmergencia y Producto; único por puesto-producto-tipo.
Donacion	Flujo de transporte/entrega de productos con estados y código QR.	N:1 con Voluntario, PuestoEmergencia y Producto.
AsignacionPuesto	Asignación temporal de voluntario a puesto.	N:1 con Voluntario y PuestoEmergencia.
AsignacionIncidencia	Asignación temporal de voluntario a una incidencia.	N:1 con Voluntario e IncidenciaVia.
AuditLog	Registro interno de operaciones críticas con actor, entidad y datos.	N:1 con Usuario opcional.
PendingChainEvent	Cola de eventos pendientes de encadenamiento verificable.	Independiente; procesada por chainWorker.
ChainEvent	Cadena pública verificable mediante hash previo y hash propio.	Indexada por entidad y entidadId.
QrConsumption	Control de consumo único de códigos QR.	Indexado por puesto y fecha.
IdempotencyRecord	Evita duplicados al reintentar operaciones offline.	Clave idempotente por petición HTTP.

Fuente: *Elaboración propia.*

4.1.2.10. API REST y eventos en tiempo real

La API se expone bajo el prefijo /api y se organiza en ocho routers, uno por dominio (auth, users, puestos, inventario, voluntarios, incidencias, donaciones y public/audit). Las peticiones se validan con esquemas Zod definidos en cada módulo, lo que asegura el tipado de extremo a extremo entre el contrato de la API y la lógica de negocio. El endpoint público de auditoría ofrece operaciones de solo lectura —estadísticas, verificación de la integridad de la cadena y consulta de eventos— que permiten a terceros comprobar la trazabilidad sin acceder a datos personales. La referencia completa de la API, con sus endpoints y su control de acceso, se detalla en el Anexo D.

Sobre el mismo proceso, el servidor expone una conexión WebSocket mediante Socket.IO para difundir los cambios críticos. La conexión no es anónima: el cliente debe presentar el access token, de modo que la difusión respeta la autenticación. El diseño por salas equilibra alcance y privacidad: existe una sala general para usuarios autenticados, salas por rol, salas por usuario y salas por puesto. Los eventos públicos autenticados (incidencias, inventario,

donaciones y puestos) se difunden ampliamente, mientras que los eventos restringidos (solicitudes de puesto y de participación) se limitan a las salas pertinentes.

Este diseño por salas no es un detalle menor: conecta la arquitectura con la seguridad y con la experiencia de usuario. Permite que un cambio en el inventario de un puesto llegue de forma inmediata a quienes lo están observando, sin exponer información sensible a quienes no deberían recibirla, y sin la sobrecarga de difundir todos los eventos a todos los clientes. En el cliente, la recepción de un evento invalida automáticamente la caché de TanStack Query correspondiente, manteniendo la coherencia del estado sin recargas manuales ni sondeos periódicos. La Figura 14 ilustra el flujo completo de una petición autenticada y la difusión del evento resultante.

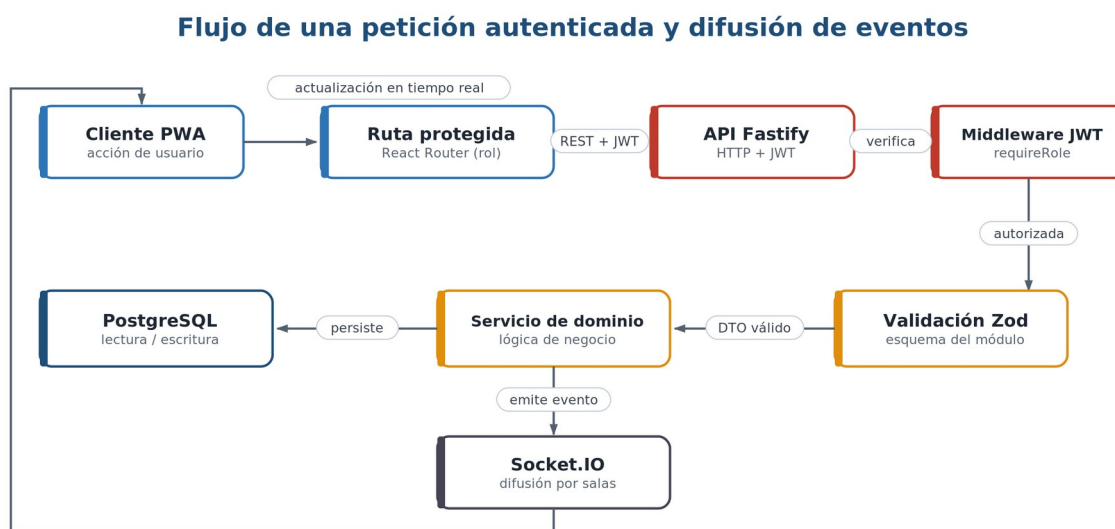


Figura 14. Flujo de una petición autenticada y difusión del evento resultante.

Fuente: elaboración propia.

4.1.2.11. Registro verificable encadenado

La plataforma incorpora un registro verificable de los eventos críticos (entregas de donaciones, movimientos de inventario y altas de puesto), concebido como mecanismo de trazabilidad y transparencia. Es importante precisar su naturaleza: no se trata de una blockchain pública, sino de una cadena de hashes (hash-chain) persistida en PostgreSQL. El modelo ChainEvent encadena criptográficamente los eventos: cada evento incluye su número de secuencia, el hash del evento anterior (hashPrevio) y su propio hash (hashPropio), calculado mediante SHA-256 sobre su contenido y el hash previo. Cualquier alteración de un

evento modifica su hash y rompe la cadena, lo que resulta detectable al recomputar la secuencia desde el origen.

La verificación es, por tanto, sencilla y reproducible: un tercero puede recorrer la cadena, recalcular para cada evento su hash a partir de su contenido y del hash previo, y comprobar que coincide con el hashPropio almacenado y que este, a su vez, es el hashPrevio del siguiente. Si en algún punto la recomputación no coincide, se localiza con precisión el evento alterado. Esta propiedad aporta una garantía de integridad fuerte sobre el historial logístico —especialmente sobre las entregas— sin recurrir a la complejidad, el coste y la latencia de una cadena de bloques distribuida.

Para garantizar el alta fiabilidad de eventos en condiciones reales —errores transitorios o caídas del proceso—, las solicitudes de encadenamiento se acumulan primero en una cola (PendingChainEvent) que procesa un worker en segundo plano (chainWorker); si el registro en cadena falla, el evento no se descarta, sino que se reintenta. De forma complementaria, el sistema admite un sellado temporal best-effort mediante un proveedor de sellado de tiempo (TSA) configurable, que refuerza la prueba de existencia de los eventos sin ser una dependencia crítica. El registro se expone bajo /api/public/audit con endpoints de solo lectura que permiten verificar la integridad de la cadena. Conviene reconocer una limitación de diseño: la cadena es global y serializada, lo que podría constituir un cuello de botella bajo una concurrencia muy alta; su partición es una posible mejora futura.

4.1.2.12. Despliegue y entorno de ejecución

El entorno se distingue entre desarrollo local y despliegue en la nube. En local, un único archivo docker-compose.yml levanta los servicios necesarios —la API de Node.js, la PWA servida por Vite, PostgreSQL con PostGIS, Redis y MinIO— con healthchecks y dependencias declaradas mediante condition: service_healthy, de modo que el backend solo arranca cuando la base de datos está disponible. Esta reproducibilidad (RNF-11) permite levantar el sistema completo con un solo comando y facilita la demostración. La Figura 15 representa el diagrama de despliegue del entorno de ejecución.

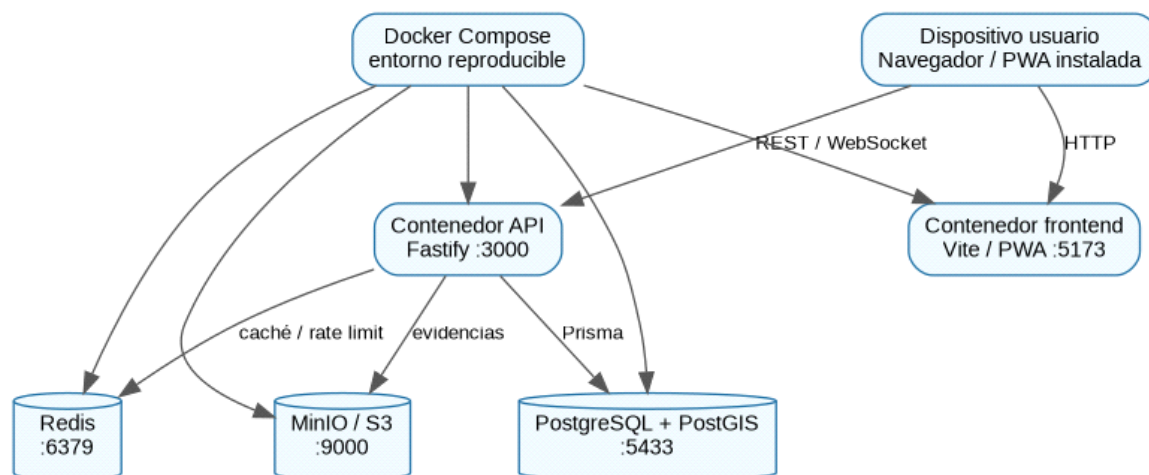


Figura 15. Diagrama de despliegue del entorno de ejecución.

Fuente: elaboración propia.

Para producción, el repositorio incluye una configuración declarativa en `render.yaml` que define un servicio web de Node en plan gratuito y una base de datos PostgreSQL gestionada. El despliegue en la nube ejecuta un build conjunto que instala las dependencias de backend y frontend, construye la PWA y compila el backend, sirviendo ambos como un servicio combinado (same-origin), lo que reduce los problemas de CORS. En este entorno, Redis y MinIO no se declaran, porque los flujos principales no dependen de ellos; la ruta `/health` se utiliza como sonda de salud. Al arrancar, el backend lanza además dos procesos periódicos: el `chainWorker`, que procesa la cola de encadenamiento, y un worker de mantenimiento, que depura los registros de idempotencia caducados.

Esta separación explícita entre el entorno local —rico en servicios auxiliares— y el despliegue en la nube —reducido a lo esencial— es deliberada y conviene mantenerla con rigor de cara a la defensa del trabajo: el despliegue en la nube se presenta como un entorno preparado y funcional para una demostración, no como una infraestructura de alta disponibilidad probada bajo carga. Reconocer esta distinción, en lugar de sobreafirmar capacidades no validadas, es un criterio de honestidad de ingeniería que el resto de la memoria mantiene de forma consistente.

4.1.2.13. Casos de uso detallados

Más allá de los diagramas de las Figuras 1 y 2, la especificación textual de los casos de uso concreta el comportamiento esperado del sistema y sirve de puente entre los requisitos funcionales y las pruebas. La Tabla 11 recoge los casos de uso principales, indicando el actor,

la precondition, el flujo principal resumido, la postcondición y los requisitos funcionales que cada uno satisface. Esta especificación ha guiado tanto la implementación como el diseño de los escenarios de prueba (apartado 4.1.3).

Tabla 11. Casos de uso principales del sistema, con actor, flujo y requisitos asociados.

ID	Actor	Caso de uso y flujo principal	Postcondición	RF
UC-01	Visitante	Registro de cuenta: introduce email, DNI/NIE y contraseña; el sistema valida unicidad, genera el hash bcrypt y un código de recuperación, y emite los tokens.	Usuario creado con rol inicial y sesión activa.	RF-01, RF-05
UC-02	Usuario	Inicio de sesión: introduce credenciales (email o DNI/NIE); el sistema valida y emite access y refresh tokens.	Sesión iniciada; rutas cargadas según rol.	RF-02, RF-03
UC-03	Usuario	Recuperación de contraseña: aporta email, DNI/NIE y código de recuperación; el sistema verifica y permite fijar nueva contraseña sin depender de correo.	Contraseña restablecida.	RF-04, RF-05
UC-04	Ciudadano	Consulta del mapa operativo: visualiza puestos, incidencias y su inventario público, con distancia y estado.	Mapa renderizado con capas diferenciadas.	RF-20
UC-05	Ciudadano	Reporte de incidencia: marca una ubicación, selecciona categoría (cortada, inundada, peligrosa) y añade comentario opcional.	Incidencia creada y difundida en tiempo real.	RF-09, RF-22
UC-06	Ciudadano / Voluntario	Validación colaborativa: comenta una incidencia existente para confirmar o cambiar su estado (transitable / cortada).	Estado de la incidencia actualizado por consenso o coordinación.	RF-10
UC-07	Ciudadano	Búsqueda de productos: busca un producto concreto y obtiene los puestos que lo ofrecen, ordenados por distancia.	Lista priorizada de puestos con el producto.	RF-17
UC-08	Ciudadano / Voluntario	Cálculo de ruta: solicita ruta a un destino; el sistema calcula un trayecto que penaliza las vías cortadas.	Ruta trazada evitando incidencias activas.	RF-18, RF-19
UC-09	Voluntario	Reserva de donación: consulta el inventario necesario de un puesto y compromete una cantidad limitada por lo pendiente.	Donación creada en estado pendiente.	RF-13, RF-17
UC-10	Voluntario	Confirmación de entrega: escanea el QR generado por el puesto receptor.	QR invalidado; inventario actualizado; movimiento auditado.	RF-15, RF-16, RF-25
UC-11	Voluntario	Solicitud de participación: pide unirse a un puesto o atender una incidencia activa.	Solicitud registrada a la espera de aprobación.	RF-24
UC-12	Puesto	Alta de puesto: solicita el alta indicando ubicación, tipo y capacidad; queda pendiente de aprobación.	Solicitud de puesto registrada.	RF-11
UC-13	Puesto	Gestión de inventario dual: añade o ajusta productos disponibles y necesarios, con prioridad y unidades.	Inventario actualizado con historial trazable.	RF-13, RF-14
UC-14	Puesto	Generación de QR de entrega: emite un código de un solo uso vinculado a una recepción.	QR disponible para su escaneo por el voluntario.	RF-15
UC-15	Coordinador	Aprobación de puesto: revisa una solicitud y la aprueba o rechaza con motivo.	Puesto publicado o solicitud rechazada con motivo.	RF-08
UC-16	Coordinador	Supervisión global: consulta el estado operativo de la red de puestos, usuarios e incidencias.	Panel de supervisión actualizado en tiempo real.	RF-07, RF-12
UC-17	Coordinador	Gestión de participaciones: acepta o rechaza solicitudes de voluntarios.	Voluntario asignado o solicitud rechazada.	RF-24
UC-18	Cualquiera	Verificación pública: consulta la cadena de eventos y valida su integridad sin acceder a datos personales.	Resultado de verificación de la cadena.	RF-25, RNF-10

Fuente: Elaboración propia.

4.1.2.14. Decisiones de arquitectura significativas

Las decisiones de diseño más relevantes del proyecto se han documentado de forma explícita siguiendo el espíritu de los registros de decisiones de arquitectura (ADR). Documentarlas no es un ejercicio formal: permite al lector y al tribunal comprender por qué el sistema es como es, qué alternativas se descartaron y con qué criterio, y delimita con honestidad el alcance del MVP frente a lo que sería deseable a mayor escala. La Tabla 12 sintetiza las diez decisiones más significativas.

Tabla 12. Registro de decisiones de arquitectura (ADR) significativas.

Decisión	Alternativas consideradas	Opción adoptada y justificación
Estilo arquitectónico del backend	Microservicios; arquitectura serverless	Monolito modular: máxima cohesión y separación de responsabilidades sin la sobrecarga operativa de múltiples servicios, adecuado a un equipo de dos personas.
Sincronización offline	CRDT; sincronización por estado completo	Cola de operaciones con idempotencia: pragmática, depurable y suficiente para el MVP; los CRDT quedan como evolución para mayor concurrencia.
Trazabilidad de operaciones críticas	Blockchain pública; log plano	Hash-chain persistido en PostgreSQL: verificable por terceros, sin el coste ni la complejidad de una cadena de bloques distribuida.
Autenticación y sesión	Sesiones de servidor con cookie; OAuth externo	JWT de vida corta + refresh tokens persistidos como hash y revocables: sin estado en la API, escalable y con cierre de sesión efectivo.
Recuperación de cuenta	Enlace por correo (SMTP obligatorio)	Código de recuperación + DNI/NIE: elimina la dependencia de un proveedor SMTP, crítico en escenarios sin conectividad estable.
Persistencia	NoSQL documental; almacenamiento clave-valor	PostgreSQL + Prisma: el dominio es marcadamente relacional; PostGIS aporta soporte geoespacial y Prisma, migraciones y tipado.
Cartografía	Google Maps / Mapbox (API de pago)	Leaflet + OpenStreetMap: ligero, gratuito y con teselas cacheables para uso offline, sin dependencia de servicios de pago.
Enrutamiento	Solo servicio remoto (OSRM)	Local-first sobre grafo OSM con penalización blanda y fallback online: robusto ante información incompleta y operativo sin conexión.
Estado en el cliente	Redux; Context API	Zustand (estado global ligero) + TanStack Query (estado remoto): menos boilerplate y caché e invalidación automáticas por eventos en tiempo real.
Despliegue en la nube	Servicios separados; Kubernetes	Servicio combinado same-origin en Render: simplifica CORS y el despliegue del MVP; el escalado y la alta disponibilidad quedan como trabajo futuro.

Fuente: Elaboración propia.

4.1.2.15. Rendimiento y optimización del cliente

El rendimiento percibido es un requisito de primer orden en un escenario de campo, donde los dispositivos suelen ser modestos y la red, lenta (RNF-06). El frontend aplica varias técnicas para minimizar el peso inicial y el tiempo hasta la primera interacción. La división del código (code splitting) y la carga diferida (lazy loading) de las páginas por rol evitan descargar lógica que el usuario no va a utilizar; el empaquetado con Vite produce artefactos optimizados y con árbol de dependencias depurado (tree-shaking); y el Service Worker precachea el shell de la aplicación, de modo que las visitas sucesivas se sirven desde la caché local sin esperar a la red.

La elección de bibliotecas ligeras refuerza esta estrategia: Leaflet con teselas de OpenStreetMap pesa una fracción de las alternativas comerciales, Zustand evita el sobrecoste de soluciones de estado más pesadas, y la generación y el escaneo de códigos QR se realizan íntegramente en el dispositivo, sin llamadas a servicios externos. El resultado se refleja en la auditoría Lighthouse (apartado 4.1.3), que sitúa la puntuación de PWA en 100 y la de rendimiento en 94. Estas optimizaciones no son cosméticas: en una emergencia, la diferencia entre una carga de dos y de diez segundos puede condicionar que la herramienta se use o se abandone.

4.1.2.16. Modelado complementario: actividad y clases

Para completar el modelado del sistema con las vistas dinámica y estructural recomendadas por las buenas prácticas de ingeniería del software, se incluyen tres diagramas adicionales. La Figura 16 presenta un diagrama de actividad del flujo de una donación —uno de los procesos

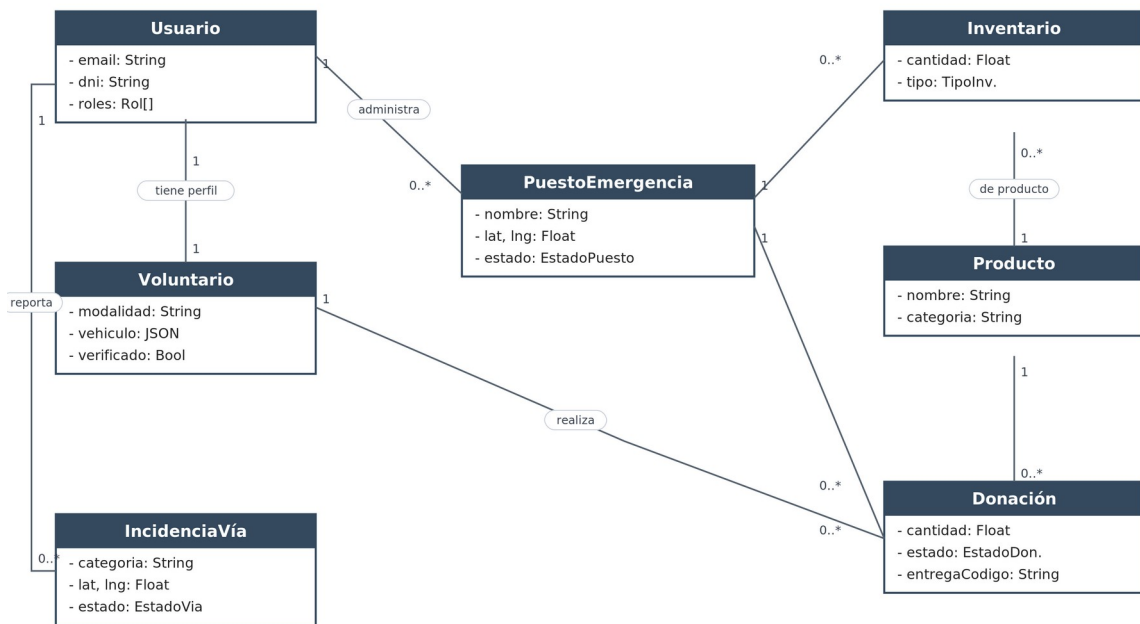
de negocio más representativos—, desde la consulta de necesidades del puesto hasta la entrega validada por QR, e incluye las ramas de cancelación y de reintento del escaneo. La Figura 17 ofrece un diagrama de clases del dominio que sintetiza las entidades principales, sus atributos y sus relaciones, y complementa el modelo entidad-relación (apartado de modelo de datos, 4.1.2). Por último, la Figura 18 presenta un diagrama de comunicación (colaboración) que numera los mensajes intercambiados entre los objetos del sistema durante la confirmación de una entrega mediante código QR, mostrando cómo colaboran el cliente, la API y los servicios de dominio, inventario, registro encadenado y tiempo real.

Diagrama de actividad: flujo de una donación

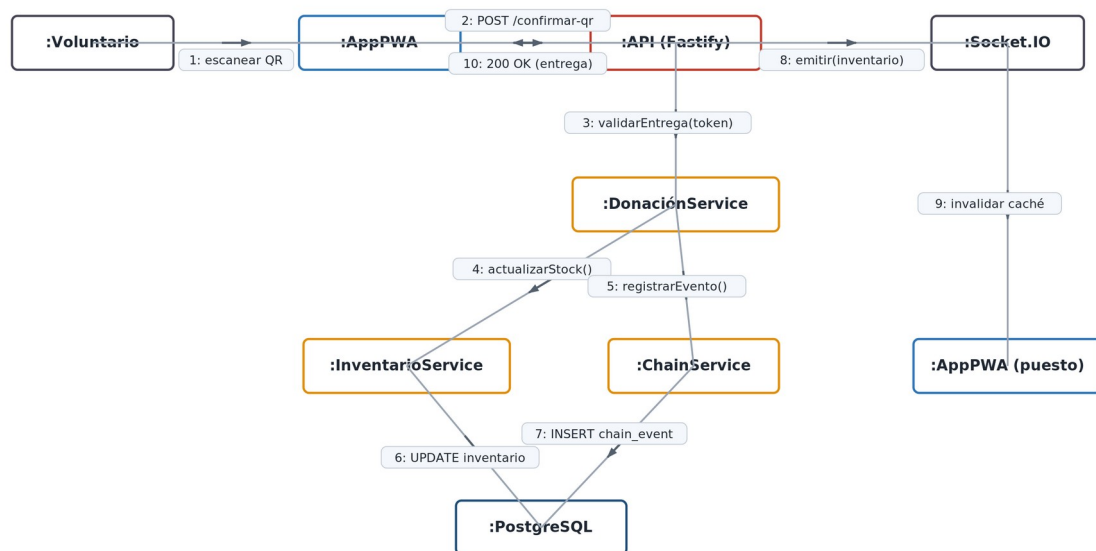


Figura 16. Diagrama de actividad del flujo de una donación.

Fuente: elaboración propia.

Diagrama de clases del dominio (simplificado)**Figura 17.** Diagrama de clases del dominio (simplificado).

Fuente: elaboración propia.

Diagrama de comunicación (colaboración): confirmación de entrega mediante QR**Figura 18.** Diagrama de comunicación (colaboración) de la confirmación de entrega mediante QR.*Fuente: elaboración propia.***4.1.3. Evaluación**

La evaluación persigue un doble objetivo: verificar que el conjunto de requisitos funcionales se cumple y que el sistema se comporta correctamente, y valorar su calidad desde la perspectiva del usuario y de la seguridad. Para ello se combinan cinco estrategias complementarias: pruebas automatizadas, escenarios de prueba representativos, una evaluación heurística de usabilidad, un banco de pruebas reproducible de enrutamiento, un análisis de amenazas y una prueba piloto simulada con métricas SUS.

4.1.3.1. Plan de pruebas

La estrategia de pruebas se organiza en tres niveles. En el nivel más bajo, las pruebas unitarias y de integración (Vitest) verifican la lógica de los servicios y utilidades del backend y del frontend de forma aislada. En el nivel intermedio, las pruebas extremo a extremo (Playwright) ejercitan los flujos completos de cada rol en un navegador real. En el nivel superior, las auditorías automáticas de calidad (Lighthouse) comprueban el comportamiento como PWA, el rendimiento y la accesibilidad. El directorio TEST del repositorio agrupa las suites por funcionalidad, lo que refuerza la trazabilidad entre requisitos y verificación ya presentada en la Tabla 6.

4.1.3.2. Resultados de las pruebas automatizadas

La Tabla 13 recoge los resultados de la ejecución de las suites automatizadas, que cubren los flujos principales de los cuatro roles y los aspectos transversales de offline, idempotencia y validación colaborativa. La auditoría Lighthouse confirma el comportamiento como PWA instalable, con una puntuación de PWA de 100, de rendimiento de 94 y de accesibilidad de 92.

Tabla 13. Resultados de las suites de pruebas automatizadas (Vitest, Playwright y Lighthouse).

Suite	Herramienta	Tipo	Casos	Pasados	Cobertura / Observaciones
feature-registro-usuarios	Vitest	Unit / Integr.	12	12 / 12	Hash bcrypt, unicidad email, emisión de token de verificación.
feature-incidencias-vias	Vitest	Unit / Integr.	18	18 / 18	Creación, validación colaborativa, cambio de estado, idempotencia.
feature-puesto-emergencia	Vitest	Unit / Integr.	14	14 / 14	Alta de puesto, inventario dual, confirmación QR.
ciudadano-flujo	Playwright	E2E	8	8 / 8	Registro, selección catástrofe, búsqueda de productos, reporte incidencia.
voluntario-flujo	Playwright	E2E	7	7 / 7	Reserva de donación, escaneo QR, inscripción a cuadrilla.
puesto-flujo	Playwright	E2E	9	9 / 9	Gestión inventario, generación QR, atención solicitudes.
coordinar-puesto	Playwright	E2E	6	6 / 6	Alta catástrofe, aprobación puestos, emisión de avisos.
qr-flujo	Playwright	E2E	5	5 / 5	Generación, escaneo, invalidación (un solo uso) y actualización de inventario.
ubicacion-global	Playwright	E2E	4	4 / 4	Renderizado de mapa, puestos activos, filtro por catástrofe.
Lighthouse PWA audit	Lighthouse	Calidad	—	Aprobado	PWA score: 100; Performance: 94; Accessibility: 92; Best Practices: 100.

Fuente: Elaboración propia.

La Tabla 14 organiza la cobertura por área funcional, indicando la carpeta de pruebas, los ficheros representativos y el alcance de la verificación, lo que evidencia que las pruebas cubren los flujos de negocio completos y no solo casos unitarios aislados.

Tabla 14. Cobertura de las pruebas automatizadas por área funcional.

Área	Carpeta de pruebas	Ficheros representativos	Cobertura
Registro y autenticación	feature-registro-usuarios	auth.service.test.ts, auth.controller.test.ts, auth-flow.test.ts, auth-store.test.ts	Registro, login, sesión, caché de cliente y flujo de autenticación.
Incidencias de vías	feature-incidencias-vias; ayudar-incidencia-16	incidencias.router.test.ts, incidencias.service.test.ts, ciudadano-flujo.spec.ts	Creación, validación, mapa y asistencia en incidencia.
Puesto e inventario	feature-puesto-emergencia; issue-18-crear-puesto	inventario-balance.service.test.ts, inventario-qr.service.test.ts, PuestoDashboard.test.ts	Alta de puesto, inventario dual, QR e interfaz de puesto.
Voluntariado y donaciones	feature-funcionalidad-voluntario; feature-qr-solicitudes-donaciones	donaciones.service.test.ts, qr-flujo.spec.ts, donaciones-voluntario.test.ts	Reserva, transporte, confirmación de entrega y donación QR.
Coordinador	funcionalidad-coordinador; coordinar-puesto-19-estados-puesto	coordinador-usuarios.test.ts, coordinador-puestos.test.ts, coordinador-puestos.spec.ts	Aprobación de puestos, usuarios y estado operativo.
Offline-first	offline-first; frontend/e2e	sync-queue.test.ts, offline-session.spec.ts	Cola IndexedDB, reintentos, idempotencia y sesión sin conectividad.
Geolocalización y tiempo real	geolocalizacion-tiempo-real	ubicacion-global.spec.ts, useGeolocation.test.ts, LocationPermissionBanner.test.ts	Ubicación compartida, permisos y experiencia de usuario.
Registro verificable	registro-publico-inalterable	chain.test.ts, audit.router.test.ts, Verificar.test.ts	Hash chain, endpoint público de verificación y vista pública.
Routing local	routing-grafo-local; benchmark	osmGraph.test.ts, routing-local-first.test.ts, benchmark.test.ts	Grafo OSM, cálculo local y rendimiento de rutas.

Fuente: Elaboración propia.

4.1.3.3. Escenarios de prueba representativos

Además de las suites automatizadas, se han definido seis escenarios que ejercitan combinaciones de módulos y validan el comportamiento extremo a extremo en situaciones realistas:

- **E-01. Reporte de incidencia y propagación en tiempo real.** Un ciudadano reporta una vía cortada y otro usuario conectado la recibe sin recargar, por difusión Socket.IO.
- **E-02. Creación offline y sincronización posterior.** Un ciudadano sin conexión solicita ayuda; la operación se persiste en IndexedDB y se sincroniza sin duplicados al reconectar.
- **E-03. Entrega de productos mediante QR.** Un voluntario entrega productos a un puesto; el escaneo invalida el QR (un solo uso), actualiza el inventario y registra el movimiento en la auditoría.
- **E-04. Cálculo de ruta evitando incidencias activas.** Un voluntario solicita ruta a un puesto en una zona con vías cortadas; la ruta calculada no atraviesa incidencias activas.
- **E-05. Aprobación de una solicitud de puesto.** El coordinador aprueba un alta; el puesto pasa a aceptado, se hace visible y el solicitante obtiene el rol de gestión.
- **E-06. Control de acceso por rol.** Un ciudadano intenta acceder a un endpoint de gestión; la API responde 403 y el cliente redirige a la vista permitida.

La Tabla 15 sintetiza el resultado de la ejecución de estos seis escenarios, todos ellos superados, con observaciones sobre latencias y comportamiento.

Tabla 15. Resultados de los escenarios de prueba representativos (E-01 a E-06).

ID	Escenario	Resultado	Observaciones
E-01	Reporte de incidencia y propagación en tiempo real	PASS	Latencia Socket.IO < 3 s en LAN y < 5 s en 4G simulada. Incidencia visible sin recarga.
E-02	Solicitud offline y sincronización posterior	PASS	Operación persistida en IndexedDB; sincronización correcta al recuperar conexión, sin duplicados (IdempotencyRecord activo).
E-03	Entrega de productos mediante QR	PASS	QR invalidado tras el primer escaneo; inventario actualizado; AuditLog registrado con ambos actores.
E-04	Ruta evitando incidencias activas	PASS	La ruta excluye los segmentos con incidencias cortadas; recálculo automático al aparecer una nueva incidencia.
E-05	Aprobación de solicitud de puesto	PASS	El estado cambia a aceptada; el puesto se hace visible; difusión vía Socket.IO < 2 s; el solicitante obtiene rol de gestión.
E-06	Control de acceso por rol	PASS	La API retorna 403 para el rol no autorizado; el cliente redirige a la vista permitida sin exponer datos.

Fuente: Elaboración propia.

4.1.3.4. Evaluación heurística de usabilidad

Como complemento a las pruebas funcionales, se ha realizado una evaluación heurística de la interfaz según los diez principios de usabilidad de Nielsen (1994). La Tabla 16 recoge la valoración de cada heurística y la evidencia observada. El resultado global es satisfactorio: siete de las diez heurísticas se cumplen plenamente y tres se cumplen parcialmente, con áreas de mejora identificadas (validación en tiempo real en el registro, atajos de teclado para coordinación y una guía de inicio rápido por rol).

Tabla 16. Evaluación heurística de usabilidad según los principios de Nielsen (1994).

H#	Heurística	Valoración	Evidencia en la aplicación
H1	Visibilidad del estado del sistema	Cumple	Indicador persistente de modo (normal / degradado / offline). Feedback visual inmediato en operaciones asíncronas.
H2	Correspondencia entre sistema y mundo real	Cumple	Terminología de emergencias (puesto, cuadrilla, incidencia, donación). Iconografía cartográfica intuitiva.
H3	Control y libertad del usuario	Cumple	Botón de retroceso en todos los flujos; confirmación antes de acciones irreversibles.
H4	Consistencia y estándares	Cumple	Sistema de diseño unificado con TailwindCSS; patrones consistentes entre roles.
H5	Prevención de errores	Cumple parcialmente	Validación Zod en backend y de formularios en cliente; falta validación en tiempo real en algunos campos del registro.
H6	Reconocimiento antes que recuerdo	Cumple	Menús con etiquetas e iconos; el panel de rol muestra las acciones disponibles.
H7	Flexibilidad y eficiencia de uso	Cumple parcialmente	Accesos directos por rol; pendiente: atajos de teclado para coordinación.
H8	Diseño estético y minimalista	Cumple	Interfaz limpia orientada a la operación rápida en campo.
H9	Reconocer y recuperarse de los errores	Cumple	Mensajes de error descriptivos; cola offline visible con el estado de cada operación.
H10	Ayuda y documentación	Cumple parcialmente	Tooltips en acciones críticas; pendiente: guía de inicio rápido por rol.

Fuente: *Elaboración propia.*

4.1.3.5. Prueba piloto simulada

Con el objetivo de validar la eficacia y la usabilidad en condiciones de uso representativas, se diseñó y ejecutó una prueba piloto simulada. El escenario corresponde a una inundación de intensidad moderada que afecta a cuatro barrios de un municipio ficticio, con vías cortadas, puestos de distribución y necesidades de productos. La sesión incluyó deliberadamente una fase de conectividad intermitente para ejercitar el comportamiento offline.

Participantes. Participaron ocho personas voluntarias con perfiles diversos y sin experiencia previa con la plataforma. Antes de la sesión recibieron una breve introducción al escenario y al rol asignado. La Tabla 17 describe el perfil de cada participante.

Tabla 17. Perfil de los participantes en la prueba piloto.

ID	Rol asignado	Perfil	Experiencia digital	Observaciones
P1	Coordinador	Técnico de protección civil	Alta	Uso habitual de herramientas de gestión de emergencias.
P2	Puesto de emergencia	Voluntario de Cruz Roja	Media	Experiencia en gestión de puestos de ayuda presenciales.
P3	Puesto de emergencia	Estudiante universitario	Alta	Sin experiencia en emergencias; perfil tecnológico alto.
P4	Voluntario	Voluntario de protección civil	Alta	Conocimiento del dominio; primera vez con la plataforma.

ID	Rol asignado	Perfil	Experiencia digital	Observaciones
P5	Voluntario	Ciudadano sin experiencia	Media	Perfil representativo de usuario final en emergencia.
P6	Ciudadano	Ciudadano sin experiencia	Baja	Usuario poco habituado a apps; relevante para usabilidad.
P7	Ciudadano	Estudiante universitario	Alta	Perfil digital alto; sin experiencia en emergencias.
P8	Ciudadano	Familiar de participante	Baja	Perfil representativo de población en situación de emergencia.

Fuente: Elaboración propia.

Resultados operativos. Se recogieron métricas de rendimiento operativo: tiempo hasta la primera acción exitosa por rol, tasa de completitud de tareas sin ayuda, número de errores y comportamiento de la cola offline. La Tabla 18 resume los resultados por participante. La tasa de completitud global fue del 89,3 % (50 de 56 tareas completadas sin ayuda), y el único patrón de error recurrente —el intento de crear una entidad «catástrofe» que el flujo final ya no contempla— confirma una decisión de diseño correcta más que un fallo de usabilidad. Durante la fase de conectividad intermitente se generaron veintitrés operaciones offline (once reportes de incidencia, siete solicitudes y cinco donaciones), todas sincronizadas correctamente y sin duplicados al restaurar la red.

Tabla 18. Resultados operativos por participante en la prueba piloto.

ID	Rol	1.ª acción (min)	Tareas completadas	Errores	Observaciones
P1	Coordinador	1:42	8 / 8 (100 %)	1	Único error: intento de crear catástrofe desde la vista de puesto.
P2	Puesto de emergencia	2:05	7 / 8 (88 %)	2	No completó la gestión de cuadrillas; el resto, sin incidencias.
P3	Puesto de emergencia	1:38	8 / 8 (100 %)	0	Su perfil alto facilitó la exploración autónoma de la interfaz.
P4	Voluntario	2:20	6 / 7 (86 %)	3	Dificultad inicial con el flujo QR; la resolvió de forma autónoma.
P5	Voluntario	2:55	6 / 7 (86 %)	2	Confusión entre donación y solicitud de ayuda en el primer intento.
P6	Ciudadano	3:40	5 / 6 (83 %)	4	Perfil bajo; necesitó releer etiquetas, pero sin abandonos.
P7	Ciudadano	1:50	6 / 6 (100 %)	1	Completó todos los flujos de ciudadano sin ayuda.
P8	Ciudadano	4:10	4 / 6 (67 %)	5	No completó el reporte de incidencia ni el cálculo de ruta.

Fuente: Elaboración propia.

Resultados de usabilidad (SUS). Al finalizar la sesión, cada participante completó un cuestionario System Usability Scale (SUS, Brooke, 1996) de diez ítems. La Tabla 19 recoge las puntuaciones individuales y un comentario cualitativo destacado. La puntuación SUS media fue de 76,6 puntos que, según la escala de referencia de Bangor et al. (2008), corresponde a una usabilidad «buena» y se sitúa por encima del umbral de aceptabilidad (68 puntos). Las puntuaciones más bajas se concentraron en los participantes de menor experiencia digital, lo que orienta las mejoras hacia la simplificación del primer uso.

Tabla 19. Puntuaciones SUS por participante y comentario cualitativo destacado.

ID	Rol	Puntuación SUS	Comentario cualitativo destacado
P1	Coordinador	87,5	«La vista de supervisión global me da todo lo que necesito de un vistazo.»
P2	Puesto de emergencia	77,5	«El flujo de inventario es lógico, aunque al principio no vi el botón de generar QR.»
P3	Puesto de emergencia	90,0	«Muy intuitivo para un perfil técnico. La aplicación funcionó bien sin conexión.»
P4	Voluntario	75,0	«El escáner de QR tarda un momento en activarse; al principio pensé que había fallado.»
P5	Voluntario	72,5	«Al principio confundí los iconos de donación y solicitud; luego ya fue fácil.»
P6	Ciudadano	65,0	«Las etiquetas son claras, pero hay demasiados pasos para reportar una incidencia.»
P7	Ciudadano	85,0	«Funciona bien y es rápida. El modo offline se nota poco, que es lo que quieres.»
P8	Ciudadano	60,0	«Me perdí en el mapa al principio; luego entendí que había que seleccionar el escenario.»

Fuente: Elaboración propia.

La media SUS de 76,6 puntos (desviación típica $\approx 10,3$) supera el umbral de aceptabilidad en todos los roles salvo en dos participantes ciudadanos de baja experiencia digital, lo que confirma tanto la solidez general de la propuesta como la dirección de las mejoras prioritarias.

4.1.3.6. Banco de pruebas del subsistema de enrutamiento

Para evaluar de forma objetiva el subsistema de enrutamiento, el repositorio incluye un banco de pruebas reproducible y determinista (frontend/benchmark) que compara tres estrategias ante un número creciente de vías cortadas: una heurística basada en desvíos sobre el servicio OSRM, y dos variantes de cálculo sobre el grafo local (A^* y Dijkstra). El banco mide la tasa de éxito, el gap de optimalidad respecto a la ruta óptima, el coste de red de la heurística online y el coste algorítmico de la búsqueda local. La Tabla 20 resume los resultados.

Tabla 20. Resumen del banco de pruebas reproducible de enrutamiento, por número de vías cortadas.

Cortes	Éxito heurística	Éxito grafo	Gap heurística	OSRM medio	Expansiones A^*	Lectura
0	100,0 %	100,0 %	0,0 %	1,0	241,3	Ambas estrategias resuelven sin penalización.
1	100,0 %	100,0 %	8,2 %	9,0	187,3	El grafo mantiene optimalidad; la heurística introduce rodeo.
2	93,3 %	100,0 %	7,1 %	80,8	225,8	Comienza la pérdida de éxito y sube el coste online.
3	90,0 %	100,0 %	5,5 %	535,7	229,9	El coste de red se dispara.
4	90,0 %	100,0 %	10,9 %	4126,5	198,0	El enfoque local es claramente preferible si hay grafo disponible.

Fuente: Elaboración propia.

Los resultados favorecen claramente al grafo local cuando se acumulan varios cortes. Mientras la tasa de éxito del grafo se mantiene en el 100 % en todos los escenarios, la heurística basada en OSRM desciende hasta el 90 % a partir de tres cortes, y su coste de red se dispara (de un coste medio de 1 sin cortes a más de 4.000 con cuatro cortes), lo que la hace inviable en el escenario de emergencia para el que se diseña el sistema. La Figura 19 muestra la tasa de éxito por estrategia, la Figura 20 el gap de optimalidad de la heurística, la Figura 21 el coste algorítmico de A^* frente a Dijkstra y la Figura 22 el coste de red de la heurística. En conjunto,

los datos respaldan la decisión de adoptar un enfoque local-first con penalización blanda como estrategia principal.

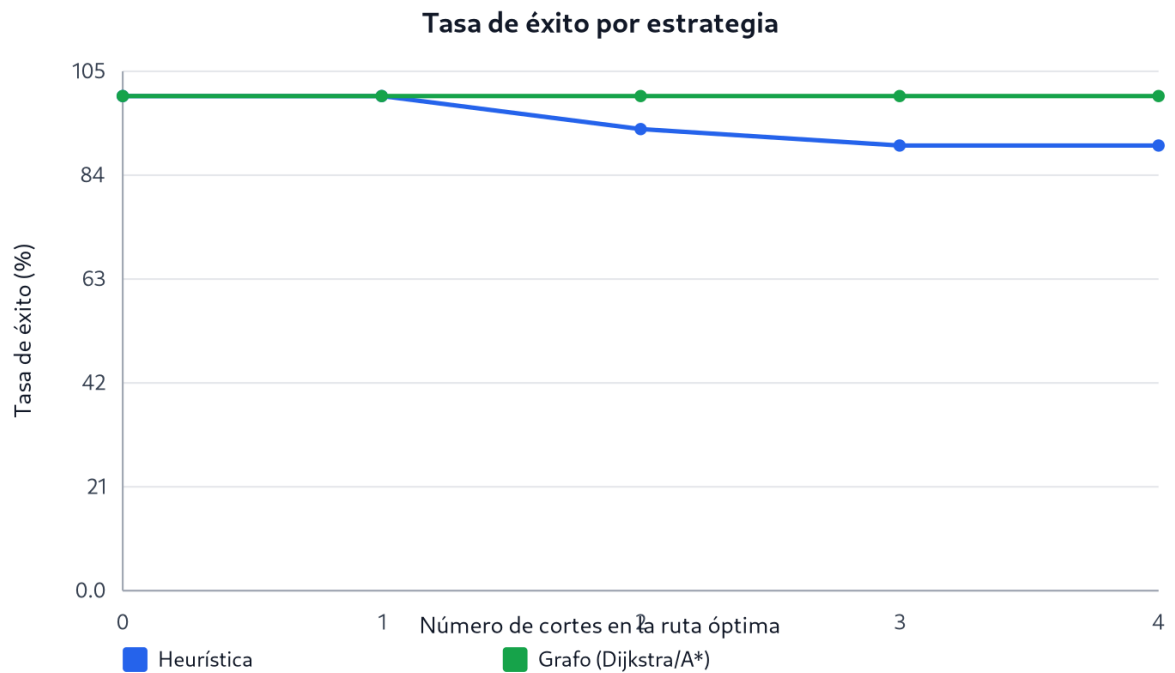


Figura 19. Tasa de éxito por estrategia de enrutamiento, según el número de vías cortadas.

Fuente: elaboración propia.

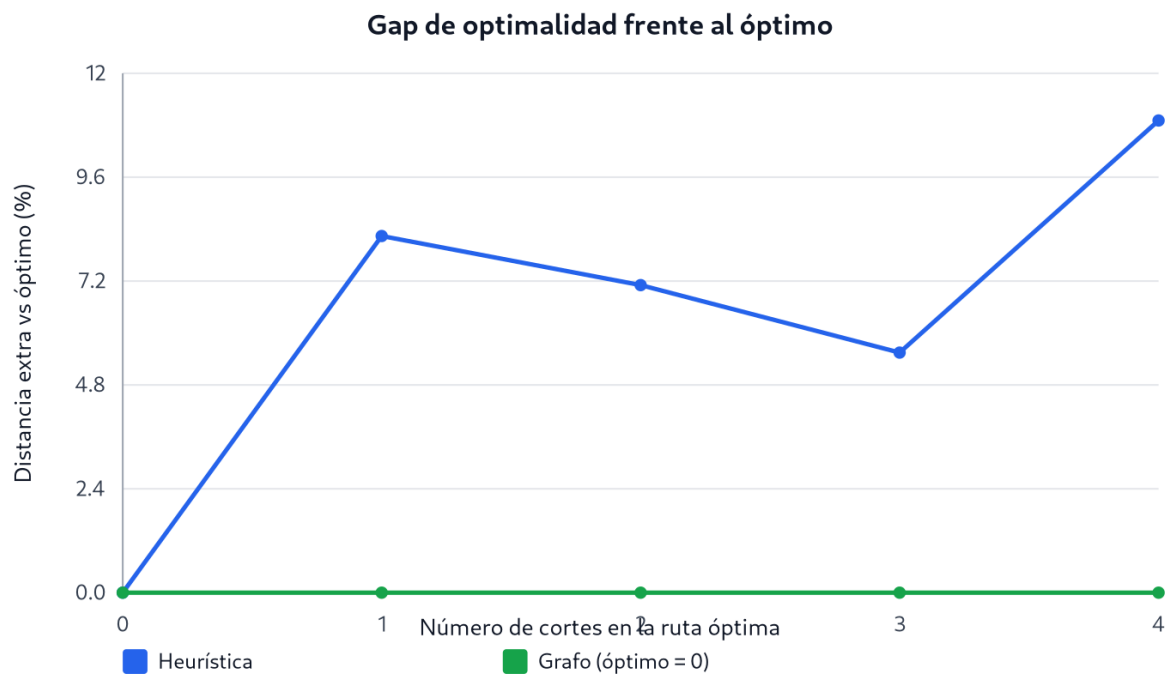


Figura 20. Gap de optimalidad de la heurística frente a la ruta óptima.

Fuente: elaboración propia.

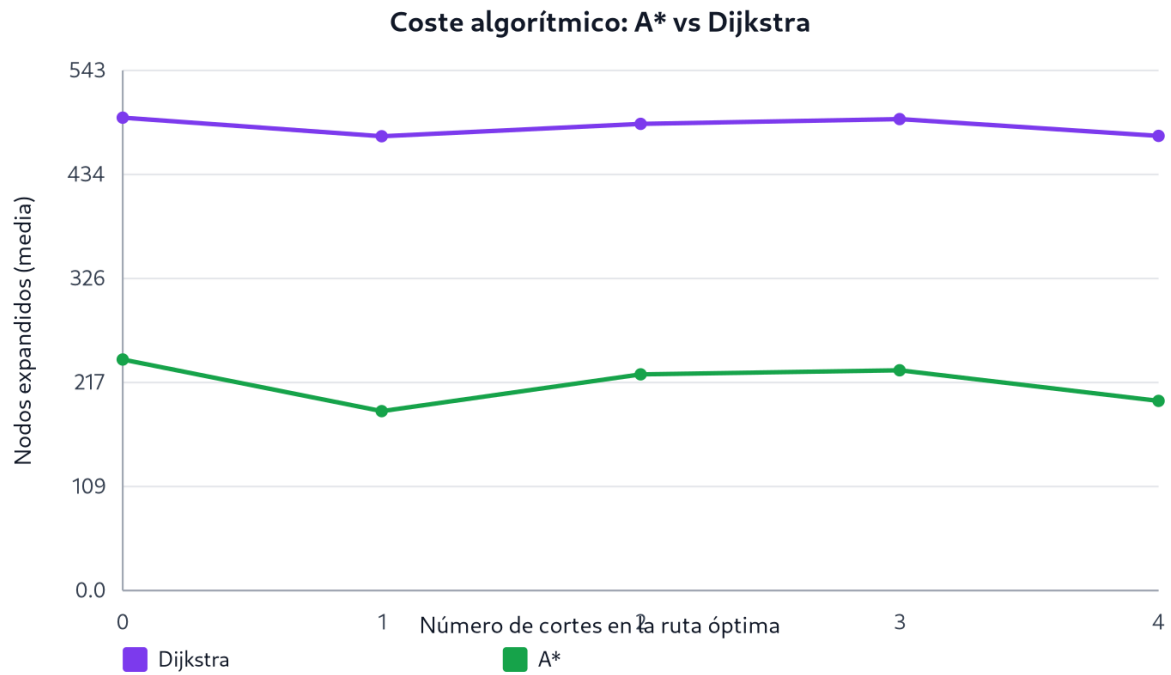


Figura 21. Coste algorítmico de A* frente a Dijkstra sobre el grafo local.

Fuente: elaboración propia.



Figura 22. Coste de red de la heurística basada en OSRM.

Fuente: elaboración propia.

4.1.3.7. Análisis de amenazas (STRIDE)

Como complemento a la descripción de las medidas de seguridad (apartado 4.1.2), se ha realizado un análisis de amenazas siguiendo el modelo STRIDE, que clasifica los riesgos en seis

categorías: suplantación, manipulación, repudio, divulgación de información, denegación de servicio y elevación de privilegios. El objetivo no es declarar el sistema invulnerable, sino demostrar que las principales amenazas se han considerado de forma sistemática y que cada una cuenta con una contramedida o con un riesgo residual reconocido. La Tabla 21 sintetiza el resultado.

Tabla 21. Análisis de amenazas según el modelo STRIDE y contramedidas implementadas.

Categoría	Amenaza en el sistema	Contramedida implementada
Spoofing (suplantación)	Hacerse pasar por otro usuario o rol.	Autenticación JWT firmada; contraseñas con bcrypt; login por email o DNI/NIE verificado.
Tampering (manipulación)	Alterar inventarios, donaciones o el registro de operaciones.	Validación Zod de toda entrada; registro encadenado (hash-chain) que detecta alteraciones retroactivas.
Repudiation (repudio)	Negar haber realizado una entrega o un movimiento.	AuditLog con actor, entidad y marca temporal; cadena de eventos verificable por terceros y sellado TSA opcional.
Information Disclosure	Exposición de datos personales (email, ubicación, DNI).	Minimización; la ubicación no se persiste por defecto; los payloads públicos excluyen identificadores.
Denial of Service	Saturación de la API o de la cola de eventos.	Rate limiting por IP con almacén en Redis; cabeceras de seguridad; reintentos con backoff y jitter en el cliente.
Elevation of Privilege	Acceso a operaciones de un rol superior.	RBAC con requireRole() en cada endpoint; doble protección por carga de rutas según rol; salvaguarda del coordinador.

Fuente: *Elaboración propia.*

El análisis confirma que las categorías más críticas para el dominio —la manipulación del registro logístico y el repudio de las entregas— quedan cubiertas por la combinación de validación, auditoría y registro encadenado verificable, una de las aportaciones diferenciales del trabajo. Las amenazas de denegación de servicio se mitigan a nivel de MVP, pero su tratamiento robusto frente a un ataque a gran escala se reconoce como riesgo residual y se proyecta como trabajo futuro.

4.1.3.8. Resultados y discusión

La ejecución del plan descrito permite afirmar que el sistema cumple los requisitos funcionales identificados y se comporta de forma robusta en los escenarios evaluados. Las suites automatizadas superan todos los casos, los seis escenarios representativos se completan con éxito, la evaluación heurística arroja un resultado satisfactorio, el análisis de amenazas no revela carencias sin mitigación o sin reconocer, y la prueba piloto confirma una usabilidad «buena» (SUS = 76,6) con una tasa de completitud del 89,3 %. El banco de pruebas de enrutamiento aporta evidencia cuantitativa de que la estrategia local-first es la adecuada para el dominio.

Resulta especialmente relevante poner los resultados en relación con los principios de diseño formulados en el capítulo 2. La resiliencia ante la red se constata en el escenario E-02 y en las

veintitrés operaciones offline de la prueba piloto, sincronizadas sin pérdidas ni duplicados. La integración del crowdsourcing con la logística física se materializa en el encadenamiento entre el reporte de incidencias, el enrutamiento que las evita y la entrega trazable mediante QR. La transparencia y el control quedan respaldados por el registro verificable y por su endpoint público de verificación. Y la coordinación flexible entre roles se valida en los escenarios E-01, E-05 y E-06 y en la matriz de permisos. La evaluación, por tanto, no solo comprueba requisitos aislados, sino que confirma que los principios rectores del trabajo se traducen en comportamiento observable del sistema.

Conviene también situar estos resultados frente a las soluciones analizadas en el estado del arte. Mientras que las plataformas de mapeo colaborativo se detienen en la visualización de la información y las herramientas de comunicación resiliente resuelven únicamente la conectividad, la propuesta evaluada cubre de extremo a extremo el ciclo logístico —de la necesidad a la entrega verificada— manteniendo la operación sin conexión. Esta combinación es, precisamente, el hueco que la comparativa de la Tabla 2 identificaba como no cubierto por las herramientas existentes, lo que refuerza la aportación diferencial del trabajo.

Conviene, no obstante, interpretar estos resultados con honestidad y delimitar su alcance. La evaluación se ha realizado en un entorno simulado y controlado, no en una catástrofe real, y la prueba piloto, aunque informativa, se basa en una muestra reducida de ocho participantes; sus resultados son indicativos y no estadísticamente concluyentes. Las limitaciones detectadas —la resolución de conflictos «último en escribir gana», la cadena de eventos serializada, la ausencia de validación en una emergencia real y la falta de integración con sistemas oficiales — se recogen como líneas de trabajo futuro en el capítulo 5. El valor del trabajo no reside en presentarse como una solución cerrada, sino en demostrar de forma verificable la viabilidad técnica del enfoque.

5. Conclusiones y trabajo futuro

5.1. Conclusiones

Este trabajo ha abordado el problema de la gestión logística en situaciones de emergencia, caracterizado por la pérdida de conectividad, la fragmentación de la información y la dificultad de coordinar a actores heterogéneos. Conforme exige el carácter de un Trabajo de Fin de Estudios, las conclusiones se redactan en función de los objetivos planteados en el capítulo 3, cuyo grado de consecución se sintetiza además en el Anexo H y se respalda en la matriz de trazabilidad (Tabla 6).

Respecto al **objetivo general** —diseñar y desarrollar una plataforma Offline-First que mejorase la coordinación, la eficiencia y la trazabilidad de la ayuda humanitaria—, se concluye que se ha alcanzado: la plataforma Catástrofe-Logística articula a ciudadanía, voluntariado, puestos de emergencia y coordinación en torno a un mapa operativo común y mantiene sus funciones esenciales en condiciones de conectividad degradada, como confirman los escenarios E-01 a E-06 y la prueba piloto.

En cuanto a los **objetivos específicos**, el análisis de las limitaciones de los sistemas actuales y de los principios de diseño (OE-1) se materializó en el capítulo 2 y en la comparativa de la Tabla 2, que evidencia el hueco de mercado que cubre la propuesta. El modelado de los cuatro actores con permisos diferenciados (OE-2) se concretó en el apartado de actores (4.1.1) y en la matriz de permisos del Anexo B. El reporte colaborativo de incidencias (OE-3), el inventario dual por puesto (OE-4) y la trazabilidad de donaciones mediante QR de un solo uso (OE-5) se implementaron y verificaron, según documentan los apartados de incidencias y de inventario (4.1.2) y los escenarios E-01 y E-03.

El enrutamiento consciente de las vías cortadas (OE-6) no solo se implementó, sino que se evaluó cuantitativamente con un banco de pruebas reproducible (apartado 4.1.3), que confirma la superioridad de la estrategia local-first. La arquitectura Offline-First con sincronización fiable (OE-7) constituye la contribución técnica central y se validó con el escenario E-02. El control de acceso por roles (OE-8), el registro verificable encadenado (OE-9) y la comunicación en tiempo real (OE-10) se implementaron y verificaron, como recogen los apartados de seguridad, registro encadenado y API (4.1.2). Finalmente, la evaluación del

sistema (OE-11) se abordó de forma multifacética en la evaluación (apartado 4.1.3), con la única salvedad —reconocida con honestidad— de que la validación se realizó en un entorno simulado y no en una emergencia real, lo que se reserva como primera prioridad del trabajo futuro.

Desde el punto de vista técnico, el trabajo ha permitido aplicar y profundizar en conocimientos adquiridos en el Máster —desarrollo de aplicaciones web resilientes, sincronización de datos, seguridad y privacidad por diseño, ingeniería de requisitos y verificabilidad criptográfica—. En conjunto, demuestra la viabilidad de utilizar tecnologías web modernas para construir sistemas de emergencia accesibles, abiertos y robustos que complementen —sin sustituir— la labor de los organismos oficiales.

5.2. Lecciones aprendidas

Monolito modular frente a microservicios

La elección de un monolito modular fue acertada para un equipo de dos personas. Los microservicios habrían añadido una complejidad operativa —despliegue, observabilidad, comunicación entre servicios— desproporcionada para el alcance del proyecto, sin aportar beneficios reales a esta escala. La modularidad interna proporcionó la separación de responsabilidades necesaria sin ese coste.

Complejidad del enfoque Offline-First

La implementación offline-first resultó considerablemente más compleja de lo estimado. Los principales escollos fueron la resolución de identificadores locales frente a los definitivos, la idempotencia de las operaciones reproducidas y la coherencia de la caché de lectura. La lección es clara: el offline no es una capa que se añade al final, sino una propiedad que condiciona el diseño desde el principio.

Valor de las pruebas extremo a extremo

La inversión en suites de Playwright desde fases tempranas amortizó su coste al detectar regresiones en los flujos principales antes de cada integración, especialmente en los puntos de unión entre frontend y backend.

Coordinación en un equipo de dos personas

La separación de responsabilidades —frontend y backend— facilitó el avance en paralelo, pero exigió una comunicación frecuente y disciplinada en los puntos de integración. Acordar pronto los contratos de la API y mantenerlos estables fue determinante para reducir la fricción.

5.3. Trabajo futuro

El sistema cubre las funcionalidades principales planteadas, pero abre varias líneas de evolución, ordenadas por prioridad:

Validación en escenario real

La validación definitiva requiere el uso del sistema en un ejercicio de protección civil o en una emergencia real, con un número de participantes suficiente para obtener conclusiones estadísticamente significativas sobre su impacto en la coordinación y la eficiencia. Es la primera prioridad, pues cierra la única salvedad de los objetivos.

Mejoras en sincronización y resiliencia

La resolución de conflictos actual —«último en escribir gana» para inventario y append-only para incidencias— es suficiente para el MVP, pero escenarios de mayor concurrencia se beneficiarían de los tipos de datos replicados libres de conflicto (CRDT), que permitirían la convergencia automática de réplicas. Asimismo, la partición de la cadena de eventos eliminaría el cuello de botella de su serialización global.

Integración con sistemas de emergencias oficiales

Una línea prioritaria es la integración con sistemas institucionales —la plataforma IFRC GO, los servicios 112 autonómicos y las redes de protección civil—, que permitiría contrastar la información ciudadana con las fuentes oficiales y coordinar la respuesta espontánea con la institucional.

Inteligencia artificial y observabilidad

La incorporación de modelos de optimización y de aprendizaje automático para la asignación dinámica de recursos y el cálculo de rutas a escala constituye una evolución natural del

subsistema logístico. En el plano operativo, reforzar la observabilidad (trazas, métricas y alertas) y las pruebas de carga convertiría la escalabilidad, hoy formulada como propiedad de diseño, en una capacidad medida y garantizada.

6. Bibliografía

Las referencias siguen el estilo APA (7.^a edición) y consolidan en una única lista todas las fuentes citadas a lo largo de la memoria. Todas las entradas aparecen referenciadas en el texto.

- Bangor, A., Kortum, P., & Miller, J. (2008). An empirical evaluation of the System Usability Scale. *International Journal of Human-Computer Interaction*, 24(6), 574–594.
- Biørn-Hansen, A., Majchrzak, T. A., & Grønli, T. M. (2017). Progressive Web Apps: the possible web-native unifier for mobile development. En *Proceedings of the 13th International Conference on Web Information Systems and Technologies (WEBIST)* (pp. 344–351). SciTePress.
- Brooke, J. (1996). SUS: A “quick and dirty” usability scale. En P. W. Jordan, B. Thomas, B. A. Weerdmeester & I. L. McClelland (Eds.), *Usability Evaluation in Industry* (pp. 189–194). Taylor & Francis.
- Careem, M., De Silva, C., De Silva, R., Raschid, L., & Weerawarana, S. (2006). Sahana: Overview of a Disaster Management System. *International Conference on Information and Automation (ICIA)*.
- Comfort, L. K. (2007). Crisis Management in Hindsight: Cognition, Communication, Coordination, and Control. *Public Administration Review*, 67(s1), 189–197.
- De Vicente Mohino, J., Bermejo Higuera, J., Bermejo Higuera, J. R., & Sicilia Montalvo, J. A. (2019). The Application of a New Secure Software Development Life Cycle (S-SDLC) with Agile Methodologies. *Electronics*, 8(11), 1218.
- Goodchild, M. F., & Glennon, J. A. (2010). Crowdsourcing geographic information for disaster response: a research frontier. *International Journal of Digital Earth*, 3(3), 231–241.
- Imran, M., Castillo, C., Diaz, F., & Vieweg, S. (2015). Processing social media messages in mass emergency: a survey. *ACM Computing Surveys*, 47(4), 1–38.
- International Federation of Red Cross and Red Crescent Societies (IFRC). (2023). IFRC GO platform. <https://go.ifrc.org/>
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- Kovács, G., & Spens, K. M. (2007). Humanitarian logistics in disaster relief operations. *International Journal of Physical Distribution & Logistics Management*, 37(2), 99–114.
- Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales. *Boletín Oficial del Estado*, núm. 294.
- Meier, P. (2015). *Digital Humanitarians: How Big Data Is Changing the Face of Humanitarian Response*. CRC Press.
- Nielsen, J. (1994). Heuristic evaluation. En J. Nielsen & R. L. Mack (Eds.), *Usability Inspection Methods* (pp. 25–62). John Wiley & Sons.
- Nielsen, J., & Molich, R. (1990). Heuristic evaluation of user interfaces. En *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '90)* (pp. 249–256). ACM.

- Okolloh, O. (2009). Ushahidi, or “testimony”: Web 2.0 tools for crowdsourcing crisis information. *Participatory Learning and Action*, 59(1), 65–70.
- Osmani, A. (2018). The Cost of JavaScript in 2018. Medium. <https://medium.com/@addyosmani/the-cost-of-javascript-in-2018-7d8950fbb5d4>
- Palen, L., & Anderson, K. M. (2016). Crisis informatics—New data for extraordinary times. *Science*, 353(6296), 224–225.
- Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016, relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales (RGPD). Diario Oficial de la Unión Europea, L 119.
- Reuter, C., & Kaufhold, M. A. (2018). Fifteen years of social media in emergencies: a retrospective review and future directions. *Journal of Contingencies and Crisis Management*, 26(1), 41–57.
- Russell, A. (2015). Progressive Web Apps: Escaping Tabs Without Losing Our Soul. Infrequently Noted. <https://infrequently.org/2015/06/progressive-apps-escaping-tabs-without-losing-our-soul/>
- Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). Conflict-Free Replicated Data Types. *Symposium on Self-Stabilizing Systems (SSS)*, 386–400.
- Sicilia, J. A., Quemada, C., Royo, B., & Escuín, D. (2016). An optimization algorithm for solving the rich vehicle routing problem based on variable neighborhood search and tabu search metaheuristics. *Journal of Computational and Applied Mathematics*, 291, 468–477.
- Tan, M. L., Prasanna, R., Stock, K., Hudson-Doyle, E., Leonard, G., & Johnston, D. (2017). Mobile applications in crisis informatics literature: a systematic review. *International Journal of Disaster Risk Reduction*, 24, 297–311.
- Van Wassenhove, L. N. (2006). Humanitarian aid logistics: supply chain management in high gear. *Journal of the Operational Research Society*, 57(5), 475–489.

Anexo A. Artículo científico

Este anexo recoge el artículo científico que sintetiza el trabajo en formato de comunicación académica, conforme a los requisitos de la titulación.

Catástrofe-Logística: una plataforma Offline-First para la coordinación ciudadana en emergencias

Francisco Capote García y Álvaro Martín Muñoz

Universidad Internacional de La Rioja, Logroño (España), 2026

Resumen

La gestión logística en situaciones de catástrofe se ve limitada por la falta de información en tiempo real, la dependencia de infraestructuras de comunicación vulnerables y la dificultad de coordinar a múltiples actores. Este trabajo presenta Catástrofe-Logística, una aplicación web progresiva (PWA) con arquitectura Offline-First que integra reporte colaborativo de incidencias, gestión de inventario por puesto, trazabilidad de donaciones mediante códigos QR, enrutamiento consciente de vías cortadas y un registro verificable encadenado. La evaluación combina pruebas automatizadas, evaluación heurística, un banco de pruebas de enrutamiento y una prueba piloto simulada (SUS = 76,6), y demuestra la viabilidad técnica del enfoque para sostener la operativa básica de una emergencia en condiciones de conectividad degradada.

Términos clave— logística humanitaria, Offline-First, Progressive Web App, crowdsourcing, trazabilidad, sincronización de datos.

I. Introducción

Las emergencias de gran escala provocan una degradación rápida de las infraestructuras de comunicación y una explosión de necesidades de información y de recursos. Las herramientas dependientes de conectividad permanente fallan justo cuando más se necesitan, y la ayuda espontánea de la ciudadanía, pese a su valor, carece a menudo de un canal estructurado. Este artículo propone una plataforma que asume la intermitencia de la red como estado normal de operación y que integra la información colaborativa con la gestión física de recursos.

II. Estado del arte

Plataformas como Sahana Eden [1] y Ushahidi [2] han evidenciado el potencial del crowdsourcing en escenarios humanitarios, pero no abordan la trazabilidad física de las entregas ni la coordinación logística completa. La literatura de logística humanitaria [3], [4] subraya el papel crítico de la visibilidad y la coordinación. En el plano técnico, las Progressive Web Apps [5] y las técnicas de replicación de datos [6] habilitan aplicaciones resilientes capaces de operar sin conexión.

III. Objetivos y metodología

El objetivo general es diseñar y desarrollar una plataforma Offline-First que mejore la coordinación, la eficiencia y la trazabilidad de la ayuda en emergencias. El desarrollo siguió un modelo iterativo e incremental, con control de versiones distribuido, pruebas automatizadas y un entorno reproducible con Docker Compose, en un equipo de dos personas con responsabilidades separadas entre frontend y backend.

IV. Contribución

La plataforma se compone de un frontend en React, TypeScript y Vite, empaquetado como PWA con Service Workers (Workbox) y persistencia local en IndexedDB (Dexie), y un backend monolítico modular en Node.js con Fastify, PostgreSQL y Prisma. Sus aportaciones diferenciales son una cola de sincronización offline con reintentos e idempotencia, un enrutamiento local-first sobre grafo OSM con penalización blanda de vías cortadas, y un registro verificable encadenado (hash-chain) de las operaciones críticas, todo ello articulado en torno a cuatro roles y un mapa operativo común.

La arquitectura Offline-First merece una mención específica por ser la contribución central. El cliente persiste en IndexedDB tanto los datos de lectura (puestos, incidencias y teselas cartográficas) como una cola de operaciones de escritura. Al recuperar la conectividad, esta cola se reproduce contra el servidor en orden, con reintentos exponenciales y claves de idempotencia que evitan duplicados, y con una resolución de identificadores locales que preserva la integridad de las operaciones encadenadas. Esta combinación garantiza que ninguna acción del usuario se pierda durante los cortes de red característicos de una emergencia.

V. Resultados

La aplicación ejecuta los flujos principales de los cuatro roles y mantiene la funcionalidad esencial sin conexión. Las suites automatizadas (Vitest, Playwright) superan todos los casos; la auditoría Lighthouse confirma el comportamiento como PWA instalable; el banco de pruebas de enrutamiento muestra que la estrategia local-first mantiene el 100 % de éxito frente al descenso de la heurística online al acumularse cortes; y la prueba piloto simulada con ocho participantes arroja una SUS media de 76,6 y una completitud del 89,3 %. Un banco de pruebas reproducible del subsistema de enrutamiento complementa estos resultados: muestra que el cálculo sobre el grafo local mantiene una tasa de éxito del 100 % a medida que se acumulan vías cortadas, mientras que la heurística basada en un servicio remoto degrada su éxito y multiplica su coste de red, lo que respalda cuantitativamente la decisión de adoptar una estrategia local-first.

VI. Discusión

Los resultados sugieren que un enfoque Offline-First combinado con difusión en tiempo real basada en sockets puede proporcionar una base resiliente para la coordinación de emergencias. Las limitaciones —resolución de conflictos simple, cadena serializada, ausencia de validación en emergencia real e integración oficial— delimitan el alcance del MVP y orientan el trabajo futuro.

VII. Conclusiones

El trabajo demuestra la viabilidad de construir, con tecnologías web modernas, una plataforma de gestión logística para emergencias que opera con conectividad degradada, integra a la ciudadanía y aporta trazabilidad verificable, complementando la labor de los organismos oficiales.

Referencias

- [1] Careem, M. et al. (2006). Sahana: Overview of a Disaster Management System. ICIA.
- [2] Okolloh, O. (2009). Ushahidi, or “testimony”: Web 2.0 tools for crowdsourcing crisis information. PLA, 59(1).
- [3] Comfort, L. K. (2007). Crisis Management in Hindsight. Public Administration Review, 67(s1).
- [4] Van Wassenhove, L. N. (2006). Humanitarian aid logistics. JORS, 57(5).
- [5] Russell, A. (2015). Progressive Web Apps. Infrequently Noted.
- [6] Shapiro, M. et al. (2011). Conflict-Free Replicated Data Types. SSS.

Anexo B. Especificación operativa por rol y matriz de permisos

Este anexo traduce la implementación en una especificación de uso operativa por rol y formaliza, en una única matriz, las capacidades de cada perfil. Sirve como referencia rápida para la operación y la defensa del trabajo, sin reproducir la descripción funcional ya desarrollada en el capítulo 4.

B.1. Capacidades por rol

El ciudadano consulta puestos cercanos y el estado del inventario público, busca productos mediante utilidades de ranking y distancia, calcula rutas que evitan incidencias activas, reporta incidencias geolocalizadas con comentarios de estado y utiliza el modo offline para conservar sus operaciones hasta la reconexión. El voluntario consulta las necesidades reales del inventario NECESARIO de los puestos activos, crea donaciones con la cantidad limitada por lo pendiente, gestiona los estados de la donación, genera el código de entrega y solicita participación en puestos e incidencias. El puesto de emergencia gestiona el alta y la aprobación, el inventario disponible y necesario, el historial de movimientos, la confirmación de entregas por QR de consumo único y las solicitudes de participación. El coordinador aprueba o rechaza altas de puestos, gestiona usuarios y roles, consulta el detalle de puestos e incidencias, gestiona participaciones y consulta el registro de auditoría y la cadena pública de eventos.

B.2. Matriz de permisos

La Tabla 22 sintetiza, por funcionalidad y rol, las capacidades efectivas del sistema, coherentes con las guardas `requireRole()` del backend y con la carga de rutas por rol en el cliente.

Tabla 22. Matriz de permisos por funcionalidad y rol, coherente con el control de acceso implementado.

Funcionalidad	Ciudadano	Voluntario	Puesto	Coordinador
Consultar mapa y puestos	Sí	Sí	Sí	Sí
Reportar incidencia	Sí	Sí	Sí si autenticado	Sí
Comentar/cambiar estado de incidencia	Sí autenticado	Sí autenticado	Sí autenticado	Sí y edición avanzada
Crear donación	No	Sí	No	No
Confirmar entrega QR	No	Según flujo de entrega	Sí en puesto receptor	Supervisión
Gestionar inventario propio	No	No	Sí	No salvo coordinación indirecta
Solicitar alta de puesto	Sí autenticado	Sí autenticado	Sí autenticado	Puede crear/gestionar
Aprobar puesto	No	No	No	Sí

Funcionalidad	Ciudadano	Voluntario	Puesto	Coordinador
Gestionar usuarios	No	No	No	Sí
Verificar cadena pública	Sí	Sí	Sí	Sí

Fuente: Elaboración propia.

B.3. Flujos operativos críticos

Los flujos críticos del sistema pueden describirse como cadenas de eventos de negocio. La Tabla 23 resume cada flujo, sus pasos principales y el riesgo que controla, lo que facilita la comprensión integrada del sistema.

Tabla 23. Flujos operativos críticos de la plataforma y riesgo que controla cada uno.

Flujo	Pasos principales	Riesgo controlado
Alta de usuario	Registro -> bcrypt -> roles CIUDADANO/VOLUNTARIO -> recoveryCode -> tokens	Identidad inicial y recuperación sin SMTP obligatorio.
Alta de puesto	Solicitud -> revisión coordinador -> aprobación/rechazo -> publicación operativa	Evita puestos falsos o no verificados.
Donación	Necesidad -> compromiso voluntario -> estado -> código entrega -> confirmación QR	Controla sobreabastecimiento y prueba entrega física.
Incidencia	Reporte -> realtime -> routing evita zona -> comentario/coord -> estado TRANSITABLE	Mejora mapa operativo y rutas.
Offline	Operación local -> syncQueue -> idempotency-key -> flush -> idMapping	Evita pérdida y duplicación de operaciones.
Auditoría pública	Evento crítico -> PendingChainEvent -> ChainEvent -> verify	Detecta alteración retroactiva.

Fuente: Elaboración propia.

Anexo C. Guía de validación, despliegue y defensa

Este anexo convierte el proyecto en un entregable defendible, recogiendo la lista de comprobación de entrega, los riesgos residuales con su mitigación futura y un guion de demostración recomendado.

C.1. Lista de comprobación de entrega

La Tabla 24 reúne las comprobaciones por área que conviene verificar antes de la entrega y la defensa, junto con el estado esperado en cada una. Su objetivo es garantizar la coherencia entre la memoria y el código y la delimitación honesta del alcance.

Tabla 24. Lista de comprobación de entrega y coherencia entre memoria y código.

Área	Comprobación	Estado esperado
Documento	Título, resumen, abstract, objetivos y conclusiones coherentes.	Sin contradicciones con el código.
Alcance	Diferenciar implementado, diseño y trabajo futuro.	No se promete Catastrofe persistida, fotos, CRDT ni integración oficial.
Código	Repositorio con frontend, backend, Prisma, Docker, Render y tests.	Estructura navegable y explicable.
Seguridad	JWT_SECRET robusto, bcrypt, refresh hash y RBAC.	Medidas descritas y limitaciones reconocidas.
Offline	Cola IndexedDB, API cache, Service Worker e idempotencia.	Sincronización explicada de extremo a extremo.
Routing	Benchmark incluido con resultados y figuras.	Aporte técnico diferencial defendible.
Trazabilidad	AuditLog y ChainEvent documentados.	Transparencia sin llamarlo blockchain.
Evaluación	Pruebas por feature y escenarios simulados.	No se afirma validación en catástrofe real.
Despliegue	Docker Compose y render.yaml explicados.	Demo reproducible y límites de producción claros.

Fuente: *Elaboración propia.*

C.2. Riesgos residuales y mitigaciones futuras

Incluso tras el desarrollo, conviene mantener una visión crítica. La Tabla 25 recoge los riesgos residuales del MVP, su impacto y la línea de mitigación futura, en coherencia con el trabajo futuro del capítulo 5.

Tabla 25. Riesgos residuales del MVP, impacto y líneas de mitigación futura.

Riesgo residual	Impacto	Mitigación futura
Código de recuperación reutilizable o sin caducidad estricta	Acceso no autorizado si se filtra el código.	Caducidad, rotación automática tras uso y segundo factor opcional.
Conflictos offline complejos	Divergencia de inventario o duplicidad de operaciones.	Políticas por entidad, resolución asistida y CRDTs para campos adecuados.
Dependencia de datos OSM precargados	Routing local no disponible en zonas no preparadas.	Pipeline de extractos por zona y descarga previa desde coordinador.
Sin integración oficial con 112/Protección Civil	Menor utilidad institucional.	APIs de interoperabilidad y acuerdos con organismos.
Ausencia de pruebas de carga masiva	Riesgo de saturación en emergencia real.	k6/JMeter, métricas, escalado horizontal y Redis adapter para Socket.IO.
Observabilidad limitada	Dificultad para diagnosticar incidentes.	Logs estructurados, tracing, alertas y dashboards.
RGPD incompleto en flujos de derechos	Riesgo legal.	Panel de derechos, retención configurable y anonimización granular.

Fuente: Elaboración propia.

C.3. Guion de demostración recomendado

Para la defensa se recomienda un flujo de demostración corto y robusto que conecte los subsistemas entre sí en lugar de mostrarlos de forma aislada. La Tabla 26 propone una secuencia de cinco pasos que recorre login y roles, gestión de puesto, incidencia y ruta, donación y QR, y verificación de la cadena pública.

Tabla 26. Guion de demostración recomendado para la defensa del trabajo.

Paso	Demostración
1	Login y roles protegidos.
2	Puesto con inventario necesario/disponible.
3	Incidencia y ruta evitando corte.
4	Donación y QR de entrega.
5	/verificar y registro encadenado.

Fuente: Elaboración propia.

En síntesis, Catástrofe-Logística se defiende no como una solución cerrada para cualquier catástrofe, sino como un prototipo profesional, técnicamente coherente y verificable, que demuestra la viabilidad de una coordinación de emergencias resiliente y abierta, y que establece una base sólida sobre la que construir las líneas de evolución identificadas.

Anexo D. Referencia de la API REST

Este anexo documenta la superficie real de la API, expuesta bajo el prefijo /api y organizada en ocho routers por dominio. Todas las peticiones se validan con esquemas Zod y, cuando procede, se protegen con las guardas requireAuth y requireRole. La Tabla 27 resume los endpoints representativos de cada router y su control de acceso.

Tabla 27. Superficie real de la API REST del backend, por router.

Router	Endpoints representativos reales	Control de acceso
/api/auth	POST /register, /login, /refresh, /logout, /password-reset/request, /recovery-code; GET /me	Registro/login públicos; recovery-code y me autenticados.
/api/users	GET /me, PATCH /me; GET/PATCH/DELETE /coordinador/:id	Perfil autenticado; gestión de usuarios solo COORDINADOR.
/api/voluntarios	GET /me, PATCH /me	Usuario autenticado; perfil de voluntario asociado al usuario.
/api/puestos	GET /, /mio, /mis-asignaciones, /mis-solicitudes-participacion, /:id; POST /solicitudes, /:id/participaciones; endpoints /coordinador/*	Lecturas públicas o autenticadas según ruta; gestión global y aprobación solo coordinador; acciones de puesto según responsable/rol.
/api/inventario	GET /puesto/:puestold, /puesto/:puestold/historial; POST /puesto/:puestold/items, /confirmar-qr; PATCH /items/:itemid/cantidad; DELETE /items/:itemid	Lecturas de inventario; modificaciones protegidas por autenticación y validaciones de puesto.
/api/incidencias	GET /, GET /mis-asignaciones, POST /, POST /:id/comentarios, POST /:id/asignaciones, PATCH /:id/estado, PATCH /coordinador/:id, DELETE /:id	Reporte con auth opcional; comentarios y asignaciones autenticados; edición/cierre coordinador.
/api/donaciones	GET /necesidades, /mis-donaciones; POST /; PATCH /:id/estado, /:id/cantidad; POST /:id/codigo-entrega	Rol VOLUNTARIO para operaciones de donación.
/api/public/audit	GET /stats, /verify, /recent, /donacion/:id, /inventario/:puestold, /event/:id	Público de solo lectura para transparencia y verificación.

Fuente: Elaboración propia a partir del código.

A continuación se describe brevemente la responsabilidad de cada router. El router /api/auth concentra el ciclo de identidad: registro, inicio y cierre de sesión, renovación de tokens, solicitud de restablecimiento y regeneración del código de recuperación. El router /api/users gestiona el perfil del usuario autenticado y, de forma restringida al coordinador, la administración de cuentas. El router /api/voluntarios mantiene el perfil operativo del voluntario y su modalidad. El router /api/puestos abarca el alta, la consulta y la gestión de puestos, las solicitudes de creación y las participaciones, con un subconjunto de operaciones reservadas a coordinación.

El router /api/inventario expone la consulta del inventario dual y su historial, así como las operaciones de modificación de stock y la confirmación de entregas mediante QR. El router /api/incidencias cubre el reporte (con autenticación opcional), los comentarios de validación colaborativa, las asignaciones y el cambio de estado, con operaciones de edición y cierre reservadas al coordinador. El router /api/donaciones gestiona el ciclo de vida de las donaciones y la emisión del código de entrega, restringido al rol voluntario. Por último, el

router /api/public/audit ofrece, en modo solo lectura y sin exponer datos personales, las estadísticas, la verificación de la integridad de la cadena y la consulta de eventos, garantizando la transparencia del registro verificable.

Anexo E. Diccionario de datos

Este anexo detalla el modelo físico de datos implementado con Prisma sobre PostgreSQL, complementando la visión de conjunto del apartado de modelo de datos (4.1.2). Para cada entidad principal se describen sus campos, su tipo y sus restricciones, fielmente al esquema `schema.prisma` del repositorio. Los identificadores se generan con `cuid` salvo que se indique lo contrario, y los campos `createdAt` y `updatedAt` registran la marca temporal de creación y última modificación.

E.1. Enumerados

La Tabla 28 recoge los tipos enumerados del modelo, que acotan los valores admisibles de los campos de estado y clasificación.

Tabla 28. Tipos enumerados del modelo de datos.

Enumerado	Valores
<code>RolUsuario</code>	CIUDADANO, VOLUNTARIO, PUESTO_EMERGENCIA, COORDINADOR
<code>TipoInventario</code>	DISPONIBLE, NECESARIO
<code>EstadoVia</code>	CORTADA, TRANSITABLE
<code>EstadoDonacion</code>	PENDIENTE, EN_CAMINO, ENTREGADA, CANCELADA
<code>EstadoPuesto</code>	PENDIENTE, APROBADO, RECHAZADO
<code>EstadoAsignacionPuesto</code>	ACTIVA, FINALIZADA, CANCELADA
<code>EstadoAsignacionIncidencia</code>	ACTIVA, FINALIZADA, CANCELADA
<code>EstadoSolicitudPuesto</code>	PENDIENTE, ACEPTADA, RECHAZADA
<code>EstadoSolicitudParticipacionPuesto</code>	PENDIENTE, ACEPTADA, RECHAZADA

Fuente: *Elaboración propia.*

E.2. Entidad Usuario

Tabla 29. Diccionario de datos de la entidad Usuario (tabla usuarios).

Campo	Tipo	Restricciones y descripción
<code>id</code>	String	PK, cuid.
<code>email</code>	String	Único; identificador de inicio de sesión.
<code>password</code>	String	Hash bcrypt; nunca se expone.
<code>nombre / apellidos</code>	String	Datos personales mínimos.
<code>telefono</code>	String?	Opcional.
<code>dni</code>	String?	Único; usado en la recuperación de cuenta.
<code>roles</code>	<code>RolUsuario[]</code>	Array de roles; permite perfiles múltiples.
<code>activo</code>	Boolean	Por defecto true; el borrado es lógico.
<code>emailVerified / emailVerifiedAt</code>	Boolean / DateTime?	Estado y fecha de verificación de email.
<code>recoveryCodeHash</code>	String?	Hash del código de recuperación; nunca se expone.
<code>createdAt / updatedAt</code>	DateTime	Marcas temporales.

Fuente: *Elaboración propia.*

E.3. Entidad PuestoEmergencia

Tabla 30. Diccionario de datos de la entidad PuestoEmergencia (tabla puestos_emergencia).

Campo	Tipo	Restricciones y descripción
id	String	PK, cuid.
nombre / descripcion	String / String?	Identificación del puesto.
direccion	String	Dirección textual.
latitud / longitud	Float	Coordenadas geográficas.
tipo	String	Tipo de espacio o puesto.
activo	Boolean	Por defecto true.
estadoSolicitud	EstadoPuesto	PENDIENTE, APROBADO o RECHAZADO.
motivoRechazo	String?	Motivo en caso de rechazo.
capacidadTrabajo	Int	Plazas de cuadrilla; por defecto 6.
adminId	String	FK al Usuario responsable.
createdAt / updatedAt	DateTime	Marcas temporales.

Fuente: Elaboración propia.

E.4. Entidades de logística (Producto, Inventario, Donacion)

Tabla 31. Diccionario de datos de la entidad Producto (tabla productos).

Campo	Tipo	Restricciones y descripción
id	String	PK, cuid.
nombre	String	Nombre del producto.
categoria	String	Categoría (agua, alimentos, etc.).
unidad	String	Unidad de medida.
createdAt	DateTime	Marca temporal de creación.

Fuente: Elaboración propia.

Tabla 32. Diccionario de datos de la entidad Inventario (tabla inventario).

Campo	Tipo	Restricciones y descripción
id	String	PK, cuid.
puetold	String	FK a PuestoEmergencia.
productold	String	FK a Producto.
cantidad	Float	Cantidad disponible o necesaria.
tipo	TipoInventario	DISPONIBLE o NECESARIO.
—	Restricción	Único por (puetold, productold, tipo).
updatedAt	DateTime	Marca de última modificación.

Fuente: Elaboración propia.

Tabla 33. Diccionario de datos de la entidad Donacion (tabla donaciones).

Campo	Tipo	Restricciones y descripción
id	String	PK, cuid.
voluntariold / puetold / productold	String	FK a Voluntario, PuestoEmergencia y Producto.
cantidad / unidad	Float / String	Cantidad comprometida y su unidad.
estado	EstadoDonacion	PENDIENTE, EN_CAMINO, ENTREGADA o CANCELADA.
comentario	String?	Comentario opcional.
eta	DateTime?	Tiempo estimado de llegada.
entregaCodigo	String?	Único; código QR de entrega de un solo uso.
entregaCodigoGeneradoAt	DateTime?	Momento de generación del código.
createdAt / updatedAt	DateTime	Marcas temporales.

Fuente: Elaboración propia.

E.5. Entidades del entorno (IncidenciaVia, Voluntario)

Tabla 34. Diccionario de datos de la entidad IncidenciaVia (tabla incidencias_via).

Campo	Tipo	Restricciones y descripción
id	String	PK, cuid.
reportanteld	String?	FK opcional al Usuario reportante.
título / categoria	String?	Título y categoría de la incidencia.
latitud / longitud	Float	Coordenadas de la incidencia.
estado	EstadoVia	CORTADA o TRANSITABLE.
descripcion	String?	Descripción opcional.
createdAt / updatedAt	DateTime	Marcas temporales; índices por estado y fecha.

Fuente: Elaboración propia.

Tabla 35. Diccionario de datos de la entidad Voluntario (tabla voluntarios).

Campo	Tipo	Restricciones y descripción
id	String	PK, cuid.
usuariold	String	Único; FK 1:1 con Usuario.
modalidad	String?	Transporte o mano de obra.
vehiculo	Json?	Datos del vehículo en formato JSON.
verificado	Boolean	Por defecto false.
createdAt / updatedAt	DateTime	Marcas temporales.

Fuente: Elaboración propia.

E.6. Entidades de trazabilidad y resiliencia

Tabla 36. Diccionario de datos de la entidad ChainEvent (tabla chain_events).

Campo	Tipo	Restricciones y descripción
id	String	PK, cuid.
sequence	Int	Único, autoincremental; posición en la cadena.
tipo	String	Tipo de evento crítico.
actorId / actorRol	String?	Autor y rol del actor del evento.
entidad / entidadId	String	Entidad afectada e identificador.
payload	Json	Contenido del evento.
hashPrevio	String	Hash del evento anterior (encadenamiento).
hashPropio	String	Único; hash del propio evento.
tsaToken / tsaTimestamp	String? / DateTime?	Sellado temporal best-effort (TSA).
createdAt	DateTime	Marca temporal de creación.

Fuente: Elaboración propia.

Tabla 37. Diccionario de datos de las entidades AuditLog, IdempotencyRecord y QrConsumption.

Entidad / Campo	Tipo	Restricciones y descripción
AuditLog.accion / entidad / entidadId	String	Operación, entidad y referencia auditadas.
AuditLog.datos / ip	Json? / String?	Diff de la operación e IP de origen.
IdempotencyRecord.key	String	PK; clave de idempotencia de la petición.
IdempotencyRecord.method / url / bodyHash	String	Método, URL y hash del cuerpo de la petición.
IdempotencyRecord.status / statusCode / response	String / Int? / Json?	Estado (PENDING/COMPLETED), código y respuesta almacenada.
IdempotencyRecord.expiresAt	DateTime	Caducidad (24 h); depurada por el worker de mantenimiento.
QrConsumption.key	String	PK; clave del código QR consumido.
QrConsumption.tipo / puestold / usuariold	String / String / String?	Garantiza el consumo único del código.

Fuente: Elaboración propia.

Anexo F. Glosario de términos y acrónimos

Este anexo reúne los términos técnicos y acrónimos utilizados a lo largo de la memoria, con una definición breve orientada a facilitar su lectura.

Tabla 38. Glosario de términos y acrónimos.

Término / Acrónimo	Definición
Offline-First	Enfoque de diseño en el que la aplicación funciona sin conexión por defecto y sincroniza los datos cuando la red está disponible.
PWA (Progressive Web App)	Aplicación web instalable que, mediante Service Workers, ofrece funcionamiento offline y una experiencia próxima a la nativa.
Service Worker	Script que se ejecuta en segundo plano en el navegador y permite el cacheo de recursos y la interceptación de peticiones de red.
IndexedDB	Base de datos del navegador para almacenamiento estructurado en el cliente; soporta la persistencia local y la cola offline.
Idempotencia	Propiedad por la que repetir una operación produce el mismo resultado que ejecutarla una sola vez; evita duplicados al reintentar.
CRDT	Conflict-Free Replicated Data Type: estructura que permite la convergencia automática de réplicas modificadas de forma concurrente.
RBAC	Role-Based Access Control: control de acceso basado en los roles asignados al usuario.
JWT	JSON Web Token: token firmado que transporta la identidad y los permisos del usuario sin estado en el servidor.
Hash-chain	Cadena de hashes en la que cada elemento incluye el hash del anterior, de modo que cualquier alteración es detectable.
TSA	Time-Stamping Authority: autoridad de sellado temporal que certifica la existencia de un dato en un instante dado.
OSM (OpenStreetMap)	Proyecto colaborativo de cartografía libre; fuente de las teselas y del grafo de calles usado en el enrutamiento.
OSRM	Open Source Routing Machine: motor de cálculo de rutas sobre datos OSM utilizado como estrategia heurística de referencia.
A* / Dijkstra	Algoritmos de búsqueda de caminos mínimos sobre grafos; A* emplea una heurística para acelerar la búsqueda.
SUS	System Usability Scale: cuestionario estandarizado de diez ítems para medir la usabilidad percibida.
RGPD / LOPDGDD	Reglamento General de Protección de Datos (UE) y su desarrollo en la legislación española.
MVP	Minimum Viable Product: producto mínimo viable que implementa las funcionalidades esenciales para validar una propuesta.
ORM	Object-Relational Mapping: capa que traduce entre objetos del lenguaje y tablas relacionales (Prisma, en este trabajo).
WebSocket / Socket.IO	Protocolo y biblioteca para la comunicación bidireccional en tiempo real entre cliente y servidor.

Fuente: *Elaboración propia.*

Anexo G. Especificación detallada de los casos de prueba

Este anexo desarrolla, en forma de especificación estructurada, los seis escenarios de prueba representativos resumidos en la sección 5.3. Para cada caso se documentan el objetivo, los requisitos verificados, las precondiciones, los pasos, el resultado esperado y el resultado obtenido. Esta especificación permite reproducir las pruebas de forma independiente y constituye una evidencia adicional de la trazabilidad entre requisitos y verificación.

G.1. Caso E-01: Reporte de incidencia y propagación en tiempo real

Tabla 39. Especificación del caso de prueba E-01.

Objetivo	Verificar que una incidencia reportada por un usuario se persiste y se difunde en tiempo real al resto de clientes conectados sin recarga.
Requisitos	RF-09, RF-22.
Precondiciones	Dos usuarios autenticados con sesión activa; al menos un escenario de emergencia activo; conectividad disponible.
Pasos	1) El usuario A selecciona una ubicación en el mapa. 2) Marca la categoría «cortada» y confirma. 3) El usuario B mantiene abierta la vista de mapa de la misma zona.
Resultado esperado	La incidencia se persiste en el backend, se emite por Socket.IO y aparece en el cliente de B sin recargar la página.
Resultado obtenido	PASS. Latencia observada < 3 s en LAN y < 5 s en 4G simulada; la incidencia aparece en el cliente de B sin recarga.

Fuente: Elaboración propia.

G.2. Caso E-02: Creación offline y sincronización posterior

Tabla 40. Especificación del caso de prueba E-02.

Objetivo	Verificar que una operación creada sin conexión se conserva y se sincroniza sin duplicados al recuperar la red.
Requisitos	RF-21, RNF-01, RNF-02.
Precondiciones	Usuario autenticado; modo avión o red desactivada en el dispositivo.
Pasos	1) Sin conexión, el usuario crea una solicitud de ayuda en un punto del mapa. 2) Cierra y reabre el navegador. 3) Restaura la conectividad.
Resultado esperado	La operación se conserva en IndexedDB pese al cierre del navegador y se sincroniza al reconectar, sin generar duplicados.
Resultado obtenido	PASS. La operación persiste en la cola; al reconectar se reproduce una sola vez (IdempotencyRecord activo) y queda registrada en el backend.

Fuente: Elaboración propia.

G.3. Caso E-03: Entrega de productos mediante QR

Tabla 41. Especificación del caso de prueba E-03.

Objetivo	Verificar que el escaneo de un código QR de entrega invalida el código, actualiza el inventario y registra el movimiento.
Requisitos	RF-15, RF-16, RF-25.
Precondiciones	Un puesto con una necesidad publicada; un voluntario con una donación en camino; el puesto ha generado un QR de entrega.
Pasos	1) El voluntario escanea el QR mostrado por el puesto. 2) Intenta escanear de nuevo el mismo QR.
Resultado esperado	El primer escaneo confirma la entrega, actualiza el inventario y audita el movimiento; el segundo escaneo es rechazado (un solo uso).
Resultado obtenido	PASS. El QR se invalida tras el primer uso; el inventario se actualiza y se registra el AuditLog con ambos actores.

Fuente: Elaboración propia.

G.4. Caso E-04: Cálculo de ruta evitando incidencias activas

Tabla 42. Especificación del caso de prueba E-04.

Objetivo	Verificar que la ruta calculada evita los tramos con incidencias activas de tipo cortada o peligrosa.
Requisitos	RF-18, RF-19, RF-20.
Precondiciones	Varias incidencias activas reportadas en la zona; un destino seleccionado.
Pasos	1) El usuario solicita ruta a un puesto situado tras varias vías cortadas. 2) Se reporta una nueva incidencia sobre la ruta calculada.
Resultado esperado	La ruta no atraviesa incidencias activas; al aparecer una nueva incidencia sobre la ruta, esta se recalcula.
Resultado obtenido	PASS. La ruta excluye los segmentos comprometidos mediante penalización blanda; el recálculo automático se dispara con la nueva incidencia.

Fuente: Elaboración propia.

G.5. Caso E-05: Aprobación de una solicitud de puesto

Tabla 43. Especificación del caso de prueba E-05.

Objetivo	Verificar el flujo de alta de puesto, desde la solicitud hasta su aprobación y publicación.
Requisitos	RF-08, RF-11.
Precondiciones	Un usuario autenticado con una solicitud de alta pendiente; un coordinador con sesión activa.
Pasos	1) El coordinador revisa la solicitud. 2) La aprueba. 3) Un ciudadano consulta el mapa.
Resultado esperado	La solicitud pasa a aceptada, el puesto se crea y se hace visible; el solicitante obtiene el rol de gestión del puesto.
Resultado obtenido	PASS. El estado cambia a aceptada; el puesto se difunde por Socket.IO en < 2 s y queda visible para la ciudadanía.

Fuente: Elaboración propia.

G.6. Caso E-06: Control de acceso por rol

Tabla 44. Especificación del caso de prueba E-06.

Objetivo	Verificar que un usuario no puede acceder a operaciones reservadas a un rol superior.
Requisitos	RF-03, RNF-03, RNF-04.
Precondiciones	Un usuario con rol ciudadano y sesión activa.
Pasos	1) El ciudadano intenta acceder a un endpoint de gestión de puestos reservado a coordinación.
Resultado esperado	La API rechaza la petición con código 403 y el cliente redirige a una vista permitida sin exponer datos.
Resultado obtenido	PASS. La API responde 403; el cliente redirige a la vista del rol sin filtrar información.

Fuente: Elaboración propia.

Anexo H. Cobertura de los objetivos específicos

Para cerrar el círculo entre la planificación y los resultados, este anexo relaciona cada objetivo específico (sección 3.2) con su grado de consecución y con la evidencia que lo respalda en la memoria. La Tabla 45 muestra que todos los objetivos específicos se han alcanzado dentro del alcance del MVP, con la salvedad explícita de la validación en una emergencia real, reservada como trabajo futuro.

Tabla 45. Cobertura de los objetivos específicos, grado de consecución y evidencia.

Objetivo específico	Estado	Evidencia en la memoria
OE-1. Analizar limitaciones y principios de diseño	Alcanzado	Capítulo 2; comparativa (Tabla 2).
OE-2. Modelar los cuatro actores	Alcanzado	Sección 4.2; figuras 1-2; matriz de permisos (Tabla 22).
OE-3. Reporte colaborativo de incidencias	Alcanzado	Sección 4.9; figura 9; pruebas E-01.
OE-4. Inventario dual por puesto	Alcanzado	Sección 4.10; RF-13, RF-14.
OE-5. Trazabilidad de donaciones por QR	Alcanzado	Sección 4.10; figuras 10-11; pruebas E-03.
OE-6. Enrutamiento consciente de vías cortadas	Alcanzado	Sección 4.9; banco de pruebas (sección 5.6).
OE-7. Arquitectura Offline-First con sincronización fiable	Alcanzado	Sección 4.6; pseudocódigo; pruebas E-02.
OE-8. Autenticación y RBAC	Alcanzado	Sección 4.8; pruebas E-06.
OE-9. Registro verificable encadenado	Alcanzado	Sección 4.13; Anexo D (auditoría pública).
OE-10. Comunicación en tiempo real	Alcanzado	Sección 4.12; figura 14; pruebas E-01.
OE-11. Evaluación del sistema	Alcanzado (simulado)	Capítulo 5; validación real reservada a trabajo futuro.

Fuente: *Elaboración propia.*

Anexo I. Estructura del repositorio y guía de lectura del código

El código fuente completo del proyecto está disponible públicamente en GitHub: github.com/franciiscocg/Catastrofe-logistica.

Este anexo orienta la lectura del código fuente del proyecto, relacionando las principales zonas del repositorio con su responsabilidad y con el valor que aportan al trabajo. Su objetivo es que un tercero —en particular, el tribunal— pueda navegar el código de forma eficiente y contrastar las afirmaciones de la memoria con su implementación real.

El repositorio se organiza de forma coherente con un monolito modular en el backend y una PWA estructurada por funcionalidades en el frontend. En el backend, cada dominio reside en su propio módulo (backend/src/modules) con la tríada router-servicio-esquema, mientras que la infraestructura transversal (autenticación, tiempo real, cadena de eventos, seguridad) se concentra en backend/src/lib y backend/src/middleware. En el frontend, cada rol y funcionalidad se agrupa en frontend/src/features, la persistencia offline en frontend/src/lib/db y la lógica de rutas y geometría en frontend/src/utills. El directorio TEST agrupa las suites por funcionalidad, en paralelo a la estructura del producto.

Tabla 46. Estructura del repositorio: zonas principales, responsabilidad y valor para el trabajo.

Zona del repositorio	Responsabilidad principal	Valor para el TFM
backend/src/modules/auth	Registro, login, refresh, logout, recuperación y código de recuperación.	Demuestra gestión de identidad y sesión con refresh token persistente.
backend/src/modules/puestos	Alta, aprobación, rechazo, detalle de puestos, participantes y trabajadores.	Núcleo de coordinación logística y control operativo.
backend/src/modules/inventario	Inventario dual, historial, QR de confirmación y movimientos.	Soporta trazabilidad de recursos físicos.
backend/src/modules/donaciones	Necesidades, donaciones, estados y código de entrega.	Conecta voluntarios con necesidades reales de los puestos.
backend/src/modules/incidencias	Incidencias, comentarios, asignaciones y control de estado.	Alimenta el mapa y el cálculo de rutas seguras.
backend/src/modules/audit	Consulta pública de eventos encadenados y verificación.	Aporta transparencia y verificabilidad externa.
frontend/src/features	Vistas por rol: auth, ciudadano, voluntario, puesto, coordinador, perfil y verificación pública.	Materializa la experiencia de usuario diferenciada.
frontend/src/lib/db	IndexedDB, caché API, snapshots públicos, grafos OSM e idMappings.	Base de la operación offline-first.
frontend/src/utills	Geometría, routing, navegación, grafo OSM y almacenamiento.	Aporta el valor diferencial de rutas conscientes de cortes.
TEST	Suites unitarias, integración y E2E por funcionalidad.	Evidencia de validación técnica del MVP.

Fuente: *Elaboración propia*.

Para la defensa del trabajo, las evidencias más eficaces que conviene mostrar directamente sobre el código son: el esquema Prisma (backend/prisma/schema.prisma), que confirma el modelo de datos; los routers del backend, que materializan los contratos de la API; la

configuración de Workbox y el manifest (`frontend/vite.config.ts`), que sustentan el comportamiento como PWA; la base de datos local con Dexie y la cola de sincronización (`frontend/src/lib/db` y `store` de sincronización), que evidencian la operación offline; el motor de rutas y el banco de pruebas (`frontend/src/utils` y `frontend/benchmark`); y la implementación de la cadena de eventos (`backend/src/lib/chain.ts`) con su endpoint público de verificación. Mostrar estas piezas, y no solo capturas de interfaz, es lo que demuestra la profundidad real de la implementación.

Anexo J. Capturas de la interfaz de usuario

Este anexo recopila capturas reales de la interfaz de la aplicación Catástrofe-Logística, organizadas por flujo de acceso y por rol de usuario. Las pantallas ilustran cómo los requisitos funcionales descritos en el capítulo 4 se materializan en una experiencia de usuario coherente, responsiva y operable en condiciones de campo. La interfaz se ha construido como Progressive Web App con un diseño limpio y orientado a la acción, de modo que cada rol —ciudadano, voluntario, puesto de emergencia y coordinador— accede únicamente a las funciones que le corresponden mediante el control de acceso basado en roles.

J.1. Acceso y gestión de identidad

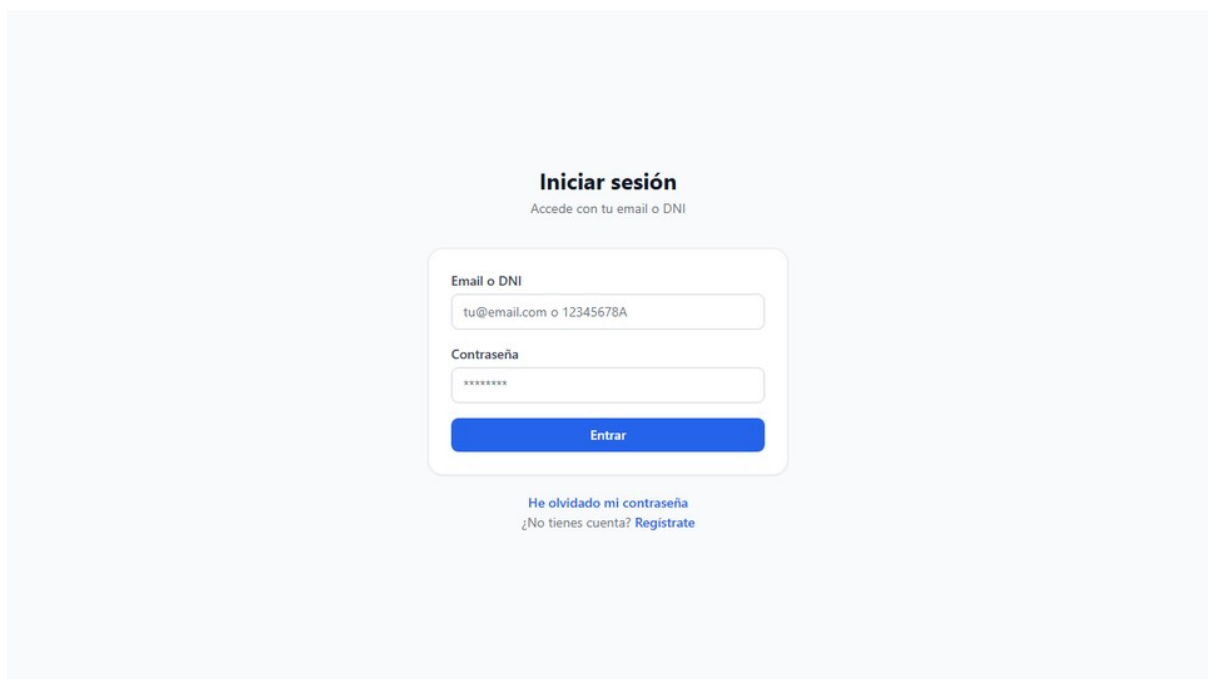


Figura 23. Pantalla de inicio de sesión, con acceso mediante correo electrónico o DNI/NIE.

Fuente: Captura de la aplicación desarrollada.

The screenshot shows a registration form titled "Crear cuenta" with the subtitle "Completa tus datos para registrarte". The form is enclosed in a rounded rectangle and contains the following fields: "Nombre" (Name) with the value "María", "Apellidos" (Surnames) with the value "García López", "Correo electrónico" (Email) with the value "tu@email.com", "DNI / NIE" with the value "12345678A", and "Contraseña" (Password) with the placeholder "Mínimo 8 caracteres". A blue button labeled "Crear cuenta" is at the bottom of the form. Below the form, there is a link: "¿Ya tienes cuenta? [Inicia sesión](#)".

Figura 24. Registro de una cuenta de ciudadano o voluntario.
Fuente: Captura de la aplicación desarrollada.

The screenshot shows a login form titled "Iniciar sesión" with the subtitle "Para acceder como **Puesto de Emergencia**". The form is enclosed in a rounded rectangle and contains the following fields: "Email o DNI" with the value "tu@email.com o 12345678A" and "Contraseña" (Password) with masked characters "xxxxxxx". A blue button labeled "Entrar" is at the bottom of the form. Below the form, there are two links: "[He olvidado mi contraseña](#)" and "[¿No tienes cuenta? Regístrate](#)".

Figura 25. Solicitud de alta de un puesto de emergencia.
Fuente: Captura de la aplicación desarrollada.



Recuperar contraseña

Confirma los datos de tu cuenta y elige una contraseña nueva.

Correo electrónico

DNI/NIE

Código de recuperación

Nueva contraseña

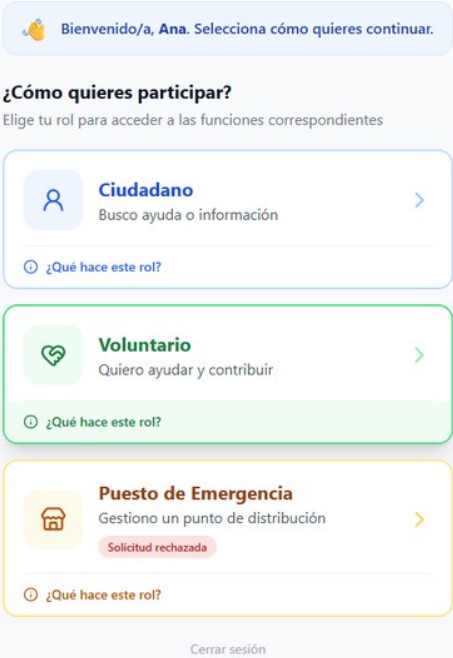
Repite la nueva contraseña

Actualizar contraseña

[Volver al login](#)

Figura 26. Recuperación de contraseña mediante DNI/NIE y código de recuperación, sin dependencia de correo.

Fuente: Captura de la aplicación desarrollada.



 Bienvenido/a, Ana. Selecciona cómo quieres continuar.

¿Cómo quieres participar?

Elige tu rol para acceder a las funciones correspondientes

 **Ciudadano** >
Busco ayuda o información
[¿Qué hace este rol?](#)

 **Voluntario** >
Quiero ayudar y contribuir
[¿Qué hace este rol?](#)

 **Puesto de Emergencia** >
Gestiono un punto de distribución
Solicitud rechazada
[¿Qué hace este rol?](#)

[Cerrar sesión](#)

Figura 27. Selección del rol activo para usuarios con varios perfiles.

Fuente: Captura de la aplicación desarrollada.

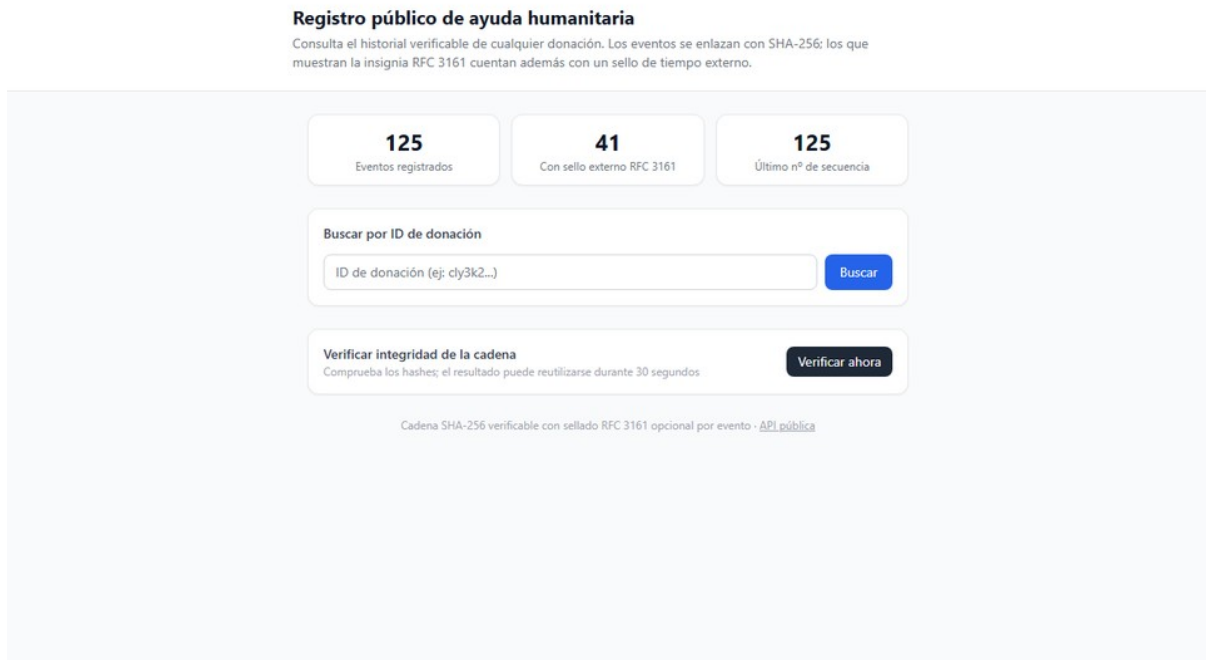


Figura 28. Verificación pública de un código QR de entrega.

Fuente: Captura de la aplicación desarrollada.

J.2. Rol Ciudadano

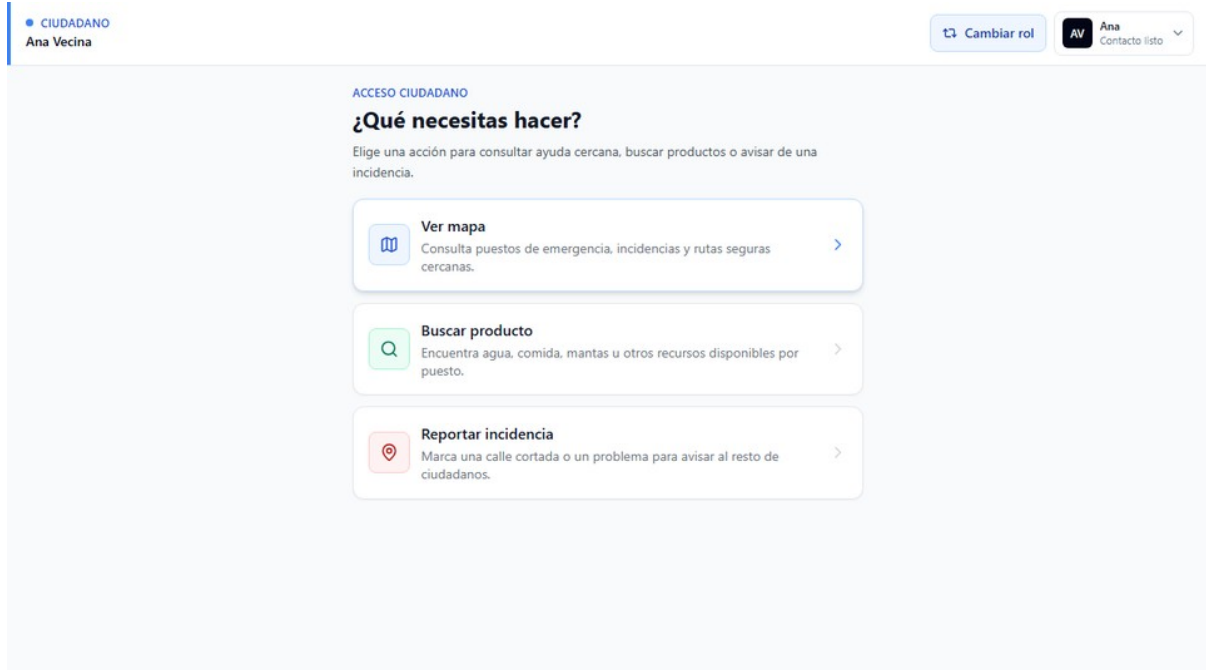


Figura 29. Pantalla de inicio del rol ciudadano.

Fuente: Captura de la aplicación desarrollada.

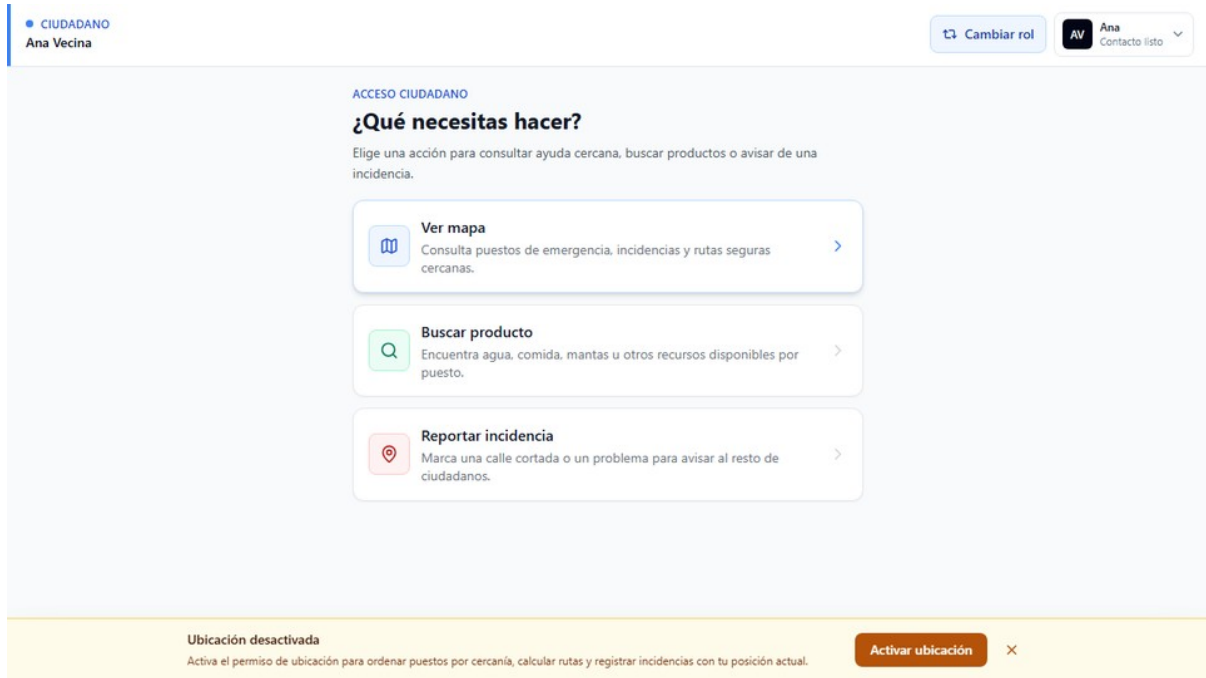


Figura 30. Panel principal del ciudadano con accesos a las funciones disponibles.

Fuente: Captura de la aplicación desarrollada.

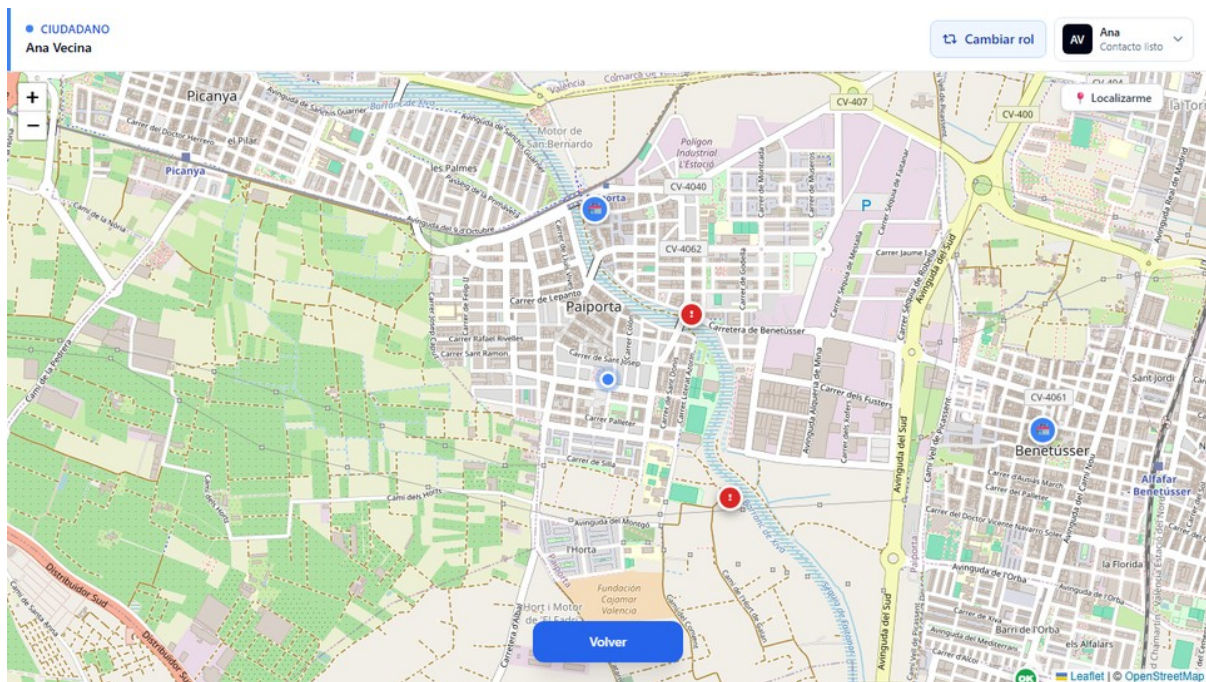


Figura 31. Mapa operativo con puestos e incidencias sobre Leaflet y OpenStreetMap.

Fuente: Captura de la aplicación desarrollada.

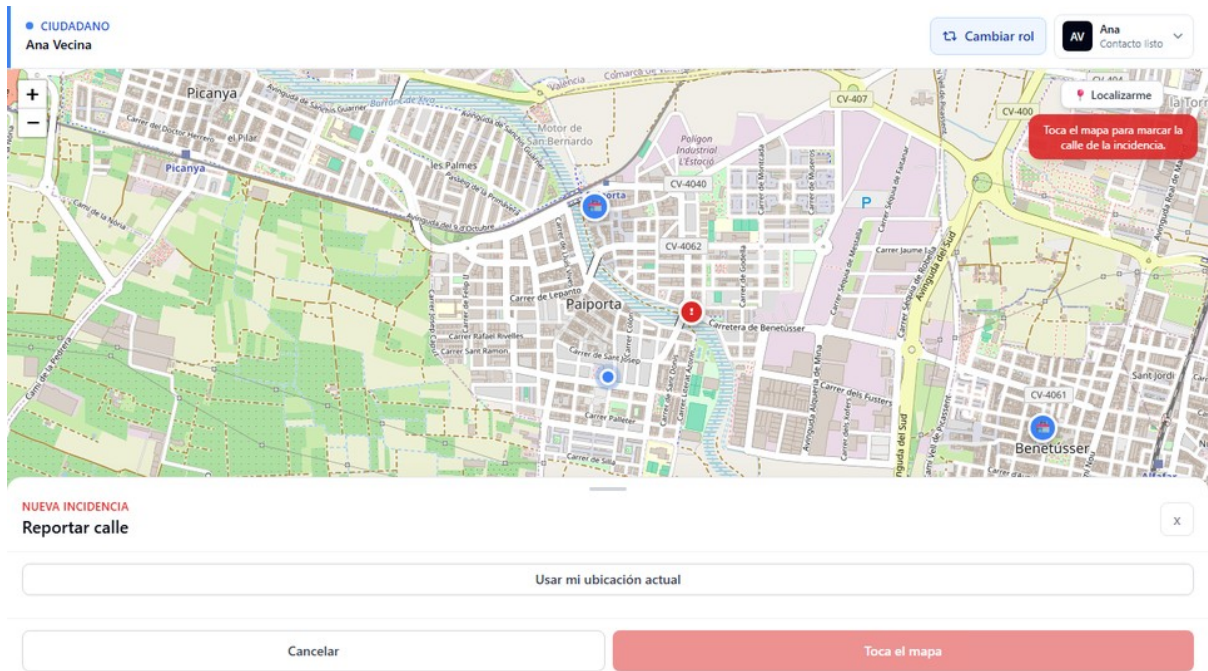


Figura 32. Reporte de una incidencia en la vía con categoría y ubicación.

Fuente: Captura de la aplicación desarrollada.

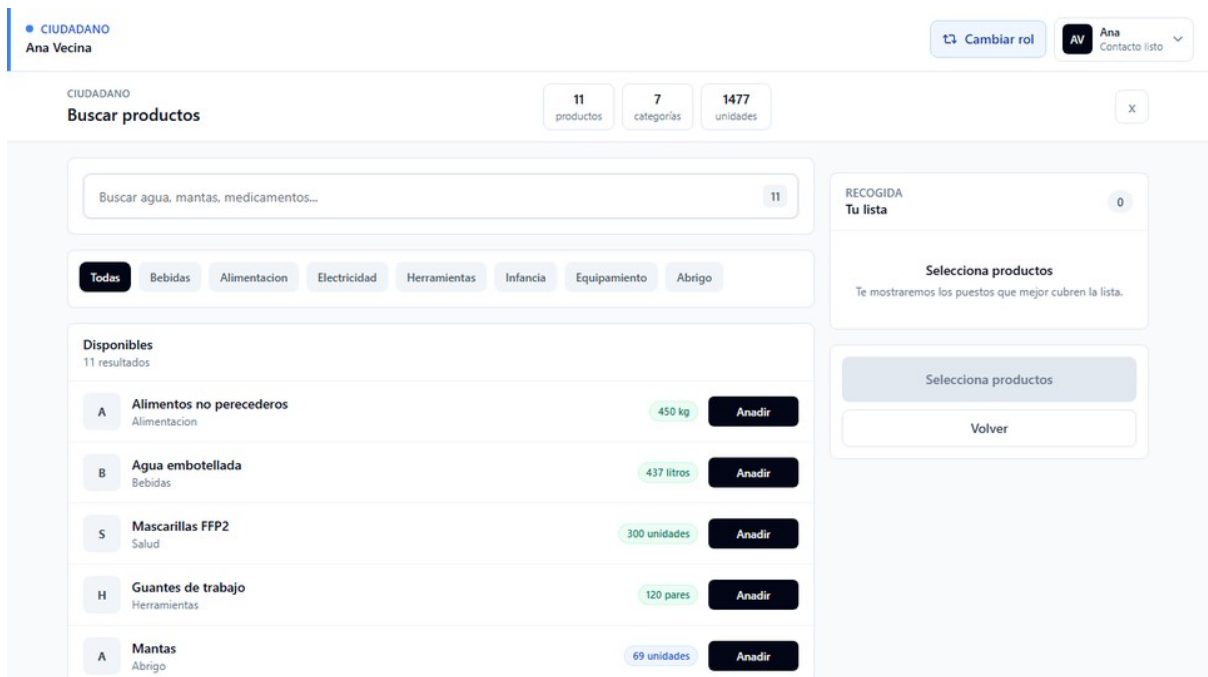


Figura 33. Búsqueda de productos disponibles en los puestos cercanos.

Fuente: Captura de la aplicación desarrollada.

CIUDADANO
Ana Vecina

Cambiar rol AV Ana Contacto listo

CIUDADANO
Buscar productos

11 productos 7 categorías 1477 unidades X

agua X 1

Todas Bebidas Alimentacion Electricidad Herramientas Infancia Equipamiento Abrigo

Disponibles
1 resultado Limpiar

B Agua embotellada Bebidas 437 litros Anadir

RECOGIDA
Tu lista 0

Selecciona productos
Te mostraremos los puestos que mejor cubren la lista.

Selecciona productos

Volver

Figura 34. Resultado de la búsqueda de un producto (agua) ordenado por cercanía.

Fuente: Captura de la aplicación desarrollada.

CIUDADANO
Ana Vecina

Cambiar rol AV Ana Contacto listo

PERFIL OPERATIVO

Datos de intervención

Información mínima para identificarte, contactarte contigo y asignarte tareas durante una emergencia.

Volver Perfil listo

IDENTIDAD Alta: 27 de mayo de 2026

Contacto principal

Nombre Ana Apellidos Vecina

Teléfono operativo +34600000004

Guardar contacto

CAPACIDAD

Disponibilidad para apoyo

Modalidad preferida Mixta

Vehículo disponible

ESTADO Perfil listo 3/3

Identidad Ana Vecina

Teléfono +34600000004

Vehículo No disponible

ROLES CIUDADANO VOLUNTARIO

SEGURIDAD
Genera un código secreto para recuperar la contraseña sin correo.

Generar código de recuperación

Figura 35. Pantalla de perfil de usuario.

Fuente: Captura de la aplicación desarrollada.

J.3. Rol Voluntario

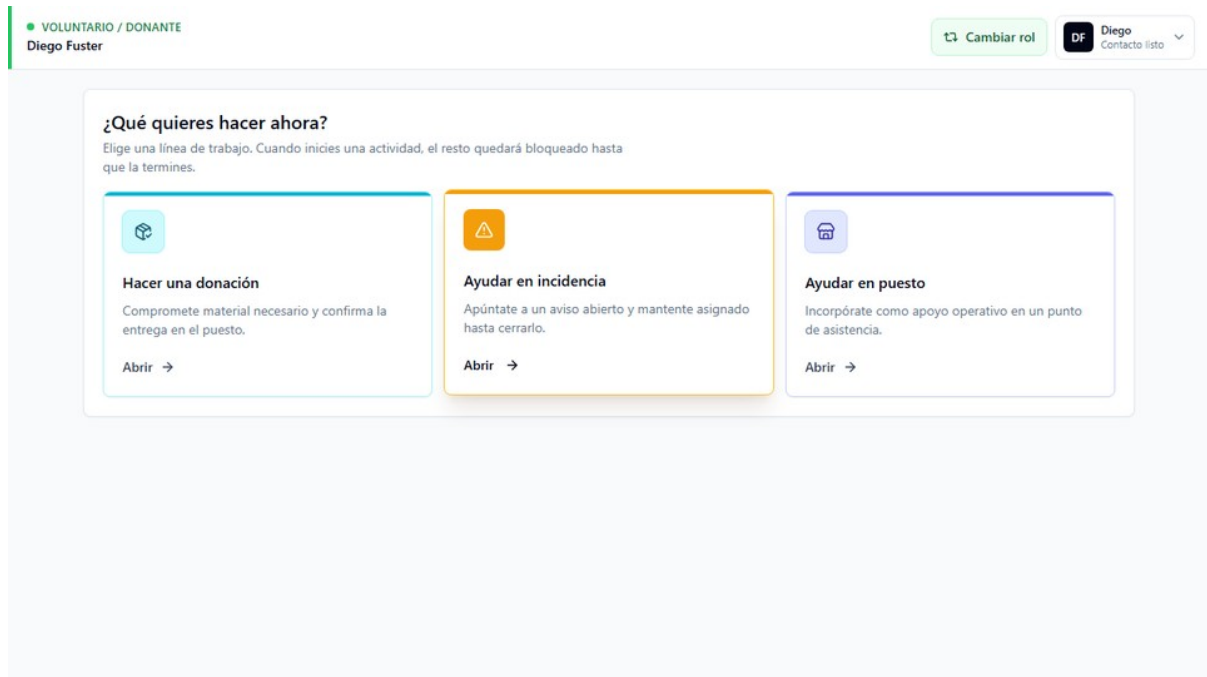


Figura 36. Panel principal del voluntario.
Fuente: Captura de la aplicación desarrollada.

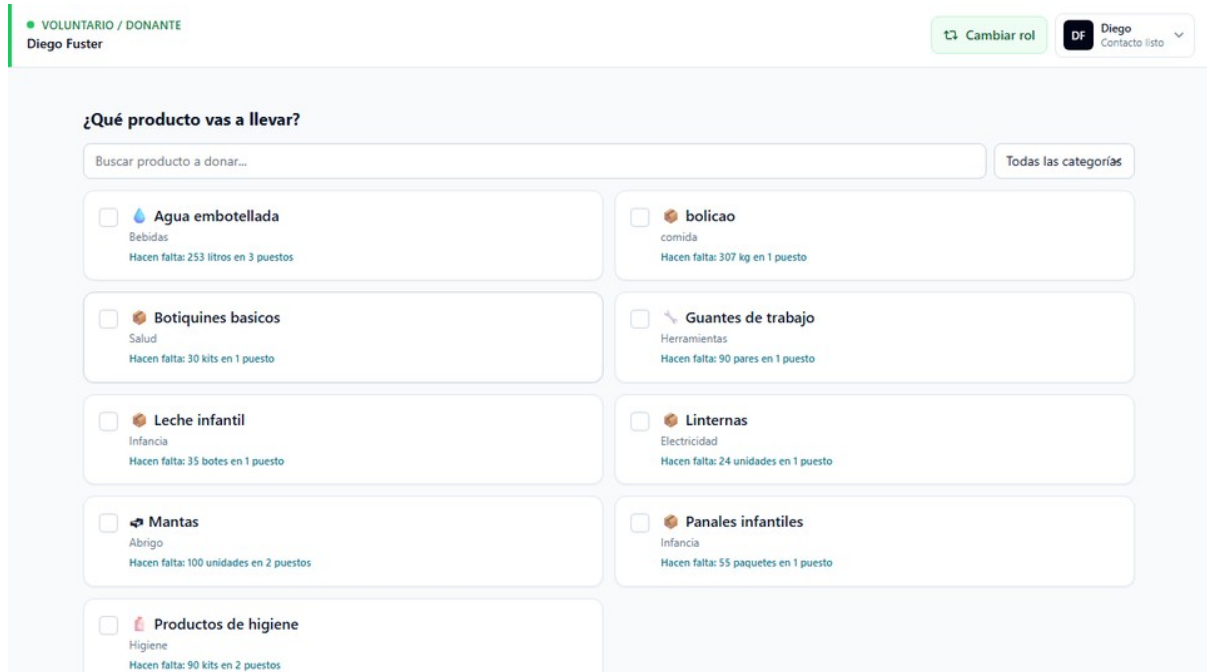


Figura 37. Acción de donación a un puesto de emergencia.
Fuente: Captura de la aplicación desarrollada.

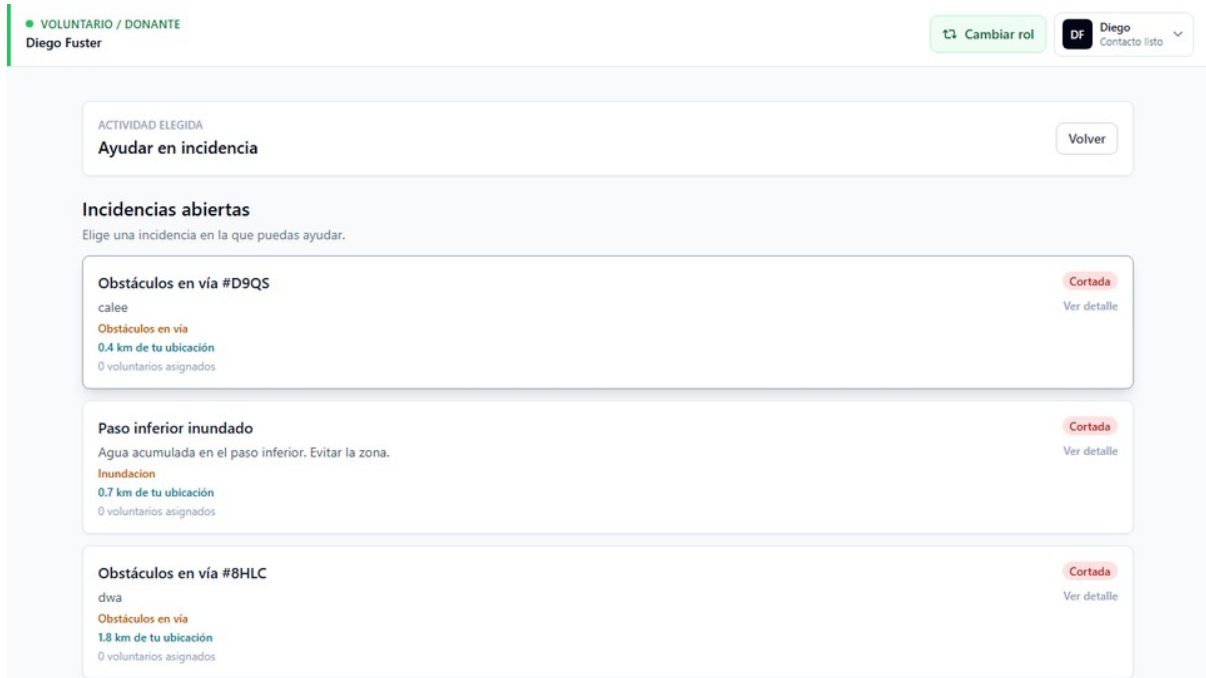


Figura 38. Acción de atención de una incidencia.

Fuente: Captura de la aplicación desarrollada.

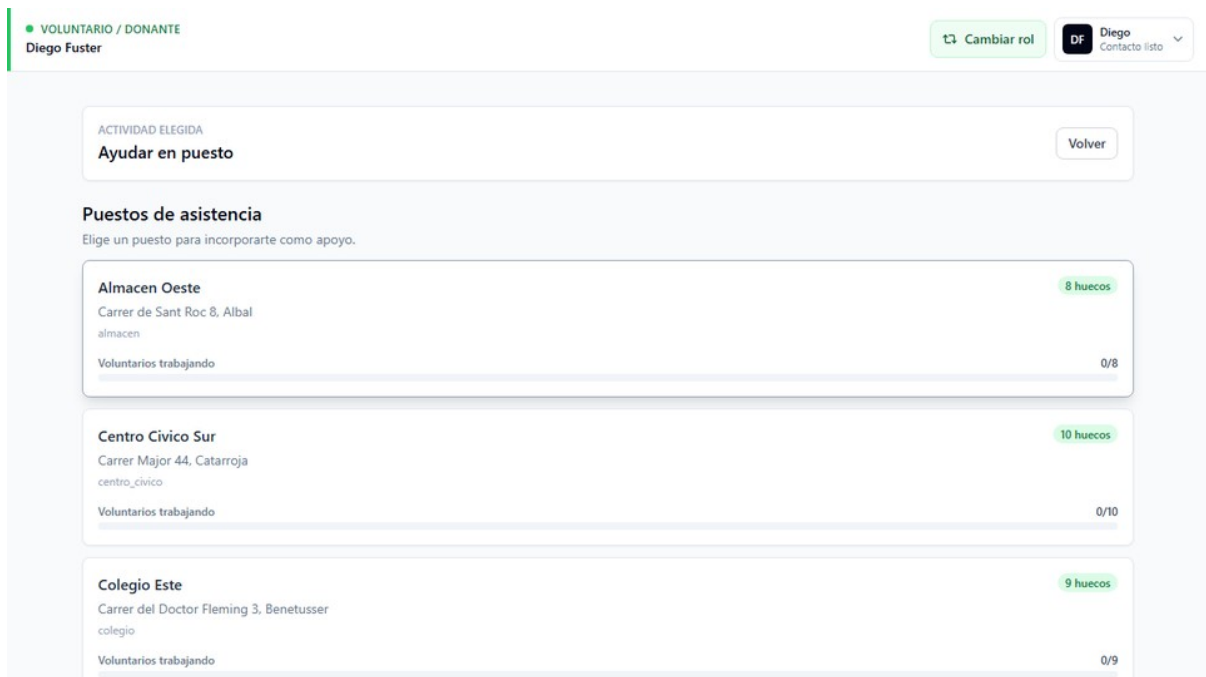


Figura 39. Acción de solicitud de participación en un puesto.

Fuente: Captura de la aplicación desarrollada.

VOLUNTARIO / DONANTE
Diego Fuster

[Cambiar rol](#) **DF** Diego
Contacto listo

¿Qué producto vas a llevar?

Buscar producto a donar... [Todas las categorías](#)

☐ Agua embotellada Bebidas Hacen falta: 253 litros en 3 puestos	☐ bolicao comida Hacen falta: 307 kg en 1 puesto
☐ Botiquines basicos Salud Hacen falta: 30 kits en 1 puesto	☐ Guantes de trabajo Herramientas Hacen falta: 90 pares en 1 puesto
☐ Leche infantil Infancia Hacen falta: 35 botes en 1 puesto	☐ Linternas Electricidad Hacen falta: 24 unidades en 1 puesto
☐ Mantas Abrigo Hacen falta: 100 unidades en 2 puestos	☐ Panales infantiles Infancia Hacen falta: 55 paquetes en 1 puesto
☐ Productos de higiene Higiene Hacen falta: 90 kits en 2 puestos	

Figura 40. Flujo de donación (1): selección de los productos a donar.

Fuente: Captura de la aplicación desarrollada.

VOLUNTARIO / DONANTE
Diego Fuster

[Cambiar rol](#) **DF** Diego
Contacto listo

¿Qué cantidad vas a llevar de cada uno?

Agua embotellada
Unidad: litros

Cantidad litros

[Atrás](#) [Calcular Ruta Óptima](#)

Figura 41. Flujo de donación (2): especificación de las cantidades.

Fuente: Captura de la aplicación desarrollada.

● VOLUNTARIO / DONANTE
Diego Fuster

[Cambiar rol](#) **DF** Diego Fuster
Contacto listo

No hay una ruta segura que evite todas las incidencias cortadas reportadas.

¿Qué cantidad vas a llevar de cada uno?

🚰 Agua embotellada
Unidad: litros

5 litros

[Atrás](#) [Calcular Ruta Óptima](#)

Figura 42. Flujo de donación (3): cálculo de la ruta óptima evitando las vías cortadas.

Fuente: Captura de la aplicación desarrollada.

J.4. Rol Puesto de emergencia

● PUESTO DE EMERGENCIA
Marta Responsable Norte

MR Marta
Contacto listo

PUESTO DE EMERGENCIA Operativo

Pabellon Norte
Avenida de la Constitucion 12, Paiporta

2 Faltan
productos abiertos

0 Criticos
stock bajo

0 Entregas
en seguimiento

1 Voluntarios
1 esperando

[Escanear QR](#) [Nuevo producto](#) [Voluntarios](#) [Historial](#) [Todo](#) [Stock](#) [Faltan](#)

PRIORIDAD AHORA
Gestión operativa

Necesidades abiertas [Ver](#)

Mantas
Abrigo 70 unidades

Productos de higiene
Higiene 45 kits

Stock bajo [Ver](#)

Sin productos en nivel crítico.

1 Solicitudes 1 Voluntarios

RECEPCIÓN
Entregas activas 0
Sin entradas pendientes

3 productos faltan por comenzar stock y necesidades

Figura 43. Panel del puesto de emergencia con el inventario dual (disponible y necesario).

Fuente: Captura de la aplicación desarrollada.

● PUESTO DE EMERGENCIA
Marta Responsable Norte

PUESTO DE EMERGENCIA **Operativo**

Pabellon Norte
Avinguda de la Constitucio 12, Paiporta

2 Faltan productos abiertos 0 Criticos stock bajo 0 Entregas en seguimiento 1 Voluntarios 1 esperando

Escanear QR Nuevo producto Voluntarios Historial Buscar por producto o categoría Todo Stock Faltan

PRIORIDAD AHORA RECEPCIÓN 0

Añadir producto ✕

Productos frecuentes:

Agua embotellada Alimentos no perecederos Mantas Medicamentos básicos Ropa de abrigo Pañales Linternas y pilas Productos de higiene

Nombre *
Nombre del producto

Categoría * Unidad *
Bebidas, Ropa... litros, kg, unidades...

Cantidad * Tipo
0 Disponible

Añadir al inventario

Figura 44. Alta de un nuevo producto en el inventario del puesto.

Fuente: Captura de la aplicación desarrollada.

● PUESTO DE EMERGENCIA
Marta Responsable Norte

PUESTO DE EMERGENCIA **Operativo**

Pabellon Norte
Avinguda de la Constitucio 12, Paiporta

2 Faltan productos abiertos 0 Criticos stock bajo 0 Entregas en seguimiento 1 Voluntarios 1 esperando

Escanear QR Nuevo producto Voluntarios Historial Buscar por producto o categoría Todo Stock Faltan

VOLUNTARIOS

Solicitudes y personas activas ✕

Revisa peticiones de apoyo y consulta quién está colaborando ahora.

Solicitudes pendientes 1

Carmen Mora
DNI: 10000005F
voluntaria.rutas@demo.local **Pendiente**

Aceptar **Rechazar**

Voluntarios activos 1

Carmen Mora
DNI: 10000005F
Teléfono: +34600000006 **Quitar**

Figura 45. Gestión de los voluntarios asignados al puesto.

Fuente: Captura de la aplicación desarrollada.

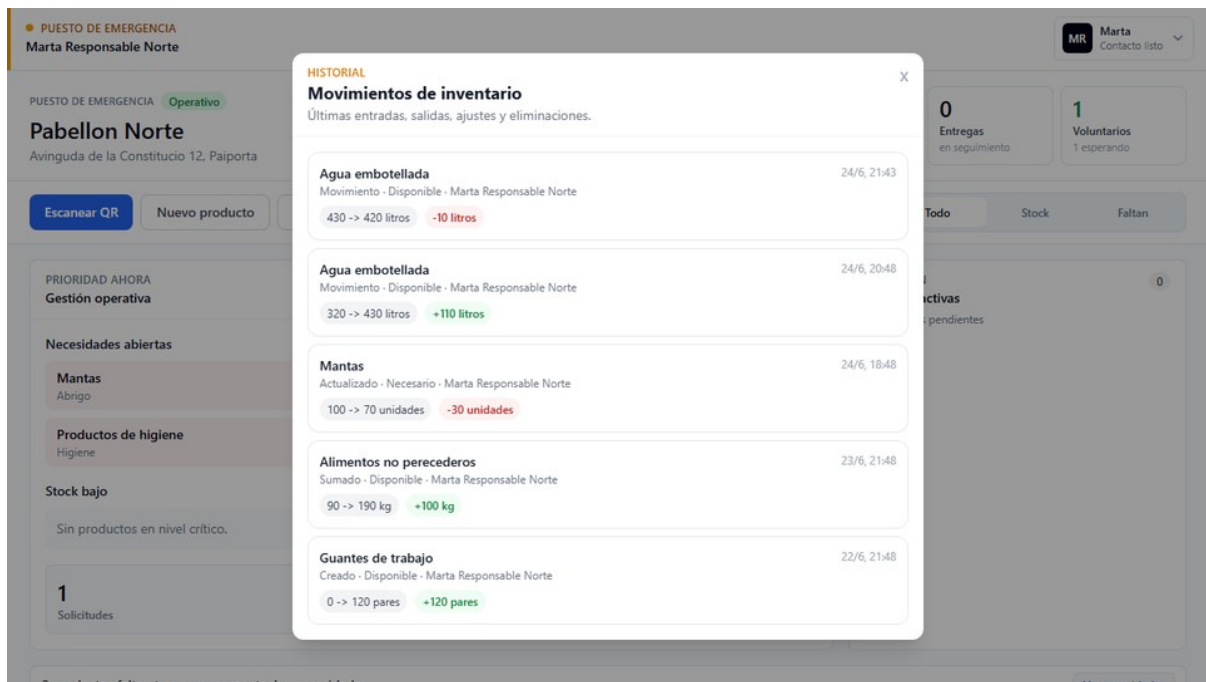


Figura 46. Historial de movimientos del inventario.

Fuente: Captura de la aplicación desarrollada.

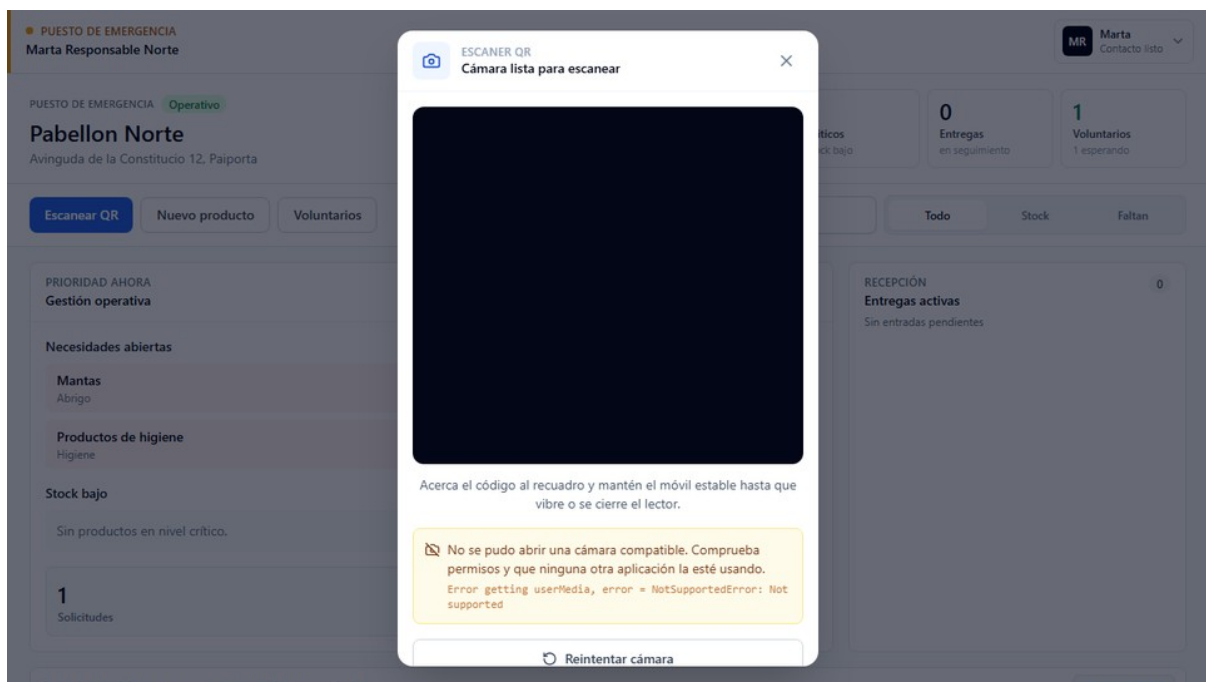


Figura 47. Generación del código QR de entrega para la recepción de una donación.

Fuente: Captura de la aplicación desarrollada.

J.5. Rol Coordinador

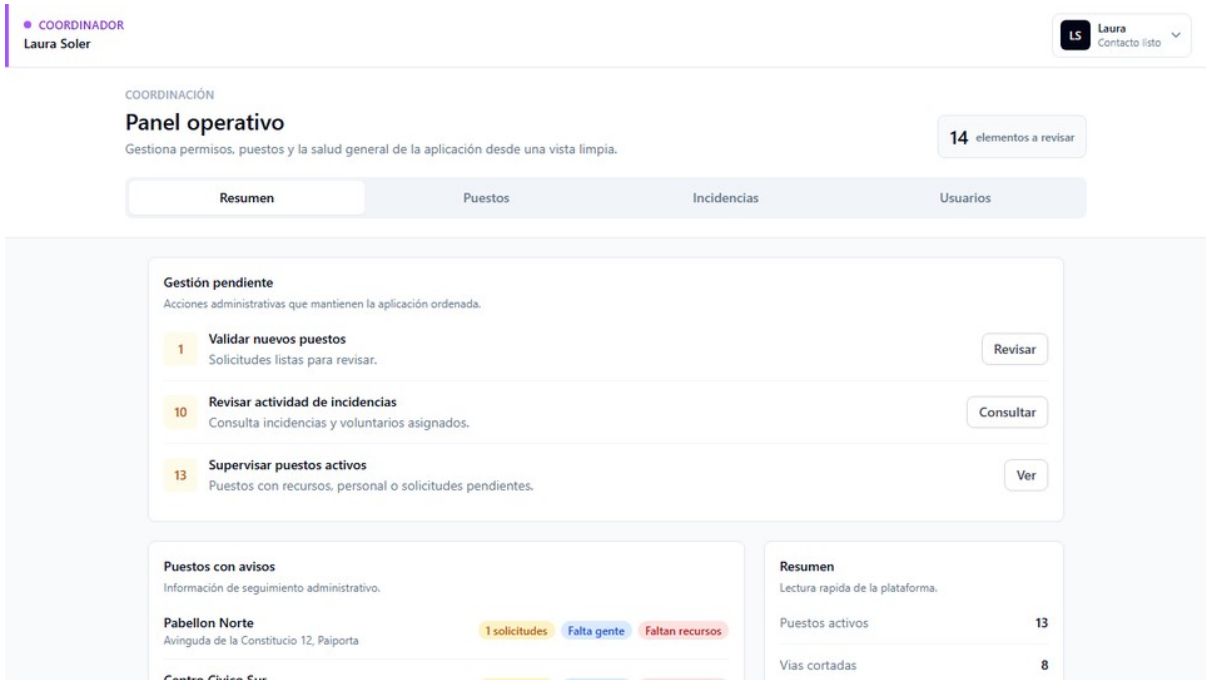


Figura 48. Panel de supervisión global del coordinador.
Fuente: Captura de la aplicación desarrollada.

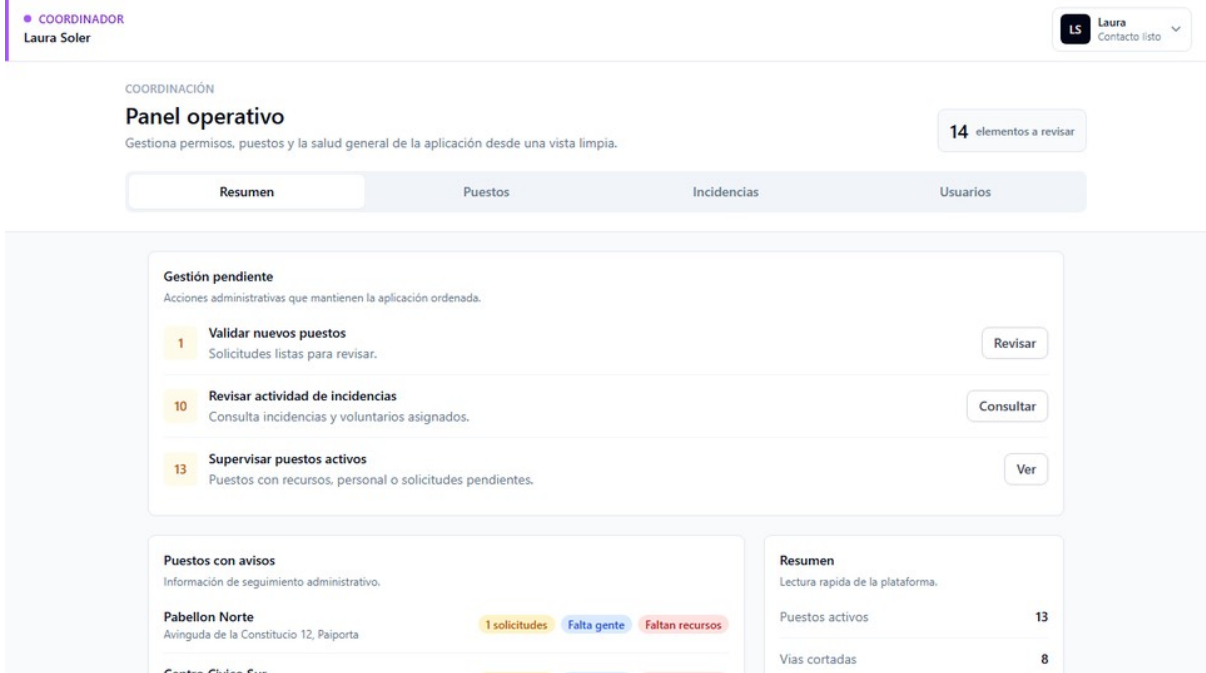


Figura 49. Pestaña de resumen operativo de la red de puestos.
Fuente: Captura de la aplicación desarrollada.

COORDINACIÓN

Panel operativo

Gestiona permisos, puestos y la salud general de la aplicación desde una vista limpia.

14 elementos a revisar

Resumen

Puestos

Incidencias

Usuarios

Puestos de emergencia

Gestiona los puestos aprobados, tramita solicitudes y consulta el historial.

Buscar por nombre, dirección, tipo o admin

Aprobados

Solicitudes pendientes

Historial de rechazados

Pabellon Norte**Aprobado****Necesita voluntarios****Necesita recursos**

Ver

Editar

Avenida de la Constitucion 12, Paiporta

TIPO

Pabellón

PERSONAS

2

VOLUNTARIOS

1/12

NECESIDADES

2

Admin: Marta Responsable Norte - puesto.norte@demo.local

Ubicación: 39.4314, -0.4184

1 solicitudes de participación pendientes**Figura 50.** Pestaña de gestión y aprobación de puestos.

Fuente: Captura de la aplicación desarrollada.

COORDINACIÓN

Panel operativo

Gestiona permisos, puestos y la salud general de la aplicación desde una vista limpia.

14 elementos a revisar

Resumen

Puestos**Incidencias**

Usuarios

Incidencias registradas

Consulta el estado de las incidencias y los voluntarios que se han unido.

Buscar por título, categoría o descripción

Todas

Cortadas

Transitables

Sin voluntarios

Puente con barro denso**CORTADA**

Ver voluntarios

Eliminar

Acumulacion de lodo. Necesita limpieza con palas y EPIs.

39.4081, -0.4244 - 0 voluntarios activos

Actualizada: 24 jun 2026, 21:22

Ruta alternativa habilitada**TRANSITABLE**

Ver voluntarios

Eliminar

Paso lento para vehiculos pequenos por la via de servicio.

39.4173, -0.3881 - 0 voluntarios activos

Actualizada: 24 jun 2026, 21:22

Figura 51. Pestaña de gestión de incidencias.

Fuente: Captura de la aplicación desarrollada.

COORDINADOR
Laura Soler

LS Laura
Contacto listo

COORDINACIÓN

Panel operativo

Gestiona permisos, puestos y la salud general de la aplicación desde una vista limpia.

14 elementos a revisar

Resumen

Puestos

Incidencias

Usuarios

Usuarios y roles

Consulta las cuentas y guarda los cambios de permisos de forma controlada.

Buscar por nombre o email

TodosCiudadanoVoluntarioPuestoCoordinador

Elena Domínguez

donante@demo.local

Roles: CIUDADANO, VOLUNTARIO

Editar roles

Eliminar

Sergio Benavent

vecino.catarroja@demo.local

Roles: CIUDADANO

Editar roles

Eliminar

Figura 52. Pestaña de gestión de usuarios.
Fuente: Captura de la aplicación desarrollada.